Manuel du stagiaire — Virtualisation des réseaux, SDN et OpenFlow

Oct 16, 2026 · @Tahar

Ce manuel accompagne la formation de 12 heures (réf. 1-041) et va plus loin qu'elle. Il reprend chaque notion vue en séance, l'explique en détail et la relie aux outils que les équipes d'administration système et DevOps exploitent déjà : Linux, conteneurs, Kubernetes, OpenStack.

Avant-propos

Ce manuel est votre référence après la formation : chaque notion citée en séance y est développée, illustrée et reliée aux travaux pratiques. Il se lit dans l'ordre, mais chaque chapitre peut aussi être consulté seul.

À qui s'adresse ce manuel

Il s'adresse aux administrateurs système et aux ingénieurs DevOps qui exploitent des plateformes virtualisées : hyperviseurs, conteneurs, clusters Kubernetes ou OpenShift, clouds OpenStack. Il suppose deux prérequis :

- les bases des réseaux locaux et étendus (adressage IP, commutation, routage), résumées au chapitre 1 ;
- l'aisance en ligne de commande Linux.

Aucune expérience préalable du SDN ou d'OpenFlow n'est nécessaire.

Comment l'utiliser

1. **Avant la session** : relisez le chapitre 1 si les notions de trame, de VLAN ou de table de routage vous semblent lointaines.
2. **Pendant la session** : le formateur renvoie aux chapitres ; les encadrés « En TP » indiquent où chaque notion est pratiquée.
3. **Après la session** : l'annexe A propose des questions de révision corrigées, l'annexe B un glossaire, l'annexe C les normes et lectures pour aller plus loin. L'onglet « Cahier de TP » reprend les énoncés des manipulations, à refaire sur votre propre machine.

Les énoncés des travaux pratiques sont réunis dans l'onglet Cahier de TP.

Correspondance avec la formation

Module de la formation	Durée en séance	Chapitres du manuel
Prérequis	—	1
M1 — La virtualisation des réseaux	2 h	2, 3, 4
M2 — L'architecture des réseaux et sa normalisation	4 h	5, 6, 7, 8
M3 — Les techniques et applications du SDN	3 h	9 à 13

Module de la formation	Durée en séance	Chapitres du manuel
M4 — Les réseaux OpenFlow	3 h	14 à 18
Révision et approfondissement	—	Annexes A, B, C

Le manuel va volontairement plus loin que les 12 heures de formation. Les sections marquées « Pour aller plus loin » ne sont pas traitées en séance et ne sont pas évaluées.

Conventions

- Les commandes, fichiers et champs de protocole sont en `police fixe`. Les commandes qui exigent les droits d'administration commencent par `sudo`.
- Les invites `mininet>` et `onos>` indiquent la console dans laquelle taper la commande.
- Les termes anglais consacrés (switch, overlay, underlay, flow) sont conservés et traduits à leur première apparition ; tous figurent au glossaire.
- Quatre encadrés rythment le texte : **À retenir** (l'essentiel), **En TP** (lien avec la pratique), **Attention** (piège courant), **Pour aller plus loin** (approfondissement facultatif).
- Toutes les adresses sont fictives et privées (plages RFC 1918) ; elles ne correspondent à aucun réseau réel.

La plateforme fil rouge LAB-SDML

Les exemples et les TP s'appuient sur une plateforme fictive de quatre zones, que chaque module enrichit d'une couche : réseau virtuel (M1), segmentation et overlay (M2), contrôleur et politique (M3), règles OpenFlow (M4).

Zone	VLAN	Sous-réseau	Rôle
dev	10	10.10.10.0/24	Postes et runners de développement
recette	20	10.10.20.0/24	Environnement de qualification
prod	30	10.10.30.0/24	Services exposés
admin	99	10.10.99.0/24	Administration et supervision

Logiciels de référence

Les exemples ont été écrits pour Ubuntu Server 24.04 LTS, Open vSwitch 3.x, Mininet 2.3, le contrôleur ONOS 2.7.0 et le framework Python os-ken 4.x. Ils fonctionnent avec peu ou pas de modification sur Debian 12 ou 13 et sur RHEL 9.

Chapitre 1 — Rappels essentiels

Tout réseau virtuel repose sur les mécanismes d'un réseau physique : trames, adresses MAC, commutation, adresses IP et routage. Ce chapitre les résume ; les chapitres suivants les supposent connus.

1.1 Les modèles en couches

Un réseau se décrit en couches : chacune rend un service à la couche du dessus et s'appuie sur celle du dessous. Le modèle OSI (ISO 7498) en compte sept ; le modèle TCP/IP, celui d'Internet, en regroupe l'essentiel en quatre.

Couche OSI	Couche TCP/IP	Unité de données	Exemples	Équipement typique
7 Application, 6 Présentation, 5 Session	Application	Message	HTTP, DNS, SSH, TLS	Serveur, répartiteur L7
4 Transport	Transport	Segment (TCP), datagramme (UDP)	TCP, UDP	Pare-feu à états
3 Réseau	Internet	Paquet	IPv4, IPv6, ICMP	Routeur
2 Liaison	Accès réseau	Trame	Ethernet, 802.1Q, Wi-Fi	Switch
1 Physique	Accès réseau	Bit	Cuivre, fibre, radio	Carte réseau, câble

Encapsulation. En descendant les couches, chaque niveau ajoute son en-tête devant les données reçues : une requête HTTP devient un segment TCP, puis un paquet IP, puis une trame Ethernet. Le récepteur retire les en-têtes dans l'ordre inverse. Toute la virtualisation réseau joue sur ce principe : un tunnel n'est qu'une encapsulation supplémentaire, une trame entière placée dans un nouveau paquet (chapitre 6).

1.2 Ethernet et les adresses MAC

Une trame Ethernet transporte de 46 à 1 500 octets de données utiles.

Champ	Taille	Rôle
Adresse MAC de destination	6 octets	Destinataire sur le segment
Adresse MAC source	6 octets	Émetteur
EtherType	2 octets	Protocole transporté
Données	46 à 1 500 octets	Paquet IP, ARP…
FCS	4 octets	Contrôle d'intégrité (CRC)

Une adresse MAC fait 48 bits, notés `00:00:00:00:00:01`. Les 24 premiers bits (OUI) identifient le constructeur. Le bit de poids faible du premier octet distingue l'unicast (0) du multicast (1) ; l'adresse `ff:ff:ff:ff:ff:ff` est la diffusion (broadcast).

L'EtherType indique ce que contient la trame ; ces valeurs reviendront dans les règles OpenFlow (champ `eth_type`).

EtherType	Protocole
0x0800	IPv4
0x0806	ARP
0x86DD	IPv6
0x8100	Étiquette VLAN 802.1Q
0x88A8	Étiquette de service 802.1ad (QinQ)
0x88CC	LLDP (découverte de voisins)
0x8847	MPLS

1.3 La commutation

Un switch relie plusieurs équipements d'un même réseau local. Il fonctionne par apprentissage :

1. À chaque trame reçue, il note dans sa table MAC (ou FDB, *forwarding database*) que l'adresse source se trouve derrière le port d'entrée.
2. Si l'adresse de destination est connue, il envoie la trame sur ce seul port.
3. Sinon, ou s'il s'agit d'une diffusion, il la répète sur tous les autres ports : c'est l'inondation (*flooding*).
4. Une entrée non rafraîchie expire, en général après 300 secondes.

Le trafic inondé regroupe trois catégories, abrégées **BUM** : *Broadcast*, *Unknown unicast* et *Multicast*. Ce sigle reviendra pour les overlays, où il faut transporter ce trafic d'un hyperviseur à l'autre.

L'ensemble des machines atteintes par une diffusion forme un **domaine de diffusion**. Sans précaution, une boucle entre switches fait circuler les diffusions indéfiniment (tempête de broadcast). Le protocole Spanning Tree (IEEE 802.1D, puis RSTP 802.1w et MSTP 802.1s) coupe logiquement les liens en trop, au prix de liens inutilisés. Le chapitre 5 montre comment les datacenters modernes s'en passent.

1.4 L'adressage IP

Une adresse IPv4 fait 32 bits ; le masque (ou préfixe, noté `/24`) sépare la partie réseau de la partie hôte. Deux machines du même sous-réseau communiquent directement ; sinon, elles passent par un routeur, leur passerelle.

Les plages privées de la RFC 1918 ne sont pas routées sur Internet : 10.0.0.0/8, 172.16.0.0/12 et 192.168.0.0/16. IPv6 utilise des adresses de 128 bits et remplace ARP par le protocole NDP (*Neighbor Discovery*), porté par ICMPv6.

Quelques champs de l'en-tête IPv4 comptent pour la suite :

Champ	Rôle	Lien avec la suite du cours
TTL	Nombre de sauts restants, décrémenté par chaque routeur	Action `dec_ttl` d'OpenFlow

Champ	Rôle	Lien avec la suite du cours
Protocole	1 ICMP, 6 TCP, 17 UDP	Champ `ip_proto` d'OpenFlow
DSCP	Classe de service	Marquage par un meter OpenFlow
Bit DF	Interdiction de fragmenter	Découverte du MTU (1.8)

1.5 ARP : de l'adresse IP à l'adresse MAC

Pour envoyer un paquet à 10.10.10.2 sur son réseau local, une machine doit connaître l'adresse MAC correspondante. Elle diffuse une requête ARP (« qui a 10.10.10.2 ? ») ; le détenteur répond en unicast. La réponse est gardée dans un cache (`ip neigh` sous Linux).

Deux variantes servent en virtualisation :

- l'**ARP gratuit** : une machine annonce spontanément son adresse, par exemple après la migration d'une VM vers un autre hôte ;
- le **proxy ARP** : un équipement répond à la place du destinataire. Le contrôleur ONOS le fait grâce à sa vue globale des hôtes, ce qui supprime des diffusions inutiles (chapitre 11).

1.6 Le routage

Un routeur choisit la sortie de chaque paquet dans sa table de routage, selon la règle du **préfixe le plus long** : une route vers 10.10.10.0/24 l'emporte sur une route vers 10.0.0.0/8. La route par défaut (0.0.0.0/0) s'applique quand rien d'autre ne correspond.

Les routes sont statiques (saisies à la main) ou dynamiques, apprises par un protocole : OSPF ou IS-IS à l'intérieur d'un réseau, BGP entre réseaux et, de plus en plus, dans les datacenters. Pour faire communiquer deux VLAN, il faut un routeur ou un switch de niveau 3 : c'est le routage inter-VLAN, pratiqué au TP2-A.

1.7 Ports et notion de flux

TCP et UDP identifient l'application par un numéro de port : 22 pour SSH, 53 pour DNS, 80 et 443 pour HTTP et HTTPS. Un **flux** est l'ensemble des paquets qui partagent les mêmes caractéristiques, le plus souvent le quintuplet adresse source, adresse destination, protocole, port source, port destination.

Cette notion est centrale : la répartition de charge ECMP choisit un chemin par flux (chapitre 5), et OpenFlow programme les switches flux par flux (chapitre 15).

1.8 MTU et fragmentation

Le MTU (*Maximum Transmission Unit*) est la taille maximale d'un paquet IP sur un lien : 1 500 octets en Ethernet standard, jusqu'à 9 000 environ en *jumbo frames*. Un paquet plus grand est fragmenté ou, si le bit DF est positionné, détruit avec un message ICMP « fragmentation nécessaire » (type 3, code 4). L'émetteur ajuste alors sa taille : c'est la découverte du MTU de chemin (PMTUD).

En TCP, la taille de segment négociée (MSS) vaut le MTU moins 40 octets d'en-têtes, soit 1 460 octets sur Ethernet.

Attention. Chaque encapsulation réduit la place disponible. Si un pare-feu bloque les messages ICMP, la PMTUD échoue silencieusement : les petits paquets passent (ping, début de connexion SSH), les gros se perdent (transfert de fichier, page web lourde). Ce symptôme est étudié au chapitre 6.

1.9 La boîte à outils Linux

Outil	Usage
`ip addr`, `ip link`, `ip route`, `ip neigh`	Interfaces, adresses, routes, cache ARP
`ss -tulpn`	Ports en écoute et connexions
`ping`, `traceroute`, `tracepath`	Joignabilité, chemin, MTU du chemin
`tcpdump -e -n -i <interface>`	Capture, avec affichage des en-têtes Ethernet
`ethtool <interface>`	Vitesse, délestages matériels, statistiques de la carte
`nft list ruleset`	Règles de filtrage nftables

À retenir. Un switch décide selon l'adresse MAC et apprend en observant ; un routeur décide selon l'adresse IP et le préfixe le plus long. Toute encapsulation ajoute des octets et réduit le MTU disponible.

Chapitre 2 — Pourquoi virtualiser le réseau

Virtualiser le réseau, c'est découpler le service rendu (un segment, une route, une règle de filtrage) de l'équipement qui le porte, pour le créer, le modifier et le détruire par logiciel. Ce chapitre explique d'où vient ce besoin et quelles formes il a prises. Module 1 de la formation.

2.1 Le réseau, goulot d'étranglement de la virtualisation

Dans les années 2000, la virtualisation des serveurs a changé l'échelle des datacenters. Une VM se crée en quelques minutes, se déplace à chaud d'un hôte à l'autre et disparaît quand on n'en a plus besoin. Le réseau, lui, restait configuré équipement par équipement, en ligne de commande, souvent par une autre équipe et au travers d'un ticket.

Trois évolutions ont rendu ce décalage intenable :

- **Le volume.** Un hôte porte des dizaines de VM ou des centaines de conteneurs, chacun avec son adresse et ses règles d'accès.
- **La mobilité.** Une VM migrée doit garder son adresse IP et retrouver ses règles sur l'hôte d'arrivée.
- **Le trafic est-ouest.** Dans une application découpée en services, l'essentiel du trafic circule entre serveurs du même datacenter (est-ouest) plutôt qu'entre le datacenter et l'extérieur (nord-sud). Les pare-feu placés en périphérie ne le voient pas.

Le cloud a ajouté une exigence : le libre-service. Un utilisateur qui obtient une VM par un appel d'API attend la même chose de son réseau privé, de son routeur et de son groupe de sécurité.

2.2 Définition et propriétés

La virtualisation réseau présente à ses utilisateurs des ressources réseau logiques, indépendantes de l'infrastructure physique qui les réalise. On parle par analogie d'« hyperviseur réseau » : comme l'hyperviseur découpe un serveur en VM, la couche de virtualisation découpe un réseau physique en réseaux logiques.

Elle poursuit trois propriétés :

Propriété	Signification	Exemple
Abstraction	L'utilisateur manipule un « switch logique » ou un « routeur logique », sans savoir sur quel matériel il s'exécute	Un réseau Neutron dans OpenStack
Isolation	Deux réseaux logiques ne se voient pas, même s'ils utilisent les mêmes adresses	Deux locataires en 10.0.0.0/24 sur la même infrastructure
Programmabilité	Tout se crée et se modifie par API, donc par script	Un segment créé par Terraform ou Ansible

2.3 Les formes historiques

La virtualisation réseau n'est pas née avec le cloud. Deux familles existent depuis longtemps.

Partitionner : un équipement devient plusieurs.

- **VLAN** (IEEE 802.1Q, 1998) : un switch physique porte plusieurs réseaux locaux isolés, identifiés par un numéro de 1 à 4 094 inséré dans la trame. C'est la forme la plus répandue, détaillée au chapitre 6.
- **VRF** (*Virtual Routing and Forwarding*) : un routeur porte plusieurs tables de routage indépendantes. Deux VRF peuvent utiliser les mêmes préfixes sans conflit. Linux propose des interfaces VRF depuis le noyau 4.3.
- **Instances virtuelles** : un pare-feu ou un répartiteur de charge matériel héberge plusieurs contextes indépendants.

Agréger : plusieurs équipements deviennent un seul.

- **Agrégation de liens** (IEEE 802.1AX, protocole LACP) : plusieurs câbles forment un lien logique plus rapide et redondant.
- **Empilement et châssis virtuel**, MLAG : plusieurs switches se présentent comme un seul, ce qui permet de raccorder un serveur à deux switches sans boucle.

Relier à travers un réseau partagé : les VPN. Un VPN (*Virtual Private Network*) transporte un réseau privé dans un tunnel au-dessus d'un réseau partagé. IPsec et WireGuard chiffrent le tunnel. Les VPN MPLS des opérateurs (RFC 4364) combinent tunnels et VRF pour transporter les réseaux de milliers de clients sur un seul cœur de réseau.

2.4 Les approches modernes

Le cloud a fait émerger trois approches, souvent combinées :

Approche	Où vit l'intelligence	Exemples
Overlay porté par les hôtes	Commutateurs logiciels des hyperviseurs et nœuds, reliés par des tunnels ; le réseau physique ne voit que de l'IP	OVN (OpenStack, OpenShift), VMware NSX, Calico, Cilium
Overlay porté par la fabric	Switches physiques du datacenter, qui encapsulent et échangent leurs informations par BGP EVPN	Fabrics EVPN-VXLAN multi-constructeurs
SDN à contrôleur	Un contrôleur centralisé programme les commutateurs, physiques ou logiciels	ONOS, Faucet, contrôleurs propriétaires

Ces approches ne s'excluent pas : OVN est à la fois un overlay porté par les hôtes et un SDN à contrôleur, qui programme les switches en OpenFlow (chapitre 10).

2.5 Ce que cela change pour l'exploitant

Un réseau virtuel est un objet logiciel. Il a donc un cycle de vie (création, modification, suppression), une définition qui peut être versionnée et des tests. L'exploitation se rapproche de celle des applications : infrastructure as code, revue de code, déploiement continu.

Cela suppose des compétences croisées. L'administrateur système découvre que l'hôte Linux est devenu un équipement réseau. L'ingénieur réseau doit lire du YAML et appeler une API. L'ingénieur DevOps doit comprendre pourquoi un MTU mal réglé casse un déploiement.

Un vocabulaire commun s'impose :

- **Locataire** (*tenant*) : utilisateur ou organisation isolé des autres ;
- **Segment** ou **switch logique** : domaine de niveau 2 virtuel ;
- **Routeur logique** : routage entre segments, souvent distribué sur tous les hôtes ;
- **Groupe de sécurité** : règles de filtrage attachées à une VM ou un pod plutôt qu'à une adresse.

2.6 Repères historiques

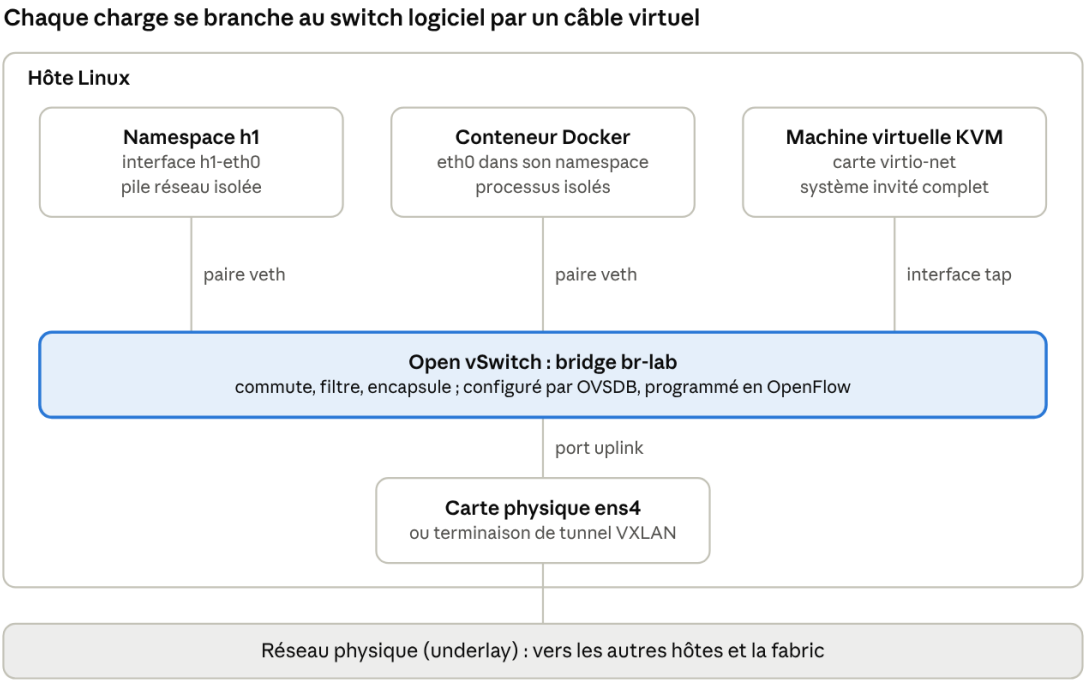
Année	Événement
1998	Norme IEEE 802.1Q : les VLAN
1999	VPN MPLS (RFC 2547, devenue RFC 4364)
2007	Projet Ethane à Stanford, ancêtre du SDN
2008	Article « OpenFlow: Enabling Innovation in Campus Networks »
2009	Première version publique d'Open vSwitch

Année	Événement
2011	Création de l'Open Networking Foundation (ONF)
2012	Livre blanc NFV des opérateurs, création du groupe ETSI ISG NFV ; rachat de Nicira par VMware, qui donnera NSX
2014	RFC 7348 (VXLAN) ; premières versions d'OpenDaylight (Hydrogen) et d'ONOS (Avocet)
2015	RFC 7432 (EVPN) ; Kubernetes 1.0 et spécification CNI
2016	OVN publié avec Open vSwitch 2.6
2020	RFC 8926 (Geneve)

À retenir. La virtualisation réseau répond à la vitesse, au volume et à la mobilité des charges virtualisées. Elle repose sur trois propriétés, abstraction, isolation et programmabilité, et prolonge des techniques anciennes : VLAN, VRF, VPN.

Chapitre 3 — Les briques de la virtualisation réseau sous Linux

Un hôte Linux moderne est un équipement réseau à part entière : il isole des piles réseau, les relie par des câbles virtuels et les commute dans un switch logiciel. Ce chapitre décrit ces briques une à une ; elles sont toutes manipulées au TP1. Module 1 de la formation.



un hôte Linux virtualisé · 3 types de charges, 1 switch logiciel

Les trois charges reposent sur les mêmes briques : une pile réseau isolée, un câble virtuel et un port sur le switch logiciel, lui-même relié au réseau physique.

3.1 Les network namespaces

Un **network namespace** est une pile réseau complète et isolée au sein du même noyau. Chaque namespace possède ses propres :

- interfaces réseau, y compris son interface de bouclage `lo` ;
- adresses, tables de routage et cache ARP ;
- règles de filtrage netfilter (iptables, nftables) ;
- sockets et ports en écoute : deux namespaces peuvent chacun écouter sur le port 80 ;
- paramètres `net.*` de sysctl, par exemple `net.ipv4.ip_forward`.

Un processus lancé dans un namespace ne voit que cette pile. C'est la brique de base des conteneurs : un conteneur Docker ou un pod Kubernetes est d'abord un namespace réseau. Linux propose d'autres types de namespaces (montage, processus, utilisateurs, nom d'hôte, IPC, cgroup, temps) ; seul le namespace réseau nous intéresse ici.

Commande	Effet
`ip netns add h1`	Crée un namespace nommé (fichier sous `/var/run/netns/`)
`ip netns exec h1 <commande>`	Exécute une commande dans le namespace
`ip -n h1 addr`	Raccourci pour les commandes `ip`
`ip netns list`, `lsns -t net`	Liste les namespaces nommés, ou tous
`nsenter -t <pid> -n <commande>`	Entre dans le namespace d'un processus, par exemple d'un conteneur
`ip netns del h1`	Supprime le namespace et les interfaces qu'il contient

3.2 Les paires veth

Une paire **veth** (*virtual Ethernet*) est un câble virtuel : deux interfaces liées, où toute trame émise sur l'une ressort sur l'autre. L'usage type place une extrémité dans un namespace et branche l'autre sur un switch logiciel de l'hôte.

```
sudo ip link add h1-eth0 type veth peer name h1-port
sudo ip link set h1-eth0 netns h1
```

Pour retrouver l'autre bout d'une veth, `ethtool -S <interface>` affiche l'index de l'interface paire (`peer_ifindex`). **Attention** : un nom d'interface Linux ne dépasse pas 15 caractères.

3.3 Le bridge Linux

Le **bridge** Linux est un switch de niveau 2 intégré au noyau. Il apprend les adresses MAC, inonde les destinations inconnues et peut exécuter le Spanning Tree. Avec l'option `vlan_filtering`, il gère aussi les VLAN par port.

```
sudo ip link add br0 type bridge vlan_filtering 1
sudo ip link set h1-port master br0
bridge fdb show br br0          # table MAC
bridge vlan show                # VLAN par port
```

C'est lui qui se cache derrière l'interface `docker0` de Docker et derrière `virbr0` de libvirt. Kubernetes impose souvent le module `br_netfilter`, qui fait passer le trafic ponté par les règles iptables (paramètre `net.bridge.bridge-nf-call-iptables`).

3.4 Les autres interfaces virtuelles

Type	Principe	Usage
tap	Interface de niveau 2 dont l'autre bout est un processus	Carte réseau d'une VM QEMU/KVM
tun	Équivalent de niveau 3 (paquets IP)	VPN (OpenVPN)
macvlan	Sous-interfaces d'une carte physique, chacune avec sa propre adresse MAC	Conteneurs directement sur le LAN
ipvlan	Variante partageant la MAC de la carte mère, en mode L2 ou L3	Environnements limitant le nombre de MAC par port
vlan	Sous-interface étiquetée 802.1Q	Routeur « sur un bâton » (TP2-A)
vxlan, geneve	Terminaison de tunnel overlay	VTEP natif Linux (TP2-B)
vrf	Table de routage séparée	Isoler des domaines de routage
bond, team	Agrégation de cartes physiques	Redondance et débit vers les switches
dummy	Interface sans lien physique	Adresse de service, tests

3.5 Open vSwitch

Open vSwitch (OVS) est un switch logiciel programmable, créé par la société Nicira et publié en open source en 2009 sous licence Apache 2.0. Il est devenu le commutateur de référence des plateformes virtualisées : OpenStack, OVN-Kubernetes et OpenShift, Antrea, XenServer et de nombreuses appliances reposent sur lui.

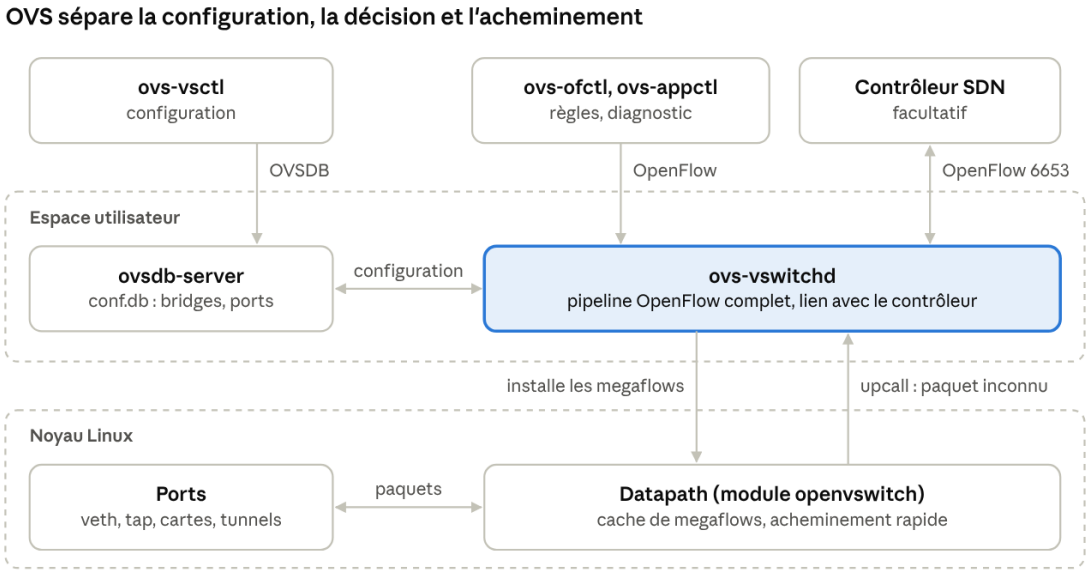
Par rapport au bridge Linux, il ajoute :

- la programmation par **OpenFlow** (versions 1.0 à 1.5), donc le pilotage par un contrôleur SDN ;
- une base de configuration normalisée, **OVSDB** (RFC 7047), interrogeable à distance ;
- les tunnels VXLAN, Geneve, GRE et STT ;
- le suivi de connexions (conntrack) pour le filtrage à états ;
- la qualité de service, le miroir de ports et l'export de flux (sFlow, NetFlow, IPFIX) ;
- l'agrégation de liens LACP.

Architecture. OVS se compose de trois éléments.

Composant	Espace	Rôle
`ovsdb-server`	Utilisateur	Conserve la configuration (bridges, ports, interfaces, contrôleurs) dans le fichier `conf.db` et la sert par le protocole OVSDB
`ovs-vswitchd`	Utilisateur	Cœur du switch : lit la configuration, exécute le pipeline OpenFlow, dialogue avec le contrôleur
Datapath	Noyau (module `openvswitch`) ou espace utilisateur avec DPDK	Achemine les paquets à grande vitesse à partir d'un cache de flux

Le datapath ne connaît pas les règles OpenFlow. Quand un paquet ne correspond à aucune entrée de son cache, il le remonte à `ovs-vswitchd` (*upcall*). Celui-ci exécute le pipeline OpenFlow complet, puis installe dans le datapath une entrée généralisée, le **megaflow**, qui servira aux paquets suivants du même type. Le premier paquet d'un flux est donc plus lent que les autres.



architecture d'Open vSwitch · 3 composants, 4 outils

Chaque outil parle à un composant : `ovs-vsctl` à la base de configuration, `ovs-ofctl` et le contrôleur au démon `ovs-vswitchd` ; le noyau ne voit que des megaflows.

Les outils. Chaque outil parle à un composant différent ; savoir lequel utiliser évite bien des confusions.

Outil	S'adresse à	Exemples
`ovs-vsctl`	La configuration (OVSDB)	`add-br`, `add-port`, `set-controller`, `show`
`ovs-ofctl`	Les tables OpenFlow d'un bridge	`dump-flows`, `add-flow`, `dump-ports`

Outil	S'adresse à	Exemples
`ovs-appctl`	Le démon `ovs-vswitchd` en fonctionnement	`fdb/show`, `ofproto/trace`, `vlog/set`
`ovs-dpctl`	Le datapath du noyau	`dump-flows` (megaflows installés)
`ovsdb-client`	Le serveur OVSDB, en lecture brute	`list-tables`, `dump`, `monitor`

Le modèle de données. La base OVSDB est organisée en tables reliées entre elles : `Open_vSwitch` (racine), `Bridge`, `Port`, `Interface`, `Controller`, `Manager`, `Mirror`, `QoS`, `Queue`. Un **port** est un point de raccordement logique ; il contient une ou plusieurs **interfaces**. Un port agrégé (bond) compte ainsi plusieurs interfaces. Un port de type `internal` crée une interface visible par le noyau, utile pour donner une adresse IP à l'hôte sur le bridge ; un port de type `patch` relie deux bridges OVS.

En TP. Au TP1-A, `ovs-vsctl show` lit la base OVSDB, `ovs-ofctl dump-flows` montre l'unique règle `actions=NORMAL` d'un bridge sans contrôleur et `ovs-appctl fdb/show` affiche les adresses MAC apprises.

3.6 Raccorder une machine virtuelle

Une VM KVM reçoit une carte réseau émulée par QEMU. Côté hôte, cette carte apparaît comme une interface **tap**, que l'on branche sur un bridge Linux ou OVS.

Pour la performance, on utilise une carte paravirtualisée **virtio-net** : le pilote invité sait qu'il est virtualisé et échange ses paquets avec l'hôte par des files partagées. Le module noyau **vhost-net** traite ces files sans passer par le processus QEMU ; avec DPDK, **vhost-user** les traite en espace utilisateur.

Avec libvirt, trois modes reviennent :

- réseau NAT par défaut (`virbr0`) : les VM sortent par l'adresse de l'hôte ;
- pont vers le réseau physique : les VM sont directement sur le LAN ;
- raccordement à Open vSwitch, déclaré par `<virtualport type='openvswitch'/>` dans la définition de l'interface.

3.7 Raccorder un conteneur

Docker. En mode `bridge` (le défaut), chaque conteneur reçoit un namespace et une paire veth branchée sur `docker0`. Les sorties sont traduites par NAT (règle `MASQUERADE`), et la publication d'un port (`-p 8080:80`) crée une règle de redirection (DNAT). Les modes `host` (pile de l'hôte), `none` (aucun réseau) et `macvlan` existent aussi.

Kubernetes. Le modèle réseau de Kubernetes impose trois règles :

1. chaque pod a sa propre adresse IP ;
2. tous les pods communiquent entre eux sans NAT, quel que soit le nœud ;
3. les agents d'un nœud communiquent avec tous les pods de ce nœud.

Kubernetes ne réalise pas lui-même ce modèle : il délègue à un plugin conforme à la spécification **CNI** (*Container Network Interface*). À la création d'un pod, le runtime crée un namespace, partagé par tous les conteneurs du pod, puis appelle le binaire du plugin (sous `/opt/cni/bin`) avec la commande `ADD` et

la configuration trouvée sous `/etc/cni/net.d`. Le plugin crée l'interface du pod, lui attribue une adresse et programme les routes ou les tunnels.

Plugin CNI	Technique principale	Remarque
Flannel	Overlay VXLAN (port UDP 8472 par défaut)	Simple, sans politique réseau
Calico	Routage IP et BGP, ou IP-in-IP, VXLAN, eBPF	Politiques réseau avancées
Cilium	eBPF, overlay VXLAN ou Geneve, ou routage natif	Politiques de niveau 3 à 7, observabilité
OVN-Kubernetes	OVN et Open vSwitch, overlay Geneve	Plugin par défaut d'OpenShift
Antrea	Open vSwitch, overlay Geneve	Piloté en OpenFlow

Les **Services** Kubernetes (adresse virtuelle stable devant un ensemble de pods) sont réalisés par kube-proxy avec iptables ou IPVS, ou directement par le plugin en eBPF. Les **NetworkPolicy** décrivent les flux autorisés entre pods ; le plugin les traduit en règles. C'est une politique réseau déclarative, proche de l'esprit SDN (chapitre 12).

3.8 Performance et accélération

Un paquet qui traverse un switch logiciel consomme du CPU : interruptions, copies mémoire, recherches dans les tables, encapsulation. Plusieurs techniques réduisent ce coût.

Technique	Principe	Contrepartie
Délestages de la carte (checksum, TSO, GRO, délestage VXLAN)	La carte segmente, assemble ou calcule les sommes de contrôle	Dépend du matériel et du pilote
Multi-files (RSS)	Répartit la réception sur plusieurs cœurs	Réglage de l'affinité des interruptions
SR-IOV	La carte physique (PF) expose des cartes virtuelles (VF) attribuées directement aux VM	Contourne le switch logiciel : pas de politique OVS, migration à chaud difficile
DPDK	Pilotes en espace utilisateur qui scrutent la carte en continu, mémoire en *hugepages*	Cœurs CPU dédiés en permanence, exploitation plus complexe
XDP et eBPF	Programmes vérifiés exécutés dans le noyau, au plus près du pilote	Programmation spécifique (chapitre 18)
Délestage matériel d'OVS (SmartNIC, DPU)	Les megaflows sont installés dans la carte	Matériel spécifique

Pour aller plus loin. `ovs-dpctl dump-flows` affiche les megaflows installés dans le datapath. Les comparer aux règles de `ovs-ofctl dump-flows` montre comment OVS résume des centaines de règles OpenFlow en quelques entrées de cache.

À retenir. Namespace = pile isolée ; veth = câble ; bridge ou OVS = switch ; tap = carte d'une VM. Un conteneur est un namespace raccordé par une veth, une VM une interface tap. Open vSwitch ajoute au bridge Linux la programmation OpenFlow et la configuration par OVSDB.

Chapitre 4 — Réseaux virtuels, SDN et NFV

Réseau virtuel, SDN et NFV sont souvent confondus : le premier découpe le réseau, le deuxième centralise sa commande, la troisième transforme ses équipements en logiciels. Ce chapitre les distingue, détaille la NFV et fait le bilan des avantages et des risques. Module 1 de la formation.

4.1 Quatre notions à distinguer

Notion	Question à laquelle elle répond	Où elle agit	Exemple
Réseau virtuel	Comment offrir à chacun un réseau isolé sur une infrastructure partagée ?	Segments, adressage, isolation	VLAN, segment VXLAN, réseau Neutron
VM et conteneur	Qui consomme le réseau virtuel ?	Points de raccordement	VM KVM sur tap, pod sur veth
SDN	Qui décide de l'acheminement, et comment le programmer ?	Plan de contrôle	Contrôleur ONOS, OVN
NFV	Comment exécuter les fonctions réseau en logiciel ?	Fonctions : pare-feu, routeur, répartiteur	Pare-feu virtuel, cœur 5G virtualisé

4.2 Les réseaux virtuels

Un réseau virtuel offre un domaine de niveau 2 (un segment, où les machines se voient comme sur un même switch) ou de niveau 3 (un réseau routé) défini par logiciel. Deux techniques le réalisent :

- l'**étiquetage** : chaque trame porte l'identifiant de son segment (VLAN), et tous les équipements du chemin doivent connaître ce segment ;
- l'**overlay** : les trames du segment sont encapsulées dans des paquets IP entre hôtes ; le réseau physique, l'**underlay**, ne transporte que de l'IP et ignore les segments. Les chapitres 5 à 7 détaillent ce modèle.

Les plateformes de cloud ajoutent par-dessus des objets de plus haut niveau :

- le **routeur logique distribué** : chaque hyperviseur route localement le trafic de ses VM, avec la même adresse de passerelle partout (passerelle *anycast*), ce qui évite de faire transiter le trafic par un routeur central ;

- l'**adresse flottante** : une adresse publique traduite par NAT vers une VM, déplaçable d'une VM à l'autre ;
- le **groupe de sécurité** : des règles de filtrage attachées à la carte réseau virtuelle, qui suivent la VM quand elle migre.

4.3 Le SDN en bref

Le SDN (*Software-Defined Networking*) sépare le plan de contrôle, qui décide, du plan de données, qui exécute, et regroupe le contrôle dans un contrôleur logiquement centralisé et programmable. Les équipements reçoivent leurs règles du contrôleur au lieu de les calculer chacun de leur côté. Le module 3 (chapitres 9 à 13) lui est consacré.

4.4 La virtualisation des fonctions réseau (NFV)

Origine. En octobre 2012, un groupe d'opérateurs télécoms publie un livre blanc intitulé « Network Functions Virtualisation ». Leur constat : leurs réseaux accumulent des boîtiers spécialisés, coûteux, longs à déployer et difficiles à faire évoluer. Leur proposition : exécuter ces fonctions comme des logiciels sur des serveurs standards, dans des centres de données. L'ETSI crée dans la foulée le groupe ISG NFV, qui normalise l'architecture (chapitre 8).

Vocabulaire.

- **VNF** (*Virtualised Network Function*) : fonction réseau exécutée dans une ou plusieurs VM.
- **CNF** (*Cloud-native Network Function*) : fonction réseau conçue en microservices et exécutée en conteneurs, généralement sur Kubernetes. C'est l'évolution actuelle de la NFV.
- **NFVI** : l'infrastructure (serveurs, stockage, réseau, virtualisation) qui héberge les VNF.
- **MANO** : la gestion et l'orchestration des VNF et des services qu'elles composent.

Exemples de fonctions virtualisées.

Domaine	Fonctions
Sécurité	Pare-feu, détection et prévention d'intrusion, passerelle VPN, proxy
Acheminement	Routeur virtuel, répartiteur de charge, NAT
Services	DNS, DHCP, accélération WAN
Télécoms	Cœur de réseau mobile 4G et 5G, passerelles d'accès, équipement client virtualisé (vCPE)

Cycle de vie d'une VNF. Une VNF est décrite par un descripteur (VNFD) : images, ressources, interfaces, règles de mise à l'échelle. Son cycle de vie suit les étapes suivantes :

1. **Intégration** (*onboarding*) : le descripteur et les images sont enregistrés dans le catalogue.
2. **Instanciation** : les VM ou conteneurs sont créés et raccordés aux réseaux virtuels.
3. **Mise à l'échelle** : ajout ou retrait d'instances selon la charge (*scale out*, *scale in*).
4. **Réparation** (*healing*) : une instance défaillante est recréée automatiquement.
5. **Mise à jour**, puis **arrêt** (*termination*) et libération des ressources.

En TP. Au TP1-B, le pare-feu nftables placé dans un namespace est une VNF minimale : on l'instancie, on lui applique une politique, on le détruit puis on le recrée par script en quelques secondes.

4.5 Le chaînage de services

Un flux doit souvent traverser plusieurs fonctions dans un ordre précis, par exemple pare-feu, puis sonde de détection, puis répartiteur de charge. C'est le **chaînage de services** (SFC, *Service Function Chaining*).

Avec des boîtiers physiques, l'ordre est figé par le câblage. Avec des VNF, il devient une politique logicielle. La RFC 7665 décrit l'architecture :

- un **classificateur** identifie le flux et lui attribue une chaîne ;
- des **relais** (*Service Function Forwarders*) acheminent le paquet de fonction en fonction ;
- un en-tête dédié, **NSH** (*Network Service Header*, RFC 8300), peut porter l'identifiant de la chaîne et la position dans celle-ci.

En pratique, beaucoup de plateformes réalisent le chaînage sans NSH, par routage de politique ou règles OpenFlow qui redirigent le trafic vers la fonction suivante. C'est là que SDN et NFV se rejoignent : le contrôleur oriente les flux, les VNF les traitent.

4.6 SDN et NFV, complémentaires mais indépendants

	SDN	NFV
Ce qui est transformé	La commande du réseau	Les équipements du réseau
Principe	Séparer décision et exécution, centraliser la décision	Remplacer le matériel dédié par du logiciel
Origine	Recherche universitaire, puis ONF	Opérateurs télécoms, puis ETSI
Possible sans l'autre ?	Oui : piloter des switches physiques en OpenFlow	Oui : un pare-feu virtuel dans un réseau classique
Ensemble	Le SDN fait circuler le trafic à travers les VNF	La NFV fournit les fonctions que l'orchestrateur instancie à la demande

4.7 Les avantages

- **Agilité** : un segment, un routeur ou un pare-feu se crée en secondes, sans intervention physique.
- **Automatisation** : tout passe par des API, donc par des scripts, des outils d'infrastructure as code et des pipelines CI/CD.
- **Élasticité** : une fonction réseau suit la charge, par ajout ou retrait d'instances.
- **Isolation** : dev, recette et production, ou plusieurs entités, cohabitent sur la même infrastructure sans se voir.

- **Résilience** : une fonction perdue se recrée à l'identique ailleurs ; la configuration versionnée facilite le plan de reprise.
- **Environnements de test fidèles** : la recette peut reproduire le réseau de production, et un lab se reconstruit par script.
- **Coûts** : matériel banalisé et mutualisé. Le gain réel dépend toutefois des licences, de la consommation CPU et des compétences à réunir.
- **Maîtrise technologique** : les briques open source (Linux, Open vSwitch, OVN) sont auditables et ne lient pas à un fournisseur.

4.8 Les enjeux et les risques

Enjeu	Risque concret	Parade
Performance	Le CPU des hôtes absorbe commutation, encapsulation et filtrage	Délestages matériels, DPDK ou SR-IOV pour les fonctions exigeantes (chapitre 3)
Complexité	Couches logiques empilées, dépannage plus abstrait	Outils de trace (`ofproto/trace`), observabilité des flux, documentation à jour
Organisation	Équipes système, réseau et sécurité en silos	Responsabilités explicites, compétences croisées, revue de code commune
Visibilité	Le trafic est-ouest échappe aux pare-feu périphériques	Micro-segmentation (4.9), miroir de trafic vers les sondes
Concentration du risque	Compromettre l'hyperviseur, le contrôleur ou l'orchestrateur donne la main sur tout le réseau	Durcissement, réseau d'administration dédié, authentification forte, journalisation (chapitre 13)
Prolifération	Réseaux et règles créés puis oubliés	Tout créer par code, inventaire automatique, réconciliation régulière
Dépendance	Solutions propriétaires difficiles à quitter	Normes ouvertes, modèles de données standard, briques open source

4.9 Micro-segmentation et Zero Trust

La sécurité périmétrique suppose qu'un attaquant arrêté à l'entrée ne circule pas à l'intérieur. Dans un datacenter virtualisé, une machine compromise peut au contraire atteindre ses voisines sans croiser aucun pare-feu : c'est le mouvement latéral.

La **micro-segmentation** applique le filtrage au plus près de chaque charge : sur la carte virtuelle de la VM ou sur l'interface du pod. Les règles s'expriment par identité ou par étiquette (« les pods `app=api` acceptent les pods `app=web` sur le port 8443 ») plutôt que par adresse IP, et suivent la charge quand elle

migre. Groupes de sécurité OpenStack, NetworkPolicy Kubernetes et pare-feu distribué NSX en sont des réalisations.

C'est une brique du modèle **Zero Trust**, décrit par le NIST (SP 800-207, 2020) : aucune confiance implicite accordée en fonction de l'emplacement réseau, chaque accès est authentifié et autorisé. L'ANSSI a publié en 2021 un avis sur ce modèle, qui le présente comme un complément et non un remplaçant de la défense en profondeur.

À retenir. Le réseau virtuel découpe, le SDN commande, la NFV transforme les équipements en logiciels. SDN et NFV se combinent mais restent indépendants. Les gains d'agilité s'accompagnent d'une concentration du risque sur les couches de contrôle, à protéger en priorité.

Chapitre 5 — Architectures des réseaux de datacenter

Les datacenters modernes séparent un réseau physique simple et routé, l'underlay, des réseaux virtuels dynamiques construits par-dessus, l'overlay. Ce chapitre explique pourquoi l'architecture leaf-spine a remplacé l'arbre à trois niveaux et comment on y loge plusieurs locataires. Module 2 de la formation.

5.1 L'architecture à trois niveaux

L'architecture classique empile trois couches de switches :

- l'**accès**, où se raccordent les serveurs ;
- l'**agrégation** (ou distribution), qui regroupe les switches d'accès et porte souvent les passerelles des VLAN ;
- le **cœur**, qui relie les agrégations entre elles et vers l'extérieur.

Ce modèle a été conçu pour un trafic surtout nord-sud, entre les serveurs et les utilisateurs. Il montre ses limites quand le trafic devient est-ouest :

- le Spanning Tree bloque les liens redondants, donc la moitié de la capacité installée reste inutilisée ;
- deux serveurs éloignés traversent jusqu'à cinq switches, avec une latence variable selon leur position ;
- les VLAN s'étendent sur de grands domaines de niveau 2 : une tempête de broadcast ou une boucle touche tout le site ;
- la capacité vers le haut est vite saturée (sursouscription forte).

5.2 L'architecture leaf-spine

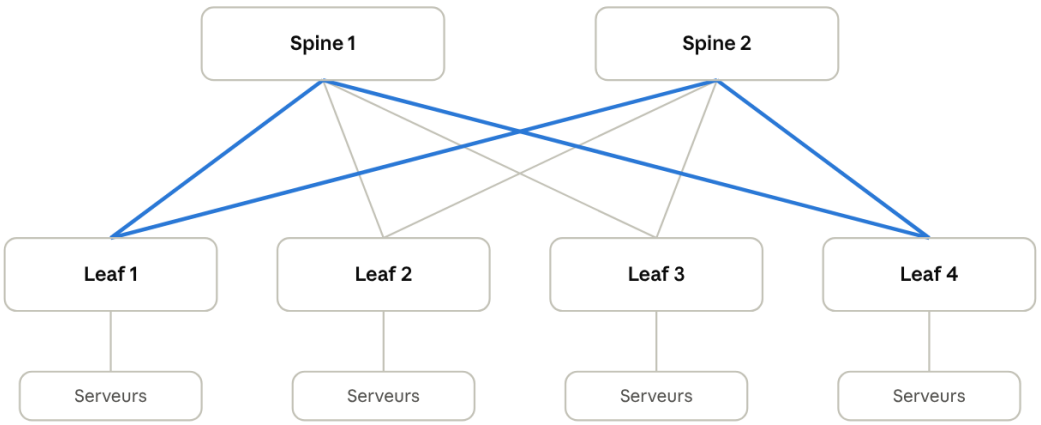
L'architecture **leaf-spine** reprend un principe de commutation téléphonique décrit par Charles Clos en 1953 : un réseau à plusieurs étages où chaque élément d'un étage est relié à tous ceux de l'étage suivant.

- Les **leafs** (feuilles) raccordent les serveurs. Chaque leaf est relié à chaque spine.
- Les **spines** (épines) ne raccordent que des leafs ; ils ne sont pas reliés entre eux.
- Deux serveurs sur des leafs différents sont toujours à la même distance : leaf, spine, leaf.

Il existe autant de chemins de même coût entre deux leafs que de spines. Le routage **ECMP** (*Equal-Cost Multi-Path*) les utilise tous à la fois. Chaque switch choisit le chemin d'un paquet par un hachage de son

quintuplet : tous les paquets d'un même flux suivent ainsi le même chemin et arrivent dans l'ordre, tandis que des flux différents se répartissent sur tous les liens.

Chaque leaf est relié à chaque spine : tous les chemins ont la même longueur



Les 2 chemins ECMP de même coût entre le leaf 1 et le leaf 4, utilisés en même temps

Liens leaf-spine routés (IP point à point) ; aucun lien entre spines ni entre leafs

fabric leaf-spine · 2 spines, 4 leafs

Avec deux spines, deux chemins égaux relient n'importe quels leafs ; ajouter un spine ajoute un chemin et de la capacité à tous les leafs à la fois.

Dimensionner. On ajoute des spines pour augmenter la capacité, des leafs pour augmenter le nombre de ports serveurs. Le **taux de sursouscription** d'un leaf compare sa capacité vers les serveurs à sa capacité vers les spines. Par exemple, 48 ports serveurs à 25 Gbit/s (1 200 Gbit/s) et 6 liens montants à 100 Gbit/s (600 Gbit/s) donnent un taux de 2:1.

Rôles particuliers. Des *border leafs* relient la fabric au WAN et à Internet ; des *service leafs* raccordent pare-feu et répartiteurs. Au-delà de quelques dizaines de leafs, on ajoute un étage de *super-spines* qui relie plusieurs ensembles leaf-spine (*pods*).

5.3 L'underlay

L'underlay est le réseau physique de la fabric. Ses principes de conception :

- **tout est routé** : chaque lien leaf-spine est un lien IP point à point (souvent un /31), il n'y a plus de Spanning Tree ;
- chaque switch possède une adresse de **loopback** stable, qui sert d'identifiant et de source des tunnels ;
- un protocole de routage annonce les loopbacks : OSPF, IS-IS ou, de plus en plus, eBGP selon la RFC 7938 (un numéro de système autonome par leaf, un numéro commun aux spines) ;
- **BFD** (*Bidirectional Forwarding Detection*) détecte une panne de lien en moins d'une seconde et déclenche le recalcul ;

- le **MTU** de la fabric est relevé, souvent à 9 000 octets ou plus, pour absorber l'encapsulation de l'overlay.

L'underlay change rarement : on y ajoute un leaf ou un spine, pas un réseau applicatif. Cette stabilité est voulue.

5.4 L'overlay

L'overlay porte les réseaux virtuels. Les trames des machines sont encapsulées par un **VTEP** (*VXLAN Tunnel End Point*), transportées dans l'underlay comme de simples paquets IP, puis décapsulées par le VTEP de destination.

Le VTEP peut se trouver :

- dans le **leaf** (overlay porté par la fabric) : les serveurs envoient des trames classiques, éventuellement étiquetées VLAN ;
- dans l'**hôte** (overlay porté par les hôtes) : l'hyperviseur ou le nœud Kubernetes encapsule lui-même, et la fabric ne voit que de l'IP.

Ce découplage est la clé de l'agilité : créer un réseau virtuel ne touche plus au réseau physique.

5.5 Loger plusieurs locataires

Le multi-locataire (*multi-tenancy*) exige deux niveaux d'isolation :

- au **niveau 2**, chaque segment reçoit son identifiant d'overlay (VNI). Les trames d'un segment ne sont jamais livrées dans un autre ;
- au **niveau 3**, chaque locataire reçoit sa propre **VRF**, donc sa propre table de routage. Deux locataires peuvent utiliser les mêmes adresses. Le trafic entre locataires, quand il est autorisé, passe par un point de contrôle explicite, en général un pare-feu.

La **passerelle anycast distribuée** complète le dispositif : la passerelle d'un segment existe sur tous les leafs (ou tous les hyperviseurs) avec la même adresse IP et la même adresse MAC. Une machine trouve sa passerelle sur son leaf, et une VM migrée la retrouve à l'identique.

Pour aller plus loin. Dans une fabric EVPN, le routage entre segments d'un même locataire se fait en mode asymétrique (routage au leaf d'entrée, qui doit connaître tous les segments) ou symétrique (routage aux deux extrémités, via un VNI de niveau 3 propre à la VRF). Le mode symétrique passe mieux à l'échelle.

5.6 Comparaison des deux architectures

Critère	Trois niveaux	Leaf-spine
Trafic privilégié	Nord-sud	Est-ouest
Liens redondants	Bloqués par le Spanning Tree	Tous actifs (ECMP)
Distance entre serveurs	Variable (jusqu'à 5 switches)	Constante (3 switches)
Domaine de panne de	Large	Limité au leaf, ou nul avec un overlay

Critère	Trois niveaux	Leaf-spine
niveau 2		
Croissance	Remplacement des équipements du cœur	Ajout de spines ou de leafs
Réseaux virtuels	VLAN étendus	Overlay VXLAN sur underlay routé

5.7 La plateforme LAB-SDML sur une fabric

Sur une fabric leaf-spine, les quatre zones du fil rouge deviendraient quatre segments VXLAN, par exemple VNI 10010 (dev), 10020 (recette), 10030 (prod) et 10099 (admin). Chaque VLAN local du leaf est associé à son VNI. Une VRF par niveau de sensibilité sépare le routage, et les flux entre zones traversent un pare-feu raccordé sur un service leaf. Au TP2, ce schéma est reproduit en miniature avec Open vSwitch : VLAN au TP2-A, VXLAN au TP2-B.

À retenir. Leaf-spine = chaque leaf relié à chaque spine, chemins égaux, ECMP par flux. L'underlay est routé, stable et doté d'un grand MTU ; l'overlay porte les réseaux virtuels. L'isolation des locataires combine VNI au niveau 2 et VRF au niveau 3.

Chapitre 6 — Encapsulations et overlays

Un réseau virtuel s'appuie soit sur une étiquette ajoutée à la trame (VLAN), soit sur un tunnel qui l'enveloppe dans un nouveau paquet (VXLAN, Geneve). Ce chapitre détaille le format de chaque encapsulation, ses limites et son coût en octets. Module 2 de la formation.

6.1 Le principe du tunnel

Un tunnel ajoute devant la trame (ou le paquet) d'origine un en-tête externe, qui la fait transporter par un autre réseau. L'équipement d'entrée encapsule, celui de sortie décapsule ; entre les deux, le réseau ne lit que l'en-tête externe.

Terme	Définition
Underlay	Réseau qui transporte les paquets encapsulés
Overlay	Réseau virtuel transporté dans les tunnels
VTEP, NVE	Équipement qui encapsule et décapsule
VNI	Identifiant du réseau virtuel, porté dans l'en-tête du tunnel
Charge interne	Trame ou paquet d'origine, inchangé

On distingue les tunnels de **niveau 2**, qui transportent des trames Ethernet entières (VXLAN, Geneve, GRETAP), et les tunnels de **niveau 3**, qui transportent des paquets IP (GRE, IPsec, WireGuard).

6.2 Les VLAN (IEEE 802.1Q)

La norme 802.1Q insère une étiquette de 4 octets entre l'adresse MAC source et l'EtherType.

Champ	Taille	Contenu
TPID	16 bits	0x8100 : signale une trame étiquetée
PCP	3 bits	Priorité de 0 à 7 (802.1p), pour la qualité de service
DEI	1 bit	Trame éligible au rejet en cas de congestion
VID	12 bits	Numéro de VLAN : 1 à 4 094 (0 et 4 095 réservés)

La trame maximale passe de 1 518 à 1 522 octets ; les équipements le gèrent nativement, le MTU IP reste de 1 500.

Ports d'accès et trunks.

- Un **port d'accès** appartient à un seul VLAN. Il reçoit et émet des trames sans étiquette ; le switch ajoute et retire l'étiquette en interne. La machine raccordée ignore tout du VLAN.
- Un **trunk** transporte plusieurs VLAN, chaque trame portant son étiquette. On l'utilise entre switches, vers un hyperviseur ou vers un routeur.
- Le **VLAN natif** d'un trunk circule sans étiquette.

Attention. Un VLAN natif utilisé pour du trafic utilisateur permet l'attaque par double étiquetage (*VLAN hopping*) : un attaquant envoie une trame portant deux étiquettes, le premier switch retire celle du VLAN natif et le second livre la trame dans le VLAN visé. La parade : un VLAN natif inutilisé, ou l'étiquetage forcé du VLAN natif.

Sous Linux et Open vSwitch.

Besoin	Linux	Open vSwitch
Port d'accès	`bridge vlan add vid 10 pvid untagged dev <port>`	`ovs-vsctl set port <port> tag=10`
Trunk	`bridge vlan add vid 10-30 dev <port>`	`ovs-vsctl set port <port> trunks=10,20,30`
Sous-interface étiquetée	`ip link add link eth0 name eth0.10 type vlan id 10`	Port `internal` avec `tag=10`
Mode du port	—	`vlan_mode=access`, `trunk`, `native-tagged`, `native-untagged`

Limite. 4 094 VLAN suffisent à un site, pas à un cloud multi-locataire. Surtout, un VLAN doit être configuré sur tous les switches qu'il traverse, ce qui recrée de grands domaines de niveau 2.

6.3 QinQ (IEEE 802.1ad)

Le QinQ empile deux étiquettes : une étiquette de service (S-tag, TPID 0x88A8) ajoutée par l'opérateur autour de l'étiquette client (C-tag, 0x8100). Chaque client conserve ses 4 094 VLAN à l'intérieur de son S-tag. L'usage est surtout opérateur : transporter les VLAN de sites clients à travers un réseau métropolitain.

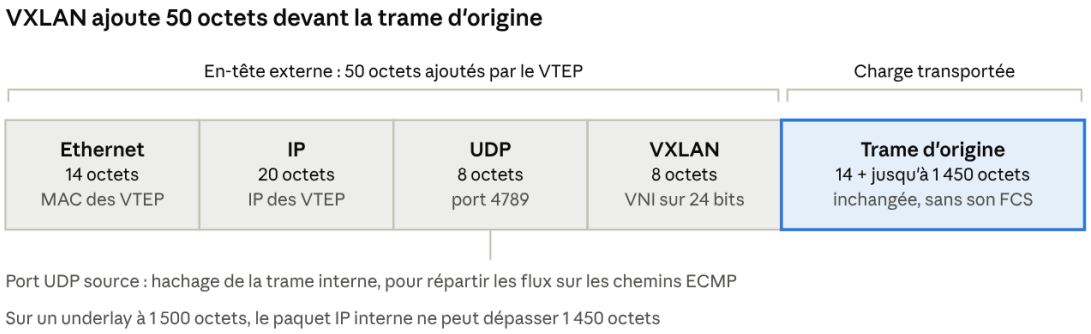
6.4 VXLAN (RFC 7348)

VXLAN (*Virtual eXtensible LAN*) transporte des trames Ethernet dans des datagrammes UDP. C'est l'encapsulation la plus répandue des fabrics de datacenter et de nombreux plugins Kubernetes.

La pile d'en-têtes. De l'extérieur vers l'intérieur :

En-tête	Taille (IPv4)	Contenu notable
Ethernet externe	14 octets	MAC des VTEP ou des routeurs de l'underlay
IP externe	20 octets (40 en IPv6)	Adresse du VTEP source et du VTEP destination
UDP externe	8 octets	Port de destination 4789 ; port source calculé par hachage de la trame interne
VXLAN	8 octets	Drapeau I à 1, VNI sur 24 bits, champs réservés
Trame Ethernet interne	14 octets + données	Trame d'origine, sans son FCS

Le surcoût est donc de **50 octets** en IPv4 et de 70 octets en IPv6, plus 4 octets si l'underlay est lui-même étiqueté.



encapsulation VXLAN sur IPv4 · 4 en-têtes ajoutés

L'underlay ne lit que les quatre en-têtes externes ; la trame d'origine, intacte, n'est lue qu'après décapsulation par le VTEP de destination.

Pourquoi UDP ? UDP traverse tous les réseaux IP et se répartit naturellement en ECMP. Le VTEP choisit le port source à partir d'un hachage des en-têtes internes : deux flux internes différents donnent deux ports sources différents, donc potentiellement deux chemins différents dans la fabric. C'est l'*entropie* de l'encapsulation.

Le VNI. Ses 24 bits offrent 16 777 216 identifiants, contre 4 094 VLAN. Le VNI sépare les segments : un VTEP ne livre une trame qu'aux ports associés au même VNI.

Le trafic BUM. Pour une diffusion ou une destination inconnue, le VTEP doit atteindre tous les VTEP du segment. Trois méthodes :

1. **Multicast** : chaque VNI est associé à un groupe multicast de l'underlay. Élégant mais exigeant : l'underlay doit router le multicast.
2. **Réplication en tête** (*head-end replication*) : le VTEP envoie une copie unicast à chaque VTEP de sa liste. Simple, mais coûteux en bande passante si les VTEP sont nombreux.
3. **Plan de contrôle** : un protocole (EVPN) ou un contrôleur distribue la liste des VTEP et les adresses MAC, ce qui réduit fortement l'inondation (chapitre 7).

L'apprentissage. Sans plan de contrôle, le VTEP apprend comme un switch : en recevant une trame encapsulée, il note que la MAC source interne se trouve derrière l'adresse IP du VTEP émetteur.

Sous Linux.

```
# VTEP natif : VNI 5010, réplication vers un VTEP distant
sudo ip link add vx5010 type vxlan id 5010 dstport 4789 local 192.168.50.11 dev ens4
sudo bridge fdb append 00:00:00:00:00:00 dev vx5010 dst 192.168.50.12   # liste de
réplication
bridge fdb show dev vx5010                                              # MAC
apprises et VTEP associés
```

Attention. Sans `dstport`, le noyau Linux utilise par compatibilité le port 8472, antérieur à l'attribution officielle du 4789. Flannel l'utilise par défaut. Deux VTEP réglés sur des ports différents ne communiquent pas, et les règles de pare-feu doivent ouvrir le bon port.

6.5 Geneve (RFC 8926)

Geneve (*Generic Network Virtualization Encapsulation*) est l'encapsulation extensible choisie par le groupe NVO3 de l'IETF. Elle reprend le principe de VXLAN (UDP, port 6081, VNI de 24 bits) et ajoute deux éléments :

- un champ **type de protocole**, qui indique ce que transporte le tunnel (une trame Ethernet, un paquet IPv4 ou IPv6) ;
- des **options** de longueur variable, au format TLV (classe, type, longueur, valeur), pour transporter des métadonnées.

OVN s'en sert pour transporter, avec chaque paquet, l'identifiant du port logique d'entrée et de sortie : le nœud de destination sait ainsi à quelle VM livrer sans refaire tout le traitement. Geneve est utilisé par OVN (OpenStack, OVN-Kubernetes), Antrea et, en option, Cilium.

6.6 NVGRE, GRE et les tunnels chiffrés

- **NVGRE** (RFC 7637) encapsule les trames dans GRE et porte un identifiant de 24 bits dans le champ clé. Sans UDP, l'entropie ECMP est faible. Promue par Microsoft pour Hyper-V, elle a cédé la place à VXLAN.
- **GRE** (RFC 2784, protocole IP 47) est un tunnel point à point générique ; sa variante GRETAP transporte des trames Ethernet.
- **IPsec** et **WireGuard** chiffrent le tunnel. Ils servent à relier des sites, et certains plugins Kubernetes (Calico, Cilium) proposent de chiffrer le trafic entre nœuds avec WireGuard.

VXLAN et Geneve ne chiffrent rien : ils isolent les segments mais laissent le contenu lisible sur l'underlay.

6.7 MPLS et Segment Routing

Dans les réseaux d'opérateurs et les WAN, l'acheminement par **étiquettes MPLS** (RFC 3031) domine : chaque paquet porte une étiquette de 20 bits, échangée à chaque saut, qui détermine son chemin. MPLS porte les VPN de niveau 3 (RFC 4364) et de niveau 2.

Le **Segment Routing** (RFC 8402) simplifie MPLS : la source inscrit dans le paquet la liste des segments à parcourir. Il existe en deux variantes, sur étiquettes MPLS (SR-MPLS) ou sur IPv6 (SRv6, RFC 8754 et 8986), où chaque segment est une adresse IPv6. Il permet une ingénierie de trafic pilotée par un contrôleur, sans état dans les routeurs intermédiaires.

6.8 Comparaison

Encapsulation	Norme	Transport	Identifiant	Surcoût (IPv4)	Extensible	Usage typique
VLAN	IEEE 802.1Q	Ethernet	12 bits	4 octets	Non	LAN, trunk vers les hyperviseurs
QinQ	IEEE 802.1ad	Ethernet	2 × 12 bits	8 octets	Non	Transport de VLAN clients
VXLAN	RFC 7348	UDP 4789	24 bits	50 octets	Non	Fabrics EVPN, Flannel, Calico
Geneve	RFC 8926	UDP 6081	24 bits + options	50 octets + options	Oui	OVN, Antrea, Cilium
NVGRE	RFC 7637	GRE	24 bits	42 octets	Non	Hyper-V, usage historique
MPLS, SR-MPLS	RFC 3031, 8402	Étiquettes	20 bits par étiquette	4 octets par étiquette	—	WAN, VPN opérateur
SRv6	RFC 8754, 8986	IPv6	Adresse IPv6 par segment	40 octets, plus 8 + 16 par segment avec un en-tête SRH	Oui	WAN, ingénierie de trafic

6.9 Le MTU, première cause de pannes des overlays

Une trame interne de 1 500 octets de données encapsulée en VXLAN produit un paquet IP externe de 1 550 octets. Si l'underlay est limité à 1 500 octets, ce paquet est fragmenté ou perdu.

Charge utile	Calcul	MTU interne maximal sur un underlay à 1 500
VXLAN sur IPv4	1 500 − 20 (IP) − 8 (UDP) − 8 (VXLAN) − 14 (Ethernet interne)	1 450 octets
VXLAN sur IPv6	1 500 − 40 − 8 − 8 − 14	1 430 octets
Geneve sans option sur IPv4	Comme VXLAN	1 450 octets, moins la taille des options
WireGuard sur IPv4	1 500 − 20 − 8 − 32	1 440 octets

Le symptôme. Le ping passe, la connexion SSH s'ouvre, puis un transfert de fichier ou une page web lourde se bloque. Les petits paquets traversent, les gros sont perdus. Si les messages ICMP « fragmentation nécessaire » sont filtrés, l'émetteur n'est jamais prévenu.

Diagnostiquer.

```
ping -M do -s 1422 10.10.10.2   # 1 422 + 28 = paquet IP de 1 450 octets, interdiction de fragmenter
ping -M do -s 1472 10.10.10.2   # 1 500 octets : doit échouer sur un overlay non ajusté
tracepath 10.10.10.2            # affiche le MTU de chemin découvert
```

Corriger. Trois options, à choisir une fois pour toutes à la conception :

1. Relever le MTU de l'underlay à 1 550 octets au moins, idéalement 9 000 ou plus ; les machines gardent 1 500.
2. Abaisser le MTU des interfaces virtuelles à 1 450, ce que font par défaut la plupart des plugins Kubernetes et OpenStack.
3. En dernier recours, ajuster la taille des segments TCP au passage (*MSS clamping*), par exemple avec nftables : `tcp flags syn tcp option maxseg size set rt mtu`.

En TP. Au TP2-B, la capture `tcpdump udp port 4789` montre l'en-tête externe et le VNI, et l'expérience `ping -M do` met en évidence la limite de 1 450 octets.

À retenir. 802.1Q : 4 octets, 12 bits, 4 094 VLAN. VXLAN : MAC dans UDP 4789, VNI de 24 bits, 50 octets de surcoût, entropie par le port source. Geneve : même principe, extensible par options. Tout overlay impose de traiter le MTU dès la conception.

Chapitre 7 — Le plan de contrôle des overlays

Un VTEP doit savoir quels autres VTEP participent à un segment et derrière lequel se trouve chaque machine. Il peut l'apprendre en observant le trafic, le recevoir d'un protocole de routage (BGP EVPN) ou le recevoir d'un contrôleur. Ce chapitre compare ces trois réponses. Module 2 de la formation.

7.1 Ce que le VTEP doit savoir

Pour livrer une trame destinée à la MAC `00:00:00:00:00:02` du segment 5010, le VTEP a besoin de deux informations :

1. **l'appartenance** : quels VTEP hébergent des machines du segment 5010, pour y envoyer le trafic BUM ;
2. **la localisation** : derrière quel VTEP (quelle adresse IP de l'underlay) se trouve la MAC de destination, et éventuellement quelle adresse IP lui correspond.

C'est le rôle du plan de contrôle de l'overlay. Les données circulent dans les tunnels ; les informations de localisation doivent circuler autrement, ou être devinées.

7.2 Inondation et apprentissage

La méthode d'origine de VXLAN n'a pas de plan de contrôle : le VTEP apprend dans le plan de données, comme un switch.

- Une trame vers une MAC inconnue est envoyée à tous les VTEP du segment, par multicast ou par réplication.
- À la réception, chaque VTEP note le couple (MAC source interne, IP du VTEP émetteur).
- Les requêtes ARP sont diffusées à tous les VTEP du segment.

C'est simple et sans dépendance, mais cela passe mal à l'échelle : l'inondation croît avec le nombre de VTEP et de machines, et une machine déplacée reste associée à son ancien VTEP tant que l'entrée n'a pas expiré. C'est la méthode du TP2-B : la liste de réplication est écrite à la main.

7.3 BGP EVPN

EVPN (*Ethernet VPN*) utilise BGP pour annoncer les adresses MAC et IP des machines et l'appartenance des VTEP aux segments. Défini d'abord pour MPLS (RFC 7432, 2015), il a été étendu aux overlays VXLAN et Geneve (RFC 8365). C'est aujourd'hui le plan de contrôle standard des fabrics de datacenter, pris en charge par tous les grands constructeurs et, sous Linux, par FRRouting.

Fonctionnement. Chaque VTEP ouvre une session BGP, en général vers les spines qui jouent le rôle de réflecteurs de routes, dans la famille d'adresses L2VPN EVPN. Il annonce ce qu'il connaît localement et apprend ce que les autres annoncent.

Type de route	Nom	Ce qu'elle annonce
1	Ethernet Auto-Discovery	Rattachement d'un segment Ethernet multi-attaché ; accélère la convergence
2	MAC/IP Advertisement	Une adresse MAC, et souvent son adresse IP, derrière tel VTEP
3	Inclusive Multicast Ethernet Tag	La participation d'un VTEP à un segment : construit la liste de réplication BUM
4	Ethernet Segment	Élection du VTEP chargé du BUM sur un lien multi-attaché
5	IP Prefix (RFC 9136)	Un préfixe IP dans une VRF : routage de niveau 3 entre segments et vers l'extérieur

Deux attributs organisent les annonces : le **Route Distinguisher** rend unique une route d'un segment ou d'une VRF, et le **Route Target** indique quels VTEP doivent l'importer.

Ce qu'EVPN apporte.

- **Suppression de l'inondation ARP** : connaissant le couple MAC/IP par les routes de type 2, le VTEP répond lui-même aux requêtes ARP de ses machines.
- **Mobilité** : quand une VM migre, le nouveau VTEP annonce sa MAC avec un numéro de séquence plus élevé, et tous les VTEP mettent à jour leur table immédiatement.
- **Multi-attachement** : un serveur raccordé à deux leafs utilise les deux liens en actif, sans protocole propriétaire de type MLAG.
- **Routage distribué** : les routes de type 5 et les VRF permettent le routage inter-segments sur chaque leaf.

Pour aller plus loin. Exemple indicatif de configuration FRRouting d'un leaf : session BGP non numérotée vers un spine et annonce de tous les VNI locaux.

```
router bgp 65011
 neighbor fabric peer-group
 neighbor fabric remote-as external
 neighbor swp51 interface peer-group fabric
 address-family l2vpn evpn
  neighbor fabric activate
  advertise-all-vni
 exit-address-family
```

7.4 Le cadre NVO3 de l'IETF

Le groupe de travail NVO3 (*Network Virtualization Overlays*) de l'IETF a défini un vocabulaire et une architecture communs à tous les overlays (RFC 7364 pour la problématique, RFC 7365 pour le cadre, RFC 8014 pour l'architecture).

Terme NVO3	Définition	Équivalent VXLAN/EVPN	Équivalent OVN
Tenant System (TS)	Machine d'un locataire	VM, conteneur, serveur	VM, pod
Virtual Network (VN)	Réseau virtuel d'un locataire	Segment, VNI	Switch logique
NVE	Fonction qui encapsule et décapsule	VTEP	Open vSwitch du nœud
VAP	Point d'accès du TS au VN	Port d'accès, VLAN local	Port logique
NVA	Autorité qui connaît la localisation des TS et renseigne les NVE	Réflecteurs BGP EVPN	Bases OVN et ovn-northd

Ce cadre permet de comparer des solutions très différentes avec les mêmes mots : une fabric EVPN et OVN réalisent les mêmes rôles par des moyens distincts.

7.5 Le contrôleur comme plan de contrôle

Dans un cloud, l'orchestrateur sait déjà où se trouve chaque VM : il vient de la créer sur tel hôte, avec telle adresse. Il est inutile de l'apprendre par le trafic ou de l'annoncer par BGP ; un **contrôleur** peut programmer directement chaque VTEP.

C'est l'approche d'OVN, de VMware NSX et de nombreuses solutions de cloud :

- le contrôleur reçoit de l'orchestrateur (Neutron, Kubernetes) la description logique : segments, ports, adresses ;
- il calcule pour chaque hôte les tunnels à établir et les règles à installer ;
- l'agent local programme Open vSwitch, souvent en OpenFlow, y compris des règles qui répondent aux requêtes ARP sans aucune inondation.

Le schéma OVSDB **hardware_vtep** étend ce modèle aux switches physiques : un contrôleur y écrit les associations MAC-VTEP, que le switch applique.

L'avantage est la précision et la rapidité : aucune inondation, mise à jour immédiate lors d'une migration. La contrepartie est la dépendance au contrôleur, qui doit être hautement disponible (chapitre 10).

7.6 Comparaison

Critère	Inondation et apprentissage	BGP EVPN	Contrôleur
Source de l'information	Le trafic observé	Les annonces des VTEP	L'orchestrateur
Trafic BUM	Maximal	Réduit (suppression ARP)	Quasi nul
Mobilité	Lente (expiration)	Immédiate (numéro de séquence)	Immédiate
Interopérabilité	Totale	Standard multi-constructeurs	Souvent liée à une solution
Dépendance	Aucune	Sessions BGP	Disponibilité du contrôleur
Usage typique	Labs, petits déploiements	Fabrics de datacenter	Clouds OpenStack, Kubernetes, NSX

À retenir. Un overlay a besoin d'un plan de contrôle pour savoir qui est où. EVPN le fait par BGP, avec la route de type 2 pour les MAC/IP et la route de type 3 pour l'appartenance aux segments. Un contrôleur le fait à partir de la connaissance de l'orchestrateur. Le cadre NVO3 donne le vocabulaire commun : NVE, NVA, VN, TS.

Chapitre 8 — La normalisation des réseaux virtualisés

Les normes permettent de mélanger les fournisseurs, de remplacer une brique sans tout refaire et d'automatiser avec les mêmes outils. Ce chapitre présente les organismes, les textes à connaître, le cadre d'architecture NFV de l'ETSI et les interfaces de gestion normalisées. Module 2 de la formation.

8.1 Pourquoi normaliser

- **Interopérabilité** : un VTEP Linux dialogue avec un VTEP matériel, un contrôleur avec des switches de plusieurs marques.
- **Substituabilité** : une brique se remplace sans réécrire le reste, ce qui limite la dépendance à un fournisseur.
- **Automatisation** : un modèle de données commun permet d'écrire un seul outil pour tout le parc.
- **Durabilité et achat public** : un cahier des charges peut exiger une norme plutôt qu'un produit.

On distingue les normes **de droit**, publiées par un organisme reconnu (IEEE, IETF, ETSI, UIT-T), et les normes **de fait**, imposées par l'usage d'une implémentation ou d'un consortium (Open vSwitch, OpenConfig, CNI).

8.2 Les organismes

Organisme	Nature	Périmètre	Exemples de textes
IEEE 802.1	Association professionnelle, groupes de travail	Ponts et réseaux locaux, niveau 2	802.1Q, 802.1ad, 802.1AX, 802.1Qbg
IETF	Communauté ouverte, travail par groupes, consensus	IP, routage, tunnels, gestion	RFC 7348, 8926, 7432, 6241, 7950, 8040
IRTF	Recherche associée à l'IETF	Prospective	RFC 7426 (terminologie SDN), issue du groupe SDNRG
ONF, projets transférés depuis à la Linux Foundation	Consortium d'industriels et d'opérateurs	SDN	OpenFlow 1.0 à 1.5.1, OF-Config
ETSI ISG NFV	Groupe de spécification de l'institut européen	Virtualisation des fonctions réseau	GS NFV 002, NFV-MAN 001, séries IFA et SOL
UIT-T (commission d'études 13)	Agence des Nations unies	Cadres de référence des réseaux futurs	Y.3011 (virtualisation des réseaux), Y.3300 (SDN)
3GPP	Partenariat des	Réseaux mobiles	Cœur 5G conçu pour le cloud et

Organisme	Nature	Périmètre	Exemples de textes
	organismes télécoms		la virtualisation
MEF	Forum industriel	Services Ethernet opérateurs, API d'orchestration	Services Carrier Ethernet, API LSO
CNCF, Linux Foundation	Fondations open source	Implémentations et spécifications de fait	Spécification CNI, Kubernetes, OpenDaylight, ONAP
OpenConfig	Groupe d'opérateurs	Modèles de gestion communs	Modèles YANG OpenConfig, gNMI

Comment naît une RFC. Un auteur publie un *Internet-Draft*. Un groupe de travail l'adopte, l'améliore et le valide par « consensus approximatif et code qui tourne » (*rough consensus and running code*). L'IESG l'approuve et il devient une RFC, dans une catégorie qui indique sa portée : norme (*Standards Track*), information (*Informational*), expérimentation (*Experimental*) ou bonne pratique (*BCP*). Une RFC n'est jamais modifiée : on publie une nouvelle RFC qui la met à jour ou la remplace. VXLAN (RFC 7348) est ainsi une RFC informative, ce qui ne l'a pas empêchée de devenir universelle.

Les groupes de travail IETF à suivre : **NVO3** (overlays), **BESS** (EVPN et VPN), **NETCONF** et **NETMOD** (gestion et modèles), **SPRING** (Segment Routing), **SFC** (chaînage de services).

8.3 Les normes de la virtualisation réseau, par thème

Thème	Normes	Chapitre
Segmentation de niveau 2	IEEE 802.1Q (VLAN), 802.1ad (QinQ), 802.1AX (agrégation de liens), 802.1Qbg (raccordement des machines virtuelles)	6
Overlays	RFC 7348 (VXLAN), 8926 (Geneve), 7637 (NVGRE) ; cadre NVO3 : RFC 7364, 7365, 8014	6, 7
Plan de contrôle des overlays	RFC 7432 (EVPN), 8365 (EVPN pour les overlays), 9135 (routage intégré), 9136 (routes de préfixes)	7
Datacenter	RFC 7938 (BGP dans les grands datacenters)	5
SDN	RFC 7426 (couches et terminologie), spécifications OpenFlow, UIT-T Y.3300	9, 14
NFV	ETSI GS NFV 002 (architecture), NFV-MAN 001 (MANO), séries IFA (interfaces) et SOL (API, descripteurs)	4, 8
Chaînage de services	RFC 7665 (architecture), 8300 (en-tête NSH)	4
Gestion	RFC 6241 (NETCONF), 6020 et 7950 (YANG), 8040	8

Thème	Normes	Chapitre
	(RESTCONF), 7047 (OVSDB) ; gNMI (OpenConfig)	

8.4 Le cadre d'architecture ETSI NFV

Le document ETSI GS NFV 002 découpe un système NFV en trois domaines.

Les VNF et leurs gestionnaires d'éléments. Les VNF sont les fonctions réseau logicielles. Chacune peut avoir un gestionnaire d'élément (EM) qui assure sa configuration métier, ses alarmes et ses compteurs, comme pour un équipement classique.

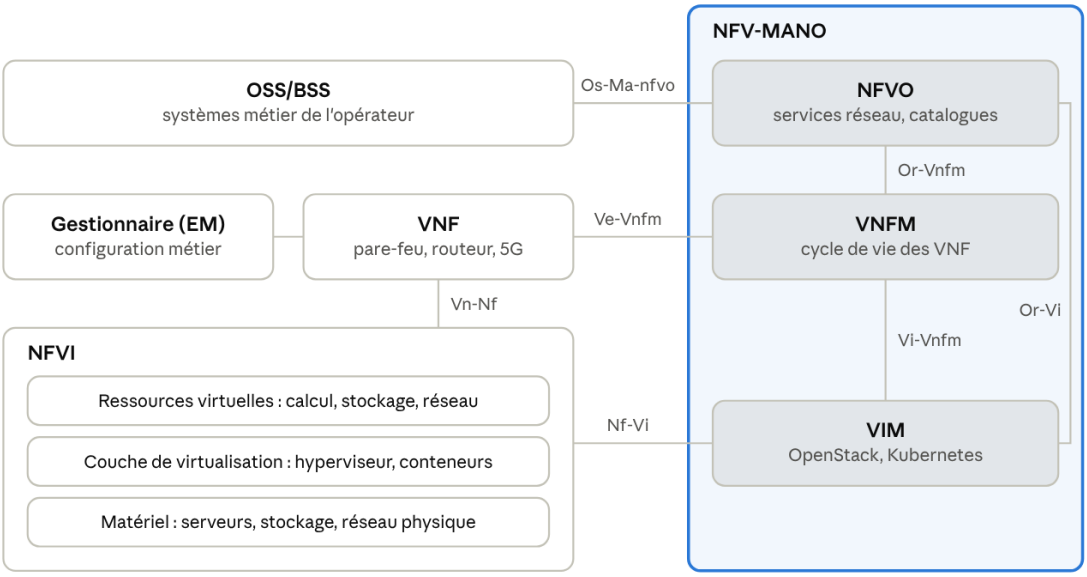
La NFVI. L'infrastructure comprend le matériel (calcul, stockage, réseau), la couche de virtualisation (hyperviseur ou moteur de conteneurs) et les ressources virtuelles qu'elle expose.

NFV-MANO. La gestion et l'orchestration se répartissent en trois blocs fonctionnels :

Bloc	Rôle	Exemples réels
NFVO (orchestrateur NFV)	Compose les services réseau à partir de VNF, gère les catalogues et la répartition des ressources entre sites	ETSI OSM, ONAP
VNFM (gestionnaire de VNF)	Gère le cycle de vie de chaque VNF : instanciation, mise à l'échelle, réparation, arrêt	Générique (OpenStack Tacker) ou fourni par l'éditeur de la VNF
VIM (gestionnaire d'infrastructure virtualisée)	Pilote les ressources de la NFVI : VM, réseaux, stockage	OpenStack, Kubernetes

Le système métier de l'opérateur (OSS/BSS) commande l'ensemble par l'orchestrateur. Des référentiels conservent les catalogues de services et de VNF, les instances en cours et l'état des ressources.

La NFV sépare les fonctions, l'infrastructure et leur orchestration



cadre d'architecture ETSI NFV · 3 domaines, 7 points de référence

À gauche, ce qui rend le service (fonctions et infrastructure) ; à droite, ce qui l'orchestre. Chaque trait est un point de référence normalisé, détaillé dans le tableau ci-dessous.

Les points de référence. L'ETSI nomme chaque interface entre blocs ; les séries IFA les décrivent et les séries SOL en font des API REST.

Point de référence	Entre	Usage
Os-Ma-nfvo	OSS/BSS et NFVO	Commander un service réseau
Or-Vnfm	NFVO et VNFM	Piloter le cycle de vie des VNF
Or-Vi	NFVO et VIM	Réserver et allouer des ressources
Vi-Vnfm	VNFM et VIM	Créer les VM ou conteneurs d'une VNF
Ve-Vnfm	VNFM et VNF ou EM	Configurer et surveiller une VNF
Nf-Vi	NFVI et VIM	Commander l'infrastructure
Vn-Nf	VNF et NFVI	Ressources virtuelles consommées par la VNF

Les VNF et les services sont décrits par des descripteurs normalisés : en TOSCA (ETSI GS NFV-SOL 001) ou en YANG (SOL 006).

8.5 Les interfaces de gestion normalisées

La configuration des équipements a longtemps reposé sur la ligne de commande, propre à chaque constructeur, et sur SNMP pour la supervision. La gestion pilotée par les modèles (*model-driven management*) les remplace progressivement.

Protocole	Norme	Transport et format	Rôle
SNMP	RFC 3411 et suivantes	UDP, MIB	Supervision par interrogation, peu utilisé pour configurer
NETCONF	RFC 6241	SSH (port 830), XML	Configuration transactionnelle
YANG	RFC 6020, 7950	Langage de modélisation	Décrit la structure des données de configuration et d'état
RESTCONF	RFC 8040	HTTPS, JSON ou XML	Accès REST aux données YANG
gNMI, gNOI	OpenConfig	gRPC, Protocol Buffers	Configuration et télémétrie en flux continu
OVSDB	RFC 7047	JSON-RPC sur TCP (port 6640) ou socket local	Configuration d'Open vSwitch

NETCONF apporte ce qui manquait à la ligne de commande : des transactions. On modifie une configuration candidate (`edit-config`), on la valide, puis on l'applique d'un bloc (`commit`). En cas d'erreur, rien n'est appliqué. Les opérations `get`, `get-config`, `lock` et `unlock` complètent le jeu.

YANG décrit les données manipulées. Un module YANG définit des conteneurs, des listes et des feuilles typées. Les modèles proviennent de l'IETF (par exemple `ietf-interfaces`, RFC 8343), d'OpenConfig ou des constructeurs.

```
container interfaces {
  list interface {
    key "name";
    leaf name    { type string; }
    leaf enabled { type boolean; default true; }
    leaf mtu     { type uint16; }
  }
}
```

RESTCONF expose ces mêmes données en REST : `GET https://<équipement>/restconf/data/ietf-interfaces:interfaces` renvoie la liste des interfaces en JSON. Les contrôleurs OpenDaylight et ONOS savent dialoguer en NETCONF avec les équipements.

La télémétrie en flux (gNMI) inverse la logique de SNMP : au lieu d'interroger l'équipement toutes les minutes, on s'abonne et il envoie ses compteurs en continu, toutes les secondes s'il le faut.

8.6 Les trois niveaux d'interopérabilité

Niveau	Ce qui doit concorder	Normes	Exemple
Données	Le format des paquets	802.1Q, VXLAN,	Un VTEP Linux et un leaf matériel

Niveau	Ce qui doit concorder	Normes	Exemple
		Geneve, MPLS	échangent du VXLAN
Contrôle	Le protocole d'échange d'informations d'acheminement	BGP EVPN, OpenFlow, OSPF	Des leafs de deux constructeurs dans la même fabric EVPN
Gestion	Le protocole et le modèle de configuration	NETCONF, YANG, gNMI, OVSDB	Un seul outil Ansible ou un seul contrôleur pour tout le parc

L'interopérabilité annoncée ne garantit pas l'interopérabilité réelle : options facultatives, extensions propriétaires et versions différentes d'une même norme créent des écarts. Les campagnes de tests croisés (ETSI Plugtests, tests de conformité OpenFlow) et un essai en laboratoire avant tout déploiement restent indispensables.

8.7 Normes et open source

Dans les réseaux virtualisés, une implémentation open source sert souvent de référence : Open vSwitch pour OpenFlow et OVSDB, FRRouting pour EVPN sous Linux, le noyau Linux pour VXLAN et Geneve. La norme dit ce qui doit être fait ; l'implémentation de référence montre comment, et sert de base de test aux autres. Le risque inverse existe aussi : une implémentation dominante peut imposer ses choix sans passer par un organisme.

À retenir. IEEE pour le niveau 2, IETF pour IP, les tunnels et la gestion, ONF pour OpenFlow, ETSI pour la NFV. L'architecture NFV distingue VNF, NFVI et MANO (NFVO, VNFM, VIM). La gestion moderne repose sur des modèles YANG transportés par NETCONF, RESTCONF ou gNMI.

Chapitre 9 — Les principes du SDN

Le SDN sépare la décision de l'exécution : un contrôleur logiquement centralisé calcule le comportement du réseau et le programme dans les équipements par des interfaces ouvertes. Ce chapitre en retrace l'origine, définit les plans et les interfaces, et distingue les différentes façons de le mettre en œuvre. Module 3 de la formation.

9.1 Le constat de départ

À la fin des années 2000, un équipement réseau est une boîte fermée : le constructeur fournit ensemble le circuit de commutation, le système d'exploitation et les fonctions. Trois difficultés en découlent.

- **L'innovation est lente.** Une nouvelle fonction suppose que le constructeur l'implémente, souvent après des années de normalisation.
- **Le contrôle est distribué et complexe.** Chaque boîtier calcule son comportement avec des protocoles distribués (Spanning Tree, OSPF, BGP) dont les interactions sont difficiles à prévoir.
- **La configuration est locale.** Une politique globale (« la recette ne parle pas à la production ») doit être traduite à la main en dizaines de configurations d'équipements, avec le risque d'incohérences.

Les chercheurs parlent alors d'« ossification » d'Internet : il est devenu impossible d'y expérimenter de nouvelles idées en conditions réelles.

9.2 Une brève histoire

Année	Jalon	Apport
2005	Projet 4D	Proposition de séparer décision, dissémination, découverte et données
2007	Ethane (Stanford)	Un contrôleur central applique la politique de sécurité d'un réseau d'entreprise sur des switches simples
2008	Article OpenFlow, contrôleur NOX	Un protocole ouvert pour programmer les tables des switches ; le premier contrôleur
2011	Open Networking Foundation	Les grands exploitants (Google, Microsoft, Facebook, Deutsche Telekom, Verizon, Yahoo) portent la normalisation d'OpenFlow
2012	Rachat de Nicira par VMware	La virtualisation réseau devient un produit (NSX)
2013	Publication de Google B4	Un WAN mondial piloté par SDN, avec des liens utilisés près de leur capacité
2014	OpenDaylight, ONOS, P4	Contrôleurs open source matures ; langage de programmation du plan de données
Depuis 2016	OVN, SD-WAN, réseaux à intentions	Le principe SDN est intégré aux clouds, aux WAN d'entreprise et aux outils d'automatisation

L'article fondateur d'OpenFlow, signé notamment par Nick McKeown, s'adressait aux campus universitaires : permettre aux chercheurs d'expérimenter sur le réseau de production sans le perturber. Le SDN est sorti des campus quand les exploitants de datacenters et de clouds y ont vu un moyen de maîtriser leur réseau à très grande échelle.

9.3 Les plans d'un réseau

Tout équipement réseau remplit trois fonctions, appelées plans.

Plan	Rôle	Rythme	Exemples
Données (*data plane*, *forwarding plane*)	Acheminer chaque paquet selon les tables	Nanosecondes, à la vitesse de la ligne	Recherche dans la table MAC, la table de routage, la table de flux
Contrôle (*control plane*)	Calculer le contenu des tables	Millisecondes à secondes	OSPF, BGP, Spanning Tree, contrôleur SDN
Gestion	Configurer, superviser,	Minutes à jours	CLI, SNMP, NETCONF

Plan	Rôle	Rythme	Exemples
(*management plane*)	mettre à jour		

Dans un réseau classique, les trois plans vivent dans chaque boîtier. Le SDN extrait le plan de contrôle des équipements et le regroupe dans le contrôleur.

Le vocabulaire de la RFC 7426. Publiée en 2015 par l'IRTF, cette RFC pose une terminologie commune. Elle distingue cinq plans et quatre couches d'abstraction.

Élément	Définition
Plan d'acheminement (*forwarding plane*)	Traite les paquets
Plan opérationnel (*operational plane*)	État de l'équipement : ports actifs, ressources, compteurs
Plan de contrôle	Décide de l'acheminement et le programme
Plan de gestion	Surveille, configure et entretient l'équipement
Plan applicatif	Services et applications qui utilisent le réseau et le définissent
DAL (*Device and resource Abstraction Layer*)	Abstraction des ressources de l'équipement, vue par les plans de contrôle et de gestion
CSAL et MSAL	Abstractions des services de contrôle et de gestion, offertes aux applications
NSAL (*Network Services Abstraction Layer*)	Abstraction des services réseau offerte au plan applicatif

Ce vocabulaire montre que le SDN ne se résume pas à OpenFlow : OpenFlow est une interface entre plan de contrôle et plan d'acheminement, NETCONF une interface entre plan de gestion et équipement.

9.4 Les quatre principes

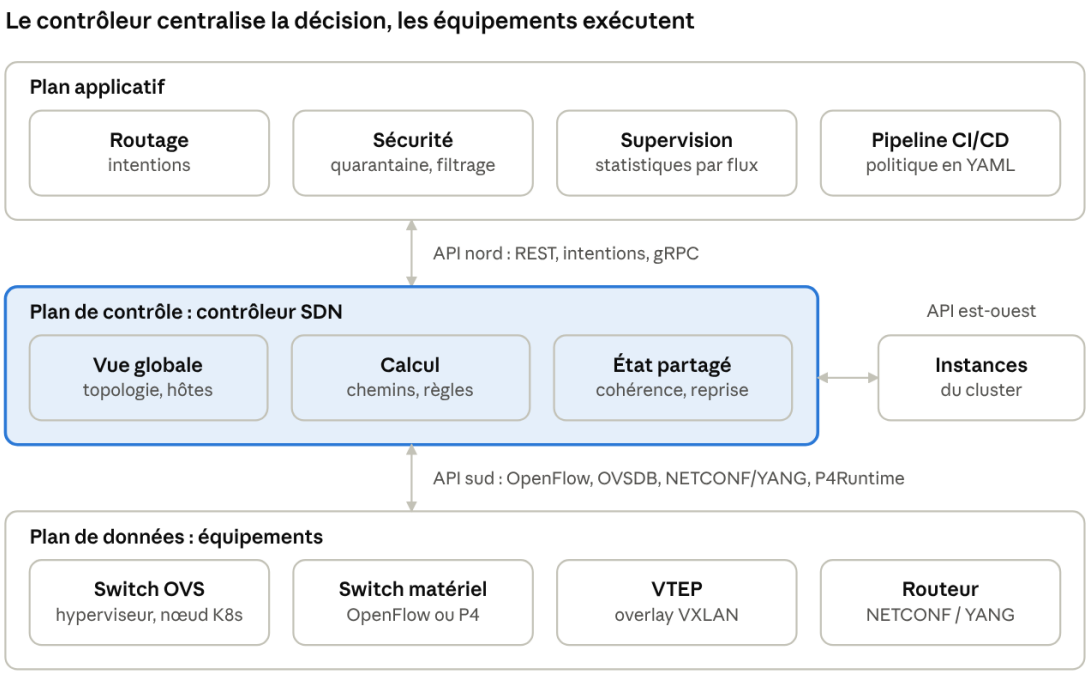
1. **Séparation** du plan de contrôle et du plan de données : les équipements exécutent, le contrôleur décide.
2. **Centralisation logique** du contrôle : le contrôleur a une vue globale du réseau. « Logique » car il est physiquement réparti sur plusieurs instances pour la disponibilité (chapitre 10).
3. **Abstraction** : les applications manipulent une représentation du réseau (un graphe de nœuds et de liens, des hôtes, des intentions) plutôt que des équipements et leurs syntaxes.
4. **Programmabilité** : tout s'exprime par des interfaces ouvertes, que l'on appelle depuis un programme.

L'analogie souvent citée est celle de l'informatique : on est passé de l'ordinateur central, où un fournisseur livrait matériel, système et applications, à un modèle en couches avec des interfaces

ouvertes. Le contrôleur joue le rôle d'un « système d'exploitation réseau », sur lequel s'exécutent des applications.

9.5 L'architecture et ses interfaces

L'architecture SDN compte trois couches : les applications, le contrôleur et les équipements. Elles communiquent par trois familles d'interfaces.



architecture SDN · 3 plans, 3 familles d'interfaces

En TP, Mininet et Open vSwitch jouent le plan de données, ONOS (TP3) puis os-ken (TP4) le contrôleur, et vos scripts le plan applicatif.

L'interface sud (*southbound*) relie le contrôleur aux équipements.

Protocole	Ce qu'il permet	Usage
OpenFlow	Programmer les tables de flux	Switches OpenFlow, Open vSwitch (chapitres 14 à 17)
OVSDB	Configurer Open vSwitch : bridges, ports, tunnels	OVN, contrôleurs de cloud
NETCONF, RESTCONF, gNMI	Configurer selon des modèles YANG	Équipements de constructeurs
P4Runtime	Programmer un plan de données défini en P4	Switches programmables (chapitre 18)

Protocole	Ce qu'il permet	Usage
BGP-LS	Collecter la topologie d'un réseau routé	WAN, ingénierie de trafic
PCEP	Imposer des chemins calculés (MPLS, Segment Routing)	WAN opérateur
CLI, SNMP	Piloter les équipements anciens par des pilotes dédiés	Transition

L'interface nord (*northbound*) relie les applications au contrôleur. Elle n'est pas normalisée : chaque contrôleur propose la sienne, en général une API REST, parfois gRPC, souvent enrichie d'un modèle d'intentions. C'est par elle que passent les scripts, les outils d'automatisation et les pipelines (TP3-B et TP3-C).

L'interface est-ouest relie entre elles les instances d'un même contrôleur, pour partager l'état et se répartir les équipements, ou des contrôleurs de domaines différents, pour se fédérer.

9.6 Les modèles de mise en œuvre

Modèle	Principe	Exemple
SDN « pur »	Tout le contrôle est centralisé ; les switches n'exécutent que les règles OpenFlow reçues	Faucet, labs ONOS, Google B4
SDN hybride	Les protocoles classiques restent en place ; le contrôleur intervient sur certaines fonctions (ingénierie de trafic, sécurité)	Contrôleur PCE sur un réseau MPLS
SDN par API	Le contrôleur configure les équipements par leurs API de gestion, sans toucher à leur plan de contrôle	OpenDaylight en NETCONF, outils d'automatisation réseau
SDN par overlay	Le contrôle porte sur les switches logiciels des hôtes ; l'underlay physique reste classique	OVN, VMware NSX

Le modèle par overlay est aujourd'hui le plus répandu dans les datacenters : c'est celui que les équipes système et DevOps rencontrent dans OpenStack et OpenShift.

9.7 Ce que le SDN n'est pas

- **Pas forcément OpenFlow.** OpenFlow est l'interface sud historique ; beaucoup de solutions SDN utilisent NETCONF, OVSDB ou des API propres.
- **Pas un point unique.** Un contrôleur de production est un cluster de plusieurs instances.
- **Pas la fin des protocoles distribués.** L'underlay d'une fabric reste routé en BGP ou OSPF, même sous un overlay piloté par contrôleur.
- **Pas automatiquement moins cher ni plus sûr.** Le gain dépend de l'automatisation réellement mise en place, et la centralisation crée une cible de choix (chapitre 13).

À retenir. Quatre principes : séparation, centralisation logique, abstraction, programmabilité. Trois interfaces : sud vers les équipements (OpenFlow, OVSDB, NETCONF, P4Runtime), nord vers les applications (REST, intentions), est-ouest entre contrôleurs. La RFC 7426 fixe le vocabulaire.

Chapitre 10 — Les contrôleurs SDN

Un contrôleur SDN est un logiciel distribué qui maintient une vue du réseau, calcule l'état voulu et le programme dans les équipements. Ce chapitre décrit son anatomie, présente les principales solutions (ONOS, OpenDaylight, os-ken, Faucet, OVN) et la façon de les rendre hautement disponibles. Module 3 de la formation.

10.1 L'anatomie d'un contrôleur

Quelle que soit la solution, on retrouve les mêmes composants.

Composant	Rôle
Pilotes sud (*providers*, *plugins*)	Parlent OpenFlow, OVSDB, NETCONF, P4Runtime ; traduisent les événements des équipements en événements internes
Inventaire	Tient la liste des équipements, de leurs ports et de leurs capacités
Topologie	Construit le graphe des liens entre équipements, à partir de la découverte (chapitre 11)
Suivi des hôtes	Localise chaque machine : adresse MAC, adresse IP, équipement et port de raccordement
Calcul de chemins	Trouve les chemins dans le graphe selon une métrique
Gestion des règles	Installe les règles, vérifie qu'elles sont bien présentes et les réinstalle si besoin
Service de paquets	Distribue aux applications les paquets remontés par les switches (PACKET_IN)
Magasin distribué	Réplique l'état entre les instances du cluster
Interfaces nord	API REST ou gRPC, ligne de commande, interface graphique
Applications	Routage, sécurité, supervision : la logique métier, chargée à la demande

Un contrôleur fonctionne par **événements** (un switch se connecte, un port tombe, un paquet remonte) et par **réconciliation** : il compare en permanence l'état voulu, issu des applications, et l'état constaté dans les équipements, puis corrige l'écart. C'est le même principe que les contrôleurs de Kubernetes.

10.2 ONOS

ONOS (*Open Network Operating System*) est un contrôleur open source écrit en Java, publié en 2014 par l'ON.Lab puis porté par l'ONF. Il vise les opérateurs : haute disponibilité, performances, abstraction des équipements. Sa dernière version à support long, la 2.7.0, date de janvier 2022 ; l'activité du projet a fortement ralenti depuis. Il reste l'un des meilleurs contrôleurs pour apprendre, grâce à son interface graphique, sa ligne de commande et son modèle d'intentions.

Architecture.

- Il s'exécute dans Apache Karaf, un conteneur OSGi : chaque application se charge et se décharge à chaud (`app activate`, `app deactivate`).
- Les **providers** forment la couche sud : OpenFlow, P4Runtime, NETCONF, gNMI, SNMP, entre autres.
- Le **cœur** est indépendant des protocoles. Il est découpé en sous-systèmes : Device, Link, Host, Topology, Path, FlowRule, Packet, Intent, Mastership, Cluster.
- Les **magasins distribués** reposent sur Atomix, qui fournit des structures de données répliquées avec le consensus Raft.
- Les **interfaces nord** : API REST, gRPC, ligne de commande Karaf, interface web.

Port	Usage
6653	OpenFlow (connexion des switches)
8181	Interface web et API REST (`/onos/ui`, `/onos/v1`)
8101	Ligne de commande par SSH
9876	Communication entre instances du cluster

Commande ONOS	Effet
`apps -a -s`	Applications actives
`devices`, `ports`, `links`, `hosts`	Inventaire, topologie, hôtes
`flows -s`	Règles installées, en résumé
`intents`, `add-host-intent`, `remove-intent`	Gestion des intentions
`paths <source> <destination>`	Chemins calculés entre deux équipements
`nodes`, `masters`	Instances du cluster, répartition des équipements

Applications utiles : `fwd` (acheminement réactif), `proxyarp` (réponses ARP par le contrôleur), `openflow` (pilotes OpenFlow), `segmentrouting` (fabric leaf-spine pilotée).

10.3 OpenDaylight

OpenDaylight (ODL), lancé en 2013 sous l'égide de la Linux Foundation, est la plateforme SDN open source la plus diffusée dans l'industrie. Elle est écrite en Java, s'exécute elle aussi dans Karaf et publie deux versions par an, nommées d'après les éléments chimiques (Titanium en août 2025, sur Java 21).

Son cœur est le **MD-SAL** (*Model-Driven Service Abstraction Layer*) : tout y est décrit par des modèles YANG, qui génèrent automatiquement le code d'accès, les magasins de données (configuration et état opérationnel) et les API RESTCONF. Ses projets phares sont NETCONF/RESTCONF, BGP-PCEP (topologie et chemins des WAN), OpenFlowPlugin et TransportPCE (réseaux optiques).

ODL est surtout employé comme plateforme d'orchestration multi-constructeurs par NETCONF, et comme base de produits commerciaux et d'autres projets open source (le contrôleur SDN d'ONAP en dérive).

10.4 Ryu et os-ken

Ryu, développé par NTT, est un framework Python pour écrire des contrôleurs. Son développement est figé ; **os-ken** en est le fork maintenu par la communauté OpenStack, qui l'utilise dans l'agent Open vSwitch de Neutron. Les deux partagent la même API.

Ce n'est pas un contrôleur clé en main mais une boîte à outils :

- prise en charge complète d'OpenFlow 1.0 à 1.5 et des extensions d'Open vSwitch ;
- bibliothèque d'analyse et de construction de paquets (Ethernet, ARP, IP, TCP, LLDP…) ;
- modèle par événements : une méthode décorée par `@set_ev_cls` est appelée à chaque événement du type indiqué ;
- applications fournies : switch apprenant, API REST de gestion des flux, découverte de topologie, locuteur BGP.

Il convient au prototypage, à l'enseignement et à des contrôleurs spécialisés de petite taille. Le chapitre 17 détaille l'anatomie d'une application os-ken.

10.5 Faucet

Faucet est un contrôleur OpenFlow de production, écrit en Python sur os-ken. Il ne se programme pas : il se configure par un fichier YAML qui décrit les switches, les VLAN, le routage et les listes de contrôle d'accès. Il expose ses métriques à Prometheus et a été testé sur des switches matériels de plusieurs constructeurs.

Exemple indicatif : deux zones et un filtrage recette vers production.

```
vlans:
  dev:
    vid: 10
  recette:
    vid: 20
acls:
  recette-sans-prod:
    - rule:
        dl_type: 0x800
        nw_dst: 10.10.30.0/24
```

```
        actions:
          allow: 0
    - rule:
        actions:
          allow: 1
dps:
  sw1:
    dp_id: 0x1
    hardware: "Open vSwitch"
    interfaces:
      1:
        native_vlan: dev
      2:
        native_vlan: recette
        acls_in: [recette-sans-prod]
```

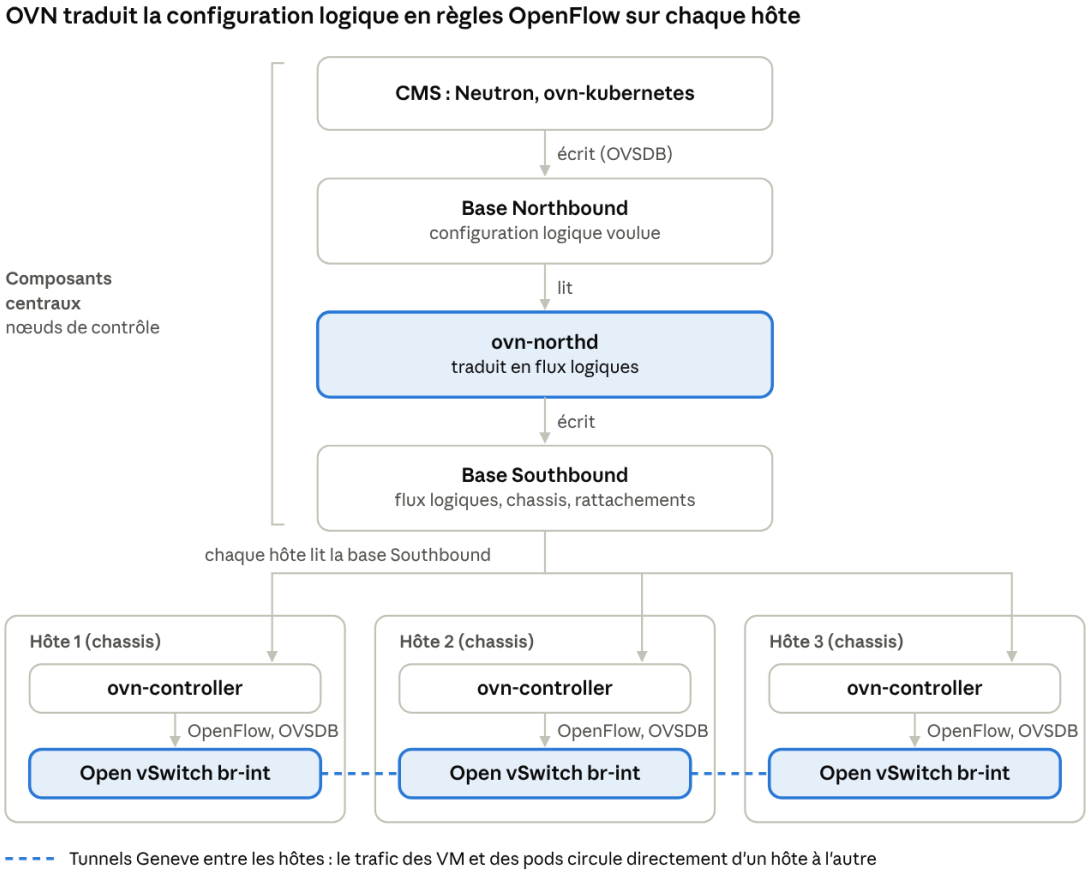
Faucet illustre bien le SDN déclaratif : la politique est un fichier versionné, le contrôleur la traduit en règles OpenFlow.

10.6 OVN

OVN (*Open Virtual Network*) est le projet de virtualisation réseau d'Open vSwitch, publié en 2016. C'est le SDN que la plupart des équipes système et DevOps exploitent sans le savoir : il porte le réseau d'OpenStack (pilote ML2/OVN) et celui d'OpenShift et d'autres distributions Kubernetes (OVN-Kubernetes).

OVN raisonne en objets logiques : switches logiques, routeurs logiques, ports, listes de contrôle d'accès, répartiteurs de charge, règles NAT. Il les réalise sur chaque hôte, appelé *chassis*.

Composant	Où	Rôle
Système de gestion du cloud (CMS)	Neutron, ovn-kubernetes	Traduit les objets du cloud en objets OVN
Base Northbound	Centrale (OVSDB)	Configuration logique voulue : switches, routeurs, ACL
`ovn-northd`	Central	Traduit la configuration logique en flux logiques
Base Southbound	Centrale (OVSDB)	Flux logiques, liste des chassis, rattachement des ports
`ovn-controller`	Chaque hôte	Lit la base Southbound et programme l'Open vSwitch local : règles OpenFlow sur le bridge `br-int`, tunnels Geneve vers les autres hôtes



architecture d'OVN · bases centrales, agent sur chaque hôte

Le contrôle descend des bases centrales vers les hôtes ; les données, elles, ne passent jamais par le centre : elles vont d'hôte à hôte dans les tunnels.

Tout est distribué : chaque hôte réalise localement la commutation, le routage entre segments, le filtrage à états (avec conntrack), la réponse ARP, le DHCP et une partie du NAT. Seul le trafic nord-sud passe par des hôtes passerelles.

Commande	Effet
`ovn-nbctl show`	Topologie logique : switches, routeurs, ports
`ovn-sbctl show`	Chassis et ports rattachés
`ovn-sbctl lflow-list`	Flux logiques calculés par `ovn-northd`
`ovn-trace`	Simule le parcours d'un paquet dans le réseau logique
`ovs-ofctl dump-flows br-int`	Règles OpenFlow réellement installées sur l'hôte

10.7 Les solutions commerciales et cloud natives

Solution	Modèle	Particularité
VMware NSX	Overlay Geneve porté par les hyperviseurs, gestionnaire en cluster	Pare-feu distribué, très répandu dans les datacenters d'entreprise
Cisco ACI	Fabric leaf-spine pilotée par un contrôleur (APIC)	Modèle de politique par groupes d'équipements et contrats
Juniper Apstra	Pilotage par intentions de fabrics multi-constructeurs	Validation continue de l'état réel par rapport à l'intention
Cilium	eBPF dans chaque nœud Kubernetes	L'API Kubernetes sert de source de vérité ; politiques de niveau 3 à 7
Calico	Routage IP et BGP, ou eBPF	Politiques réseau, intégration directe à la fabric par BGP

10.8 Haute disponibilité et cohérence

Un contrôleur unique serait un point de défaillance. En production, on déploie un **cluster** d'au moins trois instances.

- **Quorum.** Les magasins fondés sur Raft exigent qu'une majorité d'instances soit joignable : un cluster de trois instances tolère une panne, un cluster de cinq en tolère deux. Un nombre pair d'instances n'apporte rien.
- **Maîtrise des équipements.** Chaque switch est connecté à plusieurs instances, mais une seule en est maître ; les autres sont en attente et prennent le relais en cas de panne. OpenFlow prévoit pour cela des rôles (chapitre 14).
- **Cohérence.** Une partie de l'état exige une cohérence forte (qui est maître de quel switch, quelles intentions sont installées) ; une autre se contente d'une cohérence à terme (statistiques). C'est le compromis classique des systèmes distribués.
- **Panne totale.** Si tous les contrôleurs sont perdus, les switches continuent d'appliquer les règles déjà installées ; seules les nouvelles décisions sont impossibles. Une application réactive en souffre davantage qu'une application proactive.
- **Réseau de contrôle.** Le canal entre contrôleur et switches passe de préférence par un réseau d'administration dédié (*hors bande*), plutôt que par le réseau qu'il pilote (*dans la bande*).

10.9 Choisir un contrôleur

Critère	Questions à poser
Cas d'usage	Cloud, fabric de datacenter, WAN, campus, laboratoire ?
Interfaces sud	Quels équipements faut-il piloter, et par quels protocoles ?
Intégration	Se branche-t-il sur l'orchestrateur existant (OpenStack,

Critère	Questions à poser
	Kubernetes, VMware) ?
Échelle et disponibilité	Combien d'équipements et de règles ? Quel comportement en cas de panne d'une instance ?
Pérennité	Activité du projet, éditeur, support, feuille de route ?
Sécurité	TLS sur le canal sud, authentification et rôles sur l'API nord, journalisation ?
Compétences	L'équipe sait-elle l'exploiter et, si besoin, l'étendre (Java, Python, Go) ?

À retenir. Un contrôleur combine pilotes sud, services de topologie et de chemins, magasin distribué et API nord, et réconcilie sans cesse l'état voulu avec l'état réel. ONOS sert à apprendre, OpenDaylight à orchestrer en NETCONF, os-ken à prototyper, Faucet à déployer OpenFlow en production, OVN à virtualiser le réseau des clouds. En production : un cluster d'au moins trois instances et un réseau de contrôle dédié.

Chapitre 11 — Les techniques SDN

Un contrôleur découvre le réseau, calcule des chemins et les installe, en réagissant au trafic ou en anticipant. Ce chapitre détaille ces techniques : modes de pilotage, découverte, calcul de chemins, intentions, ingénierie de trafic, résilience et supervision. Module 3 de la formation.

11.1 Les modes de pilotage

Le mode réactif. Le switch ne possède au départ qu'une règle par défaut, qui envoie au contrôleur tout paquet inconnu.

1. Le premier paquet d'un flux arrive ; aucune règle ne correspond.
2. Le switch le remonte au contrôleur dans un message PACKET_IN.
3. Le contrôleur décide : il installe une règle (FLOW_MOD) pour les paquets suivants et réinjecte ce premier paquet (PACKET_OUT).
4. Les paquets suivants du flux sont traités par le switch seul, à pleine vitesse.

Le mode proactif. Le contrôleur installe les règles avant l'arrivée du trafic, à partir de sa connaissance de la topologie et de la politique.

Le mode hybride. C'est le cas le plus fréquent : les règles de politique et de routage sont proactives, et le contrôleur n'intervient de façon réactive que pour les cas imprévus (nouvel hôte, flux inconnu).

Critère	Réactif	Proactif
Premier paquet	Retardé (aller-retour vers le contrôleur)	Traité immédiatement
Taille des tables	Limitée aux flux actifs (expiration)	Toutes les règles possibles

Critère	Réactif	Proactif
Charge du contrôleur	Proportionnelle aux nouveaux flux	Proportionnelle aux changements de topologie et de politique
Panne du contrôleur	Les nouveaux flux ne passent plus	Le trafic continue
Exposition	Un débit de nouveaux flux peut saturer le contrôleur	Faible

En TP. L'application `fwd` d'ONOS (TP3-A) et le switch apprenant d'os-ken (TP4-B) sont réactifs ; les règles de blocage posées au branchement du switch dans le TP4-B sont proactives.

11.2 La découverte de la topologie

Les équipements. Un switch qui se connecte s'identifie par son *datapath ID* (dpid), un identifiant de 64 bits, et décrit ses ports. Le contrôleur l'ajoute à son inventaire.

Les liens. OpenFlow ne décrit pas les liens entre switches ; le contrôleur les découvre avec LLDP. Cette méthode, parfois appelée OFDP, se déroule ainsi :

1. Le contrôleur installe sur chaque switch une règle qui lui renvoie les trames LLDP (EtherType 0x88CC).
2. Il fait émettre par chaque switch, sur chacun de ses ports, une trame LLDP (PACKET_OUT) qui contient l'identifiant du switch et le numéro du port.
3. Le switch voisin reçoit la trame et la remonte au contrôleur (PACKET_IN), en indiquant son propre identifiant et son port d'entrée.
4. Le contrôleur en déduit un lien orienté : du port X du switch A vers le port Y du switch B.

Chaque câble produit donc deux liens orientés, un dans chaque sens : c'est pourquoi ONOS affiche 8 liens pour 4 câbles au TP3-A. L'opération est répétée périodiquement, toutes les quelques secondes ; un lien disparaît quand un port tombe (message PORT_STATUS) ou quand ses trames LLDP cessent d'arriver.

Les hôtes. Le contrôleur localise les machines grâce aux paquets qu'elles émettent (ARP, DHCP, IP) : l'adresse source et le port d'entrée du PACKET_IN indiquent où elles se trouvent. Une machine se trouve sur un port qui n'est pas un lien entre switches, un port de bordure.

Attention. Un attaquant qui forge des trames LLDP peut faire croire à des liens inexistants et détourner du trafic : c'est l'empoisonnement de topologie. Les parades consistent à authentifier les trames LLDP émises par le contrôleur (approche proposée notamment par les travaux TopoGuard) et à ne jamais accepter un lien découvert sur un port d'accès.

11.3 Le calcul de chemins

Le contrôleur représente le réseau comme un graphe : les switches sont des nœuds, les liens des arêtes, chacune dotée d'un poids.

Métrique	Effet

Métrique	Effet
Nombre de sauts	Chemin le plus court en nombre de switches
Latence mesurée	Chemin le plus rapide
Capacité disponible	Évite les liens chargés
Coût administratif	Préférence fixée par l'exploitant

L'algorithme de **Dijkstra** trouve le chemin de poids minimal. Ses variantes trouvent les *k* plus courts chemins, l'ensemble des chemins de même coût (pour l'ECMP), des chemins disjoints (pour la protection) ou des chemins sous contraintes, par exemple au moins 1 Gbit/s disponible et une latence inférieure à 5 ms.

Installer un chemin. Un chemin se traduit par une règle sur chaque switch traversé. L'ordre compte : en installant d'abord les règles des switches les plus proches de la destination, on évite que les premiers paquets ne trouvent un chemin incomplet. Le message BARRIER d'OpenFlow permet d'attendre qu'un switch ait appliqué ses règles avant de passer au suivant.

Pour aller plus loin. Modifier un chemin en service sans perte ni boucle est un problème étudié sous le nom de mise à jour cohérente : on installe les nouvelles règles avec une étiquette de version, on bascule le marquage à l'entrée du réseau, puis on retire les anciennes.

11.4 Les intentions

Une **intention** exprime le résultat voulu plutôt que la manière de l'obtenir : « l'hôte h1 doit pouvoir joindre l'hôte h3 », et non « installer telle règle sur s1, s4 et s3 ». Le contrôleur traduit l'intention en règles, l'installe et la maintient quand la topologie change.

Dans ONOS. Le cadre d'intentions propose plusieurs types :

Type d'intention	Signification
HostToHostIntent	Connectivité bidirectionnelle entre deux hôtes
PointToPointIntent	Trafic d'un port d'entrée vers un port de sortie, avec un filtre (sélecteur)
MultiPointToSinglePointIntent	Plusieurs entrées vers une sortie
SinglePointToMultiPointIntent	Une entrée vers plusieurs sorties
PathIntent	Un chemin explicite imposé

Une intention suit un cycle de vie : soumise, compilée en intentions installables, puis en règles, installée, recompilée quand la topologie change, retirée. Si aucun chemin n'existe plus, elle passe à l'état FAILED ; dès qu'un chemin réapparaît, elle est réinstallée sans intervention. C'est ce que montre le TP3-B.

Les réseaux à intentions. Le concept s'est généralisé sous le nom d'*Intent-Based Networking*, défini par la RFC 9315 de l'IRTF (2022). Un tel système enchaîne quatre fonctions :

1. **Recueillir** l'intention, exprimée dans un langage métier ;
2. **Traduire** l'intention en configuration ;
3. **Appliquer** la configuration au réseau ;
4. **Vérifier** en continu que le réseau réel respecte l'intention, et corriger sinon.

La quatrième fonction, la boucle de vérification, fait la différence avec une simple automatisation.

11.5 L'ingénierie de trafic

Un routage classique envoie tout le trafic par le plus court chemin, quitte à saturer un lien pendant qu'un autre reste vide. Grâce à sa vue globale, un contrôleur peut répartir le trafic en fonction de la charge réelle.

- **Répartition pondérée** : un groupe OpenFlow de type *select* répartit les flux sur plusieurs chemins, avec des poids proportionnels à leur capacité.
- **Classes de trafic** : les flux prioritaires (voix, supervision) passent par les chemins les moins chargés ; les sauvegardes, par les chemins restants.
- **Gros flux** : les flux volumineux, détectés par les compteurs ou par sFlow, sont déplacés pour ne pas pénaliser les autres.
- **Limitation** : un meter plafonne le débit d'un flux ou d'une classe.

L'exemple de référence est le réseau B4 de Google, qui relie ses datacenters : en planifiant centralement le trafic selon les priorités des applications, Google y a porté l'utilisation des liens près de leur capacité, là où les WAN classiques restent souvent sous les 30 à 40 %.

11.6 Résilience : restauration et protection

Mécanisme	Principe	Temps de bascule	Limite
Restauration	Le contrôleur apprend la panne (PORT_STATUS, LLDP), recalcule et installe un nouveau chemin	De quelques dizaines de millisecondes à plusieurs secondes selon la taille du réseau	Dépend du contrôleur et de la charge
Protection	Un chemin de secours est préinstallé dans un groupe *fast-failover* ; le switch bascule seul dès que le port principal tombe	Quasi immédiat	Consomme des entrées de table ; ne protège que les pannes prévues

On combine souvent les deux : la protection absorbe la panne immédiatement, la restauration recalcule ensuite un chemin optimal.

11.7 La supervision par flux

Chaque règle OpenFlow compte les paquets et les octets qu'elle traite, et sa durée de vie. Le contrôleur dispose ainsi d'une métrologie très fine, sans sonde supplémentaire.

- **Interrogation** : le contrôleur demande périodiquement les statistiques de flux, de ports, de tables ou de meters (messages MULTIPART).
- **Notification** : une règle qui expire peut remonter ses compteurs finaux (message FLOW_REMOVED).
- **Échantillonnage** : Open vSwitch exporte des échantillons de trafic en sFlow, NetFlow ou IPFIX vers un collecteur.

Ces données servent à détecter une anomalie (un hôte qui ouvre soudain des milliers de connexions), à dimensionner le réseau et à piloter l'ingénierie de trafic.

11.8 Le découpage du réseau

Le découpage (*slicing*) partage une même infrastructure entre plusieurs contrôleurs ou locataires, chacun voyant sa propre tranche du réseau. Le prototype historique, FlowVisor (2009), s'intercalait entre les switches et plusieurs contrôleurs pour leur répartir l'espace des flux. L'idée a été reprise à grande échelle par les réseaux mobiles 5G, où une « tranche » garantit à un usage (secours, industrie, grand public) ses propres performances.

À retenir. Réactif = décision au premier paquet ; proactif = règles installées d'avance ; en pratique, un mélange. Les liens se découvrent par LLDP, d'où deux liens orientés par câble. Une intention décrit le résultat et le contrôleur la maintient. La protection bascule dans le switch, la restauration passe par le contrôleur.

Chapitre 12 — Les applications du SDN et l'automatisation

Le SDN sert aujourd'hui surtout à virtualiser le réseau des clouds, à piloter les WAN et à automatiser la sécurité ; il rend surtout possible une exploitation du réseau par le code. Ce chapitre passe en revue les cas d'usage, puis détaille la démarche NetDevOps et le GitOps appliqué au réseau. Module 3 de la formation.

12.1 Datacenter et cloud

- **Réseaux virtuels multi-locataires** : OVN dans OpenStack et OpenShift, NSX dans les environnements VMware. Chaque projet reçoit ses réseaux, ses routeurs et ses groupes de sécurité par API.
- **Micro-segmentation** : le filtrage est appliqué sur chaque hôte, au plus près des charges (chapitre 4).
- **Services distribués** : répartition de charge, NAT et DHCP réalisés sur chaque hôte plutôt que par des boîtiers centraux.
- **Automatisation de la fabric** : un contrôleur ou un outil à intentions déploie et vérifie la configuration des leafs et des spines.

12.2 WAN et interconnexion de sites

- **WAN de datacenters** : Google (B4) et Microsoft (SWAN) planifient centralement le trafic entre leurs sites pour utiliser au mieux des liens très coûteux.

- **SD-WAN** : un orchestrateur central pilote des boîtiers de site reliés par des tunnels chiffrés sur plusieurs transports (MPLS, Internet, 4G/5G). Il choisit le chemin selon l'application et l'état des liens, et un nouveau site se raccorde sans intervention locale (*zero-touch provisioning*).
- **Réseaux d'opérateurs** : un contrôleur de calcul de chemins (PCE) impose des chemins Segment Routing ou MPLS selon les contraintes de latence et de capacité.

12.3 Campus et réseaux d'accès

Le contrôle d'accès au réseau (NAC) authentifie chaque poste ou objet, souvent par 802.1X, puis lui attribue dynamiquement un segment et une politique. Les invités et les objets connectés sont isolés automatiquement. C'est le prolongement direct d'Ethane, le projet qui a donné naissance au SDN.

12.4 Sécurité opérationnelle

Le contrôleur offre aux équipes de sécurité un levier d'action immédiat sur tout le réseau.

Usage	Mécanisme
Quarantaine	Une sonde détecte un comportement suspect ; un appel à l'API nord installe une règle qui isole l'hôte
Copie de trafic à la demande	Un groupe *all* duplique le trafic d'un hôte vers une sonde, le temps d'une investigation
Redirection	Le trafic d'un attaquant est détourné vers un leurre (*honeypot*)
Atténuation de déni de service	Le trafic visant une cible est filtré ou dévié vers un centre de nettoyage ; entre opérateurs, BGP FlowSpec (RFC 8955) diffuse ces règles
Filtrage dynamique	Les règles suivent les changements d'inventaire sans modification manuelle

Ces actions s'intègrent dans des chaînes de réponse automatisée : l'outil d'orchestration de la sécurité appelle l'API du contrôleur comme il appellerait celle d'un pare-feu.

12.5 Tests, recherche et formation

Mininet et les outils comme Containerlab reproduisent un réseau complet sur une seule machine. On y teste une politique, une mise à jour ou un scénario de panne avant de toucher à la production. Poussée plus loin, cette idée donne le **jumeau numérique** du réseau : une copie alimentée par la configuration réelle, sur laquelle on simule chaque changement.

12.6 L'automatisation réseau (NetDevOps)

Le SDN a ouvert la voie à une exploitation du réseau par les pratiques du développement logiciel. Les principes sont ceux que les équipes DevOps appliquent déjà aux serveurs.

1. **Une source de vérité.** L'inventaire, l'adressage et les segments sont décrits dans un référentiel (par exemple NetBox) ou dans des fichiers d'un dépôt Git, pas dans la tête des administrateurs.

2. **L'état voulu déclaratif.** On décrit ce que le réseau doit être, et l'outil calcule les changements.
3. **L'idempotence.** Appliquer deux fois la même description donne le même résultat, sans effet de bord.
4. **Le versionnement et la revue.** Tout changement passe par une demande de fusion, relue par un pair.
5. **Les tests.** Validation syntaxique, analyse de la configuration, essai en laboratoire.
6. **Le déploiement automatisé et la vérification.** Le pipeline applique le changement, puis vérifie que le réseau se comporte comme prévu.

Outil	Usage
Ansible	Configuration des équipements (CLI, NETCONF), d'Open vSwitch, des objets Kubernetes
Terraform, OpenTofu	Ressources réseau des clouds et des plateformes de virtualisation
Python (requests, ncclient, pygnmi)	Appels aux API REST, NETCONF, gNMI
Nornir	Automatisation en Python sur un grand parc
Batfish	Analyse hors ligne des configurations : joignabilité, filtres, erreurs
Containerlab, Mininet	Laboratoires réseau jetables, pour tester avant de déployer
Argo CD, Flux	Réconciliation continue entre un dépôt Git et un cluster Kubernetes

12.7 Le GitOps appliqué au réseau

Le GitOps fait du dépôt Git la source de vérité de l'infrastructure : ce qui est fusionné sur la branche principale est, par définition, ce qui doit tourner. Appliqué au réseau, le parcours d'un changement devient le suivant.

1. Un ingénieur modifie `politique.yaml` dans une branche et ouvre une demande de fusion.
2. Le pipeline valide la syntaxe et le schéma du fichier.
3. Il rejoue la politique dans un laboratoire Mininet et vérifie les flux attendus.
4. Un pair relit et approuve.
5. Après fusion, le pipeline applique la politique par l'API nord du contrôleur.
6. Une vérification post-déploiement contrôle la connectivité réelle.
7. Une tâche périodique détecte toute dérive entre le dépôt et le réseau, et la corrige ou la signale.

Exemple indicatif de pipeline GitLab CI pour le fil rouge :

```
stages: [valider, tester, deployer]

valider:
  stage: valider
```

```
  image: python:3.12-slim
  script:
    - pip install pyyaml
    - python3 -c "import yaml; yaml.safe_load(open('politique.yaml'))"

tester:
  stage: tester
  tags: [lab-mininet]          # runner disposant de Mininet
  script:
    - sudo ./tests/rejouer_politique.sh politique.yaml

deployer:
  stage: deployer
  environment: production
  script:
    - python3 apply_policy.py politique.yaml
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
      when: manual             # déclenchement manuel après fusion
```

Les identifiants du contrôleur sont fournis au pipeline par des variables protégées et masquées, jamais écrits dans le dépôt. Le journal du dépôt devient la piste d'audit de chaque changement : qui, quand, pourquoi, validé par qui.

En TP. Le TP3-C réalise les étapes 1 et 5 à la main : une politique YAML versionnée, appliquée par un script qui appelle l'API REST d'ONOS.

12.8 Performance et gestion des ressources

Gain	D'où il vient
Meilleure utilisation des liens	Ingénierie de trafic fondée sur la vue globale et la charge réelle
Latence maîtrisée	Choix des chemins selon la latence mesurée et la classe de trafic
Mise en service rapide	Un segment ou une règle se crée par API au lieu d'une intervention manuelle sur plusieurs équipements
Moins d'erreurs de configuration	Une politique unique traduite automatiquement, au lieu de dizaines de configurations saisies à la main
Rétablissement plus rapide	Visibilité centralisée, protection préinstallée, configuration rejouable
Élasticité	Les fonctions réseau et les chemins suivent la charge
Capacité mieux planifiée	Compteurs par flux et télémétrie continue

Ces gains ne sont pas automatiques : ils supposent que les processus suivent, que l'automatisation soit réellement utilisée et que les compétences soient réunies.

12.9 Cas d'usage pour une équipe de développement et d'intégration

- **Environnements éphémères** : chaque demande de fusion reçoit un segment réseau, ses règles d'accès et ses services, créés puis détruits par le pipeline.
- **Matrice des flux en code** : les flux autorisés entre composants d'une application sont décrits avec le code de l'application et déployés avec elle (NetworkPolicy, groupes de sécurité).
- **Isolation des runners** : les exécutants d'intégration continue sont confinés dans une zone qui n'atteint que les dépôts et registres nécessaires.
- **Recette fidèle** : l'environnement de qualification reproduit le réseau de production, à partir des mêmes fichiers.
- **Traçabilité** : chaque ouverture de flux est liée à une demande, à un auteur et à un relecteur.

À retenir. Le SDN sert les clouds, les WAN, les campus et la sécurité. Sa principale conséquence pour les équipes est l'exploitation du réseau par le code : source de vérité, état déclaratif, revue, tests, déploiement automatisé et réconciliation.

Chapitre 13 — Sécurité, fiabilité et limites du SDN

Centraliser le contrôle, c'est aussi centraliser le risque : qui maîtrise le contrôleur maîtrise tout le réseau. Ce chapitre recense la surface d'attaque d'une architecture SDN, les parades, les questions de fiabilité et les limites du modèle. Module 3 de la formation.

13.1 La surface d'attaque

Composant	Menace	Conséquence
Applications	Application malveillante ou boguée, privilèges excessifs	Règles arbitraires sur tout le réseau
API nord	Identifiants par défaut ou volés, absence de chiffrement, API exposée trop largement	Prise de contrôle à distance du réseau
Contrôleur	Vulnérabilité logicielle, déni de service	Perte de la capacité de décision, ou compromission complète
Canal est-ouest	Instance usurpée dans le cluster	Corruption de l'état partagé
Canal sud	Connexion OpenFlow en clair : écoute, interception, injection	Règles forgées, cartographie du réseau par un attaquant
Switches	Faux switch qui se connecte au contrôleur, saturation des tables	Fausse topologie, refus de nouvelles règles
Plan de données	Trames LLDP forgées, usurpation de l'emplacement d'un hôte	Détournement ou interception de trafic

13.2 Menaces typiques et parades

La saturation du contrôleur. En mode réactif, chaque nouveau flux déclenche un PACKET_IN. Un attaquant qui génère des milliers de flux par seconde (adresses ou ports aléatoires) sature le contrôleur et le canal de contrôle.

- Limiter le débit des remontées : OpenFlow 1.3 prévoit un meter virtuel dédié au trafic envoyé au contrôleur, et Open vSwitch des paramètres `controller_rate_limit` et `controller_burst_limit`.
- Privilégier les règles proactives pour le trafic attendu.
- Supprimer les remontées inutiles : ne pas envoyer au contrôleur ce qu'une règle peut traiter.

Le débordement des tables. Les tables d'un switch, surtout matériel, sont limitées. Des règles réactives trop fines peuvent les remplir.

- Fixer des délais d'expiration (`idle_timeout`) aux règles réactives.
- Agréger les règles (par sous-réseau plutôt que par hôte).
- Surveiller l'occupation des tables (statistiques de tables).

L'interception du canal sud. OpenFlow fonctionne par défaut sur TCP en clair.

- Chiffrer et authentifier mutuellement le canal en TLS, avec des certificats pour le contrôleur et pour chaque switch.
- Faire passer le canal par un réseau d'administration dédié, hors de la bande pilotée.

```
# Open vSwitch : canal OpenFlow en TLS
sudo ovs-vsctl set-ssl /etc/openvswitch/sc-privkey.pem \
     /etc/openvswitch/sc-cert.pem /etc/openvswitch/cacert.pem
sudo ovs-vsctl set-controller br-lab ssl:10.10.99.10:6653
sudo ovs-vsctl set-fail-mode br-lab secure
```

L'empoisonnement de topologie et l'usurpation d'hôte. Des trames LLDP forgées font apparaître de faux liens ; une machine qui émet avec l'adresse MAC d'une autre fait croire au contrôleur que celle-ci a migré.

- Authentifier les trames LLDP (signature ajoutée par le contrôleur, sur le modèle de travaux comme TopoGuard).
- Refuser la découverte de liens sur les ports d'accès.
- Contrôler les migrations d'hôtes : une migration sans événement de l'orchestrateur est suspecte.

La compromission de l'API nord.

- Changer immédiatement les identifiants par défaut (pour ONOS : `onos` / `rocks`).
- Imposer HTTPS, des comptes nominatifs, des rôles et, via un proxy d'authentification, l'authentification multifacteur.
- N'exposer l'API que sur le réseau d'administration, et journaliser chaque appel.

Les applications malveillantes ou en conflit. Deux applications peuvent installer des règles contradictoires sans le savoir.

- Définir un plan de priorités et attribuer un cookie par application.

- Restreindre les permissions des applications (ONOS propose un mode de sécurité qui limite les API accessibles à chaque application).
- Relire et tester le code des applications comme tout autre code de production.

13.3 Liste de durcissement

- ☐ Identifiants par défaut remplacés, comptes nominatifs, mots de passe ou clés robustes
- ☐ Canal sud en TLS avec authentification mutuelle
- ☐ API nord en HTTPS, accessible uniquement depuis le réseau d'administration
- ☐ Contrôleur, switches et postes d'administration dans un réseau d'administration dédié
- ☐ Mode de secours `secure` sur les switches
- ☐ Ports d'écoute réduits au strict nécessaire
- ☐ Limitation du débit des remontées vers le contrôleur
- ☐ Journaux du contrôleur et des appels d'API envoyés au SIEM
- ☐ Mises à jour de sécurité suivies (contrôleur, dépendances Java ou Python, Open vSwitch)
- ☐ Configuration et politique versionnées, sauvegarde de l'état du contrôleur

Ces mesures rejoignent les recommandations de l'ANSSI sur l'administration sécurisée des systèmes d'information (guide ANSSI-PA-022) : postes et réseau d'administration dédiés, authentification forte, cloisonnement et traçabilité des actions d'administration.

13.4 La fiabilité

- **Point unique de défaillance** : traité par un cluster d'au moins trois instances (chapitre 10).
- **Rayon d'impact** : une erreur de politique ou un bogue du contrôleur se propage à tout le réseau en quelques secondes. D'où l'importance des tests, des déploiements progressifs (un site ou un segment d'abord) et de la possibilité de revenir en arrière.
- **Dépendance au réseau de contrôle** : si les switches perdent le contrôleur, ils gardent les règles installées (mode `secure`) ou repassent en switch classique (mode `standalone`). Le choix doit être délibéré.
- **Supervision du contrôleur lui-même** : état du cluster, latence des réponses, débit de PACKET_IN, nombre de règles en échec.
- **Procédures** : que fait l'astreinte si le contrôleur est perdu ? Comment le reconstruire à partir de la configuration versionnée ?

13.5 Les limites du modèle

Limite	Explication
Passage à l'échelle	Le contrôleur et le canal de contrôle deviennent des goulots si tout passe par eux, surtout en mode réactif ou à travers un WAN
Matériel	Les switches matériels implémentent OpenFlow partiellement, avec des tables de taille limitée et un pipeline figé
Interface nord	Aucune norme : changer de contrôleur oblige à réécrire les intégrations

Limite	Explication
Maturité open source	Certains projets ont ralenti (ONOS) ; la pérennité doit être évaluée
Complexité	Les abstractions cachent la réalité jusqu'à la panne, où il faut comprendre toutes les couches
Organisation	Le SDN demande des équipes qui mêlent réseau, système, développement et sécurité

Le SDN « pur », où un contrôleur OpenFlow pilote tous les switches d'un réseau d'entreprise, est resté rare. Les principes du SDN, en revanche, se sont imposés là où le logiciel domine : réseaux virtuels des clouds, SD-WAN, réseaux des grands exploitants, automatisation pilotée par intentions.

À retenir. Le contrôleur et ses API sont les biens les plus sensibles d'un réseau SDN. Parades essentielles : TLS sur le canal sud, API nord restreinte et authentifiée, réseau d'administration dédié, limitation des remontées, journalisation. La centralisation élargit le rayon d'impact des erreurs : tester et déployer progressivement.

Chapitre 14 — Le protocole OpenFlow : architecture et canal de contrôle

OpenFlow est le protocole qui permet à un contrôleur de lire et d'écrire les tables d'acheminement d'un switch. Ce chapitre présente son histoire et ses versions, le modèle de switch qu'il définit, l'établissement du canal de contrôle, les rôles entre contrôleurs et les modes de secours. Module 4 de la formation.

14.1 Origine et gouvernance

OpenFlow naît à l'université Stanford, décrit dans l'article « OpenFlow: Enabling Innovation in Campus Networks » (2008). L'idée est pragmatique : tous les switches contiennent déjà des tables de flux, implantées différemment par chaque constructeur, mais avec un socle commun. OpenFlow expose ce socle par un protocole ouvert, sans obliger les constructeurs à dévoiler le fonctionnement interne de leurs circuits.

La version 1.0 paraît fin 2009. L'Open Networking Foundation (ONF), créée en 2011, reprend ensuite la normalisation. La dernière version publique est la 1.5.1 (2015) ; une version 1.6 a été diffusée aux seuls membres de l'ONF. La version la plus implantée reste de loin la **1.3**, celle utilisée dans les TP.

14.2 Les versions

Version	Numéro dans l'en-tête	Date	Principaux apports
1.0	0x01	2009	Une seule table, correspondance sur 12 champs fixes, actions de base
1.1	0x02	2011	Plusieurs tables enchaînées (pipeline), instructions, groupes, étiquettes MPLS et VLAN empilables

Version	Numéro dans l'en-tête	Date	Principaux apports
1.2	0x03	2011	Correspondance extensible OXM, IPv6, plusieurs contrôleurs avec rôles
1.3	0x04	2012 (révisions jusqu'à 1.3.5 en 2015)	Meters, entrée table-miss explicite, filtrage des événements par connexion, connexions auxiliaires, messages MULTIPART
1.4	0x05	2013	Lots de modifications atomiques (*bundles*), surveillance des flux, éviction, ports optiques
1.5 et 1.5.1	0x06	2014 et 2015	Tables de sortie (*egress*), pipeline conscient du type de paquet, lots programmés

Un switch et un contrôleur doivent s'accorder sur une version commune. Open vSwitch accepte les versions 1.0 à 1.5 ; on fixe celles autorisées par bridge, par exemple `ovs-vsctl set bridge br0 protocols=OpenFlow13`. Les commandes `ovs-ofctl` doivent alors préciser la version : `ovs-ofctl -O OpenFlow13 dump-flows br0`.

14.3 Le modèle du switch OpenFlow

La spécification décrit un switch logique composé de :

- **ports** par lesquels entrent et sortent les paquets ;
- une ou plusieurs **tables de flux**, enchaînées en pipeline (chapitre 15) ;
- une **table de groupes**, pour la réplication, la répartition et le secours ;
- une **table de meters**, pour la mesure et la limitation de débit ;
- un ou plusieurs **canaux OpenFlow** vers des contrôleurs.

Les ports. Outre les ports physiques et logiques (tunnels, agrégats), OpenFlow définit des ports réservés, utilisables comme destination d'une action `output`.

Port réservé	Effet
`CONTROLLER`	Envoie le paquet au contrôleur (PACKET_IN)
`FLOOD`	Diffuse sur tous les ports sauf le port d'entrée, selon le traitement classique du switch
`ALL`	Diffuse sur tous les ports sauf le port d'entrée
`IN_PORT`	Renvoie le paquet par son port d'entrée
`TABLE`	Soumet le paquet au pipeline (dans un PACKET_OUT)
`NORMAL`	Confie le paquet au traitement classique du switch (commutation L2, éventuellement routage)

Port réservé	Effet
`LOCAL`	Envoie le paquet à la pile réseau locale du switch
`ANY`	Joker utilisé dans les requêtes et les suppressions, jamais en sortie

`NORMAL` et `FLOOD` n'existent que sur les switches hybrides, qui savent aussi fonctionner sans OpenFlow. Open vSwitch en fait partie : c'est la règle `actions=NORMAL` observée au TP1-A.

14.4 Le canal OpenFlow

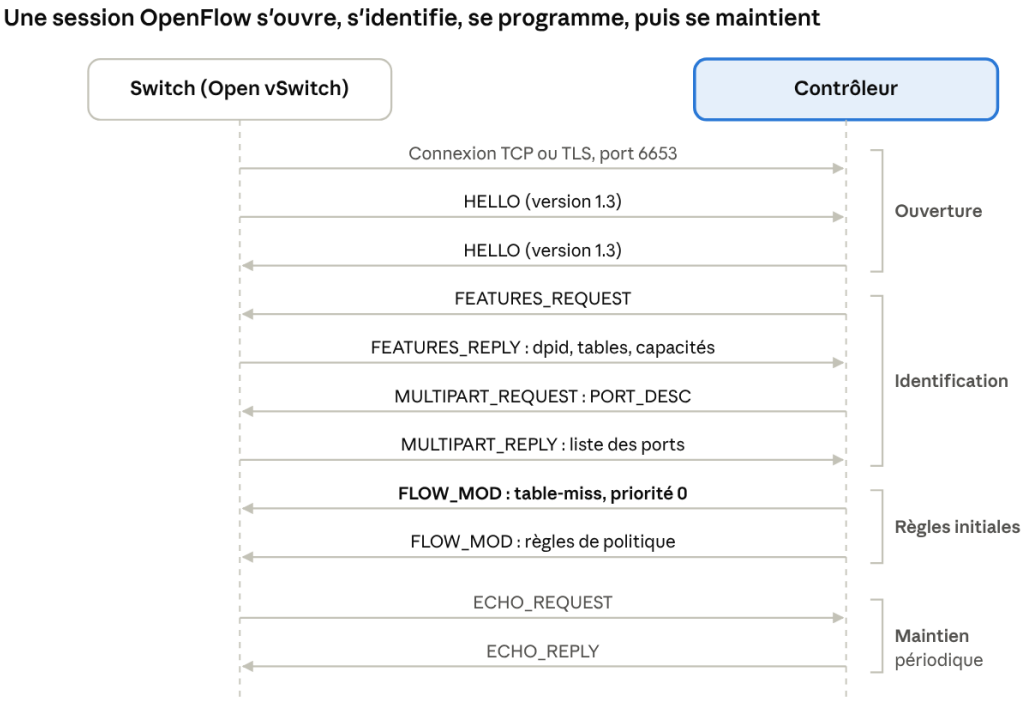
Le canal relie le switch au contrôleur. Il utilise TCP ou TLS, sur le port **6653** attribué par l'IANA (les premières implémentations utilisaient 6633). En général, c'est le switch qui ouvre la connexion vers le contrôleur.

Les messages se répartissent en trois familles.

Famille	Initiateur	Exemples
Contrôleur vers switch	Le contrôleur, le switch répond si besoin	FEATURES_REQUEST, FLOW_MOD, GROUP_MOD, METER_MOD, PACKET_OUT, MULTIPART_REQUEST, BARRIER_REQUEST, ROLE_REQUEST
Asynchrones	Le switch, sans y être invité	PACKET_IN, FLOW_REMOVED, PORT_STATUS
Symétriques	L'un ou l'autre	HELLO, ECHO_REQUEST et ECHO_REPLY, ERROR, EXPERIMENTER

L'établissement de la connexion.

1. Le switch ouvre la connexion TCP ou TLS.
2. Chacun envoie un **HELLO** annonçant la version la plus haute qu'il accepte ; la version retenue est la plus haute commune. Depuis la version 1.3.1, le HELLO peut porter la liste exacte des versions acceptées.
3. Le contrôleur envoie un **FEATURES_REQUEST** ; le switch répond par un **FEATURES_REPLY** contenant son identifiant (*datapath ID*, 64 bits, dont 48 dérivés en général d'une adresse MAC), le nombre de tables et ses capacités.
4. Le contrôleur demande la description des ports (MULTIPART de type PORT_DESC, depuis la version 1.3), règle les paramètres de remontée (SET_CONFIG) et installe ses premières règles, à commencer par l'entrée table-miss.
5. Pendant toute la session, des **ECHO_REQUEST** et **ECHO_REPLY** vérifient que l'autre côté répond. Sans réponse, la connexion est considérée comme perdue.



ouverture d'une session OpenFlow 1.3 · 4 phases

Tant que la table-miss n'est pas installée, un switch OpenFlow 1.3 détruit tout paquet inconnu : c'est la première règle qu'un contrôleur doit poser.

Chaque message porte un identifiant de transaction (*xid*) : la réponse reprend celui de la requête, ce qui permet de les associer.

En TP. Au TP4-B, la capture du port 6653 analysée avec tshark montre exactement cette séquence : types 0 (HELLO), 5 et 6 (FEATURES), 18 et 19 (MULTIPART), puis 14 (FLOW_MOD) pour la table-miss et la règle de blocage.

14.5 Plusieurs contrôleurs : les rôles

Pour la haute disponibilité, un switch se connecte à plusieurs contrôleurs. Depuis la version 1.2, chaque connexion a un rôle.

Rôle	Droits	Événements reçus
EQUAL (par défaut)	Lecture et écriture	Tous
MASTER	Lecture et écriture ; un seul maître par switch	Tous
SLAVE	Lecture seule : ses modifications sont refusées	Seulement PORT_STATUS, par défaut

Un contrôleur réclame un rôle par un message ROLE_REQUEST. Quand une instance devient maître, le switch fait passer l'ancien maître en SLAVE. Un numéro de génération (*generation_id*) croissant empêche un ancien maître, revenu après une coupure, de reprendre la main par erreur.

La version 1.3 permet aussi de choisir, connexion par connexion, les événements asynchrones reçus (SET_ASYNC), et d'ouvrir des **connexions auxiliaires** en parallèle de la connexion principale pour répartir la charge des PACKET_IN.

14.6 Les modes de secours

Que fait un switch qui perd tous ses contrôleurs ? La spécification prévoit deux modes.

Mode	Comportement	Open vSwitch
Fail secure	Les règles installées continuent de s'appliquer et d'expirer normalement ; rien n'est plus envoyé au contrôleur	`fail_mode=secure`
Fail standalone	Le switch hybride repasse en commutation classique (traitement NORMAL)	`fail_mode=standalone`, valeur par défaut

Attention. En mode `standalone`, un switch qui perd son contrôleur se met à commuter normalement : toute la politique de segmentation portée par OpenFlow disparaît. Pour un réseau dont la sécurité repose sur les règles du contrôleur, le mode `secure` s'impose. Mininet configure ses switches en `secure` par défaut, d'où l'absence totale de connectivité au début du TP4-A.

14.7 Les switches OpenFlow

- **Switches logiciels** : Open vSwitch, de très loin le plus utilisé, et quelques implémentations de recherche (ofsoftswitch13).
- **Switches matériels** : de nombreux constructeurs ont proposé un mode OpenFlow, le plus souvent en version 1.3 et avec une prise en charge partielle des champs et des actions. Il faut vérifier précisément ce qu'un modèle sait faire avant de construire une politique dessus.
- **Switches hybrides ou purs** : un switch hybride combine OpenFlow et fonctionnement classique ; un switch pur ne fait que ce qu'OpenFlow lui dit.

À retenir. OpenFlow 1.3 est la version de référence. Le canal (TCP ou TLS, port 6653) s'ouvre par HELLO, puis FEATURES, puis l'installation de la table-miss ; ECHO le maintient. Plusieurs contrôleurs se partagent un switch par rôles MASTER, SLAVE ou EQUAL. Le mode de secours `secure` préserve la politique quand le contrôleur disparaît.

Chapitre 15 — Tables de flux et pipeline

Un switch OpenFlow traite chaque paquet en le confrontant à des entrées de flux, organisées en tables enchaînées : la règle de plus haute priorité qui correspond décide de son sort. Ce chapitre détaille la structure d'une entrée, les champs de correspondance, les instructions et actions, le pipeline, les groupes et les meters. Module 4 de la formation.

15.1 L'entrée de flux

Élément	Rôle
Champs de correspondance (*match*)	Critères que le paquet doit remplir ; les champs absents sont des jokers
Priorité	De 0 à 65 535 ; la plus haute l'emporte quand plusieurs entrées correspondent
Instructions	Ce qu'il faut faire du paquet : appliquer des actions, passer à une autre table, appliquer un meter
Compteurs	Paquets et octets traités, durée de vie
Délais d'expiration	`idle_timeout` (inactivité) et `hard_timeout` (durée absolue), en secondes ; 0 = jamais
Cookie	Valeur choisie par le contrôleur, pour retrouver et supprimer ses propres règles
Drapeaux	Par exemple, signaler la suppression de la règle au contrôleur

Une entrée est identifiée par ses champs de correspondance et sa priorité : deux entrées qui ont les mêmes ne peuvent coexister dans une table.

15.2 Les champs de correspondance

Depuis la version 1.2, les champs suivent le format extensible **OXM** (*OpenFlow Extensible Match*). Les outils les nomment différemment, ce qui déroute au début.

Champ (OXM)	Nom dans `ovs-ofctl`	Nom dans os-ken	Exemple
Port d'entrée	`in_port`	`in_port`	`in_port=3`
MAC source, destination	`dl_src`, `dl_dst`	`eth_src`, `eth_dst`	`dl_dst=00:00:00:00:00:01`
EtherType	`dl_type`	`eth_type`	`dl_type=0x0800`
VLAN	`dl_vlan`, `dl_vlan_pcp`	`vlan_vid`, `vlan_pcp`	`dl_vlan=10`
Protocole IP	`nw_proto`	`ip_proto`	`nw_proto=6`
IPv4 source, destination	`nw_src`, `nw_dst`	`ipv4_src`, `ipv4_dst`	`nw_dst=10.10.30.0/24`
Classe de service	`ip_dscp`	`ip_dscp`	`ip_dscp=46`
Ports TCP, UDP	`tp_src`, `tp_dst`	`tcp_src`, `tcp_dst`,	`tp_dst=443`

Champ (OXM)	Nom dans `ovs-ofctl`	Nom dans os-ken	Exemple
		`udp_src`, `udp_dst`	
ICMP	`icmp_type`, `icmp_code`	`icmpv4_type`, `icmpv4_code`	`icmp_type=8`
ARP	`arp_op`, `arp_spa`, `arp_tpa`	`arp_op`, `arp_spa`, `arp_tpa`	`arp_op=1`
IPv6	`ipv6_src`, `ipv6_dst`	`ipv6_src`, `ipv6_dst`	`ipv6_dst=fd00::/64`
Identifiant de tunnel	`tun_id`	`tunnel_id`	`tun_id=5010`
Métadonnée inter-tables	`metadata`	`metadata`	`metadata=0x1`

`ovs-ofctl` accepte aussi des raccourcis qui fixent l'EtherType et le protocole : `arp`, `ip`, `icmp`, `tcp`, `udp`, `ipv6`. Ainsi, `tcp` équivaut à `dl_type=0x0800,nw_proto=6`.

Les prérequis. Un champ n'a de sens que si les couches inférieures sont précisées. Pour filtrer sur une adresse IPv4, il faut indiquer que le paquet est IPv4 ; pour filtrer sur un port TCP, il faut indiquer qu'il est TCP. Une règle `nw_dst=10.0.0.1` sans `ip` est refusée ou ignorée, selon l'outil. C'est l'erreur la plus fréquente des débutants.

Les masques. Adresses et métadonnées acceptent un masque : `nw_src=10.10.20.0/24`, `dl_dst=01:00:00:00:00:00/01:00:00:00:00:00` (toutes les adresses multicast).

15.3 Priorités et chevauchements

Quand plusieurs entrées correspondent à un paquet, celle de plus haute priorité s'applique. Si deux entrées de même priorité se chevauchent, le résultat n'est pas défini par la spécification : il faut l'éviter. Le drapeau `CHECK_OVERLAP` fait refuser une règle qui en chevauche une autre de même priorité.

La bonne pratique est un **plan de priorités** écrit et partagé :

Plage	Usage
0	Table-miss
1 à 99	Acheminement appris (réactif)
100 à 199	Acheminement calculé (proactif)
200 à 999	Politique de sécurité
1 000 et plus	Exceptions et urgences (quarantaine)

15.4 Instructions et actions

Une entrée de flux ne porte pas directement des actions, mais des **instructions**.

Instruction	Effet
Apply-Actions	Exécute immédiatement une liste d'actions, dans l'ordre
Write-Actions	Ajoute des actions à l'*ensemble d'actions*, exécuté à la fin du pipeline
Clear-Actions	Vide l'ensemble d'actions
Write-Metadata	Écrit une métadonnée lisible par les tables suivantes
Goto-Table	Poursuit le traitement dans une table de numéro supérieur
Meter	Fait passer le paquet par un meter

Les **actions** proprement dites :

Action	Effet
Output	Envoie le paquet sur un port, physique ou réservé
Group	Confie le paquet à un groupe
Set-Queue	Choisit une file d'attente de qualité de service sur le port de sortie
Push-VLAN, Pop-VLAN	Ajoute ou retire une étiquette VLAN
Push-MPLS, Pop-MPLS	Ajoute ou retire une étiquette MPLS
Set-Field	Réécrit un champ : adresse MAC, adresse IP, VLAN, port…
Decrement TTL	Décrémente le TTL, comme le ferait un routeur

Il n'existe pas d'action « drop » : un paquet dont la liste d'actions est vide, et qui n'a plus de table à visiter, est détruit. `ovs-ofctl` accepte toutefois le mot `drop` pour plus de lisibilité.

L'ensemble d'actions. Les actions écrites par Write-Actions sont exécutées à la fin du pipeline, dans un ordre fixé par la spécification et non dans l'ordre d'écriture : copie du TTL vers l'intérieur, retrait d'étiquettes, ajout d'étiquettes MPLS, PBB puis VLAN, copie du TTL vers l'extérieur, décrément du TTL, réécritures de champs, file d'attente, groupe, puis sortie. Si un groupe est présent, l'action de sortie est ignorée. Apply-Actions est plus intuitif ; c'est le seul employé dans les TP.

15.5 Le pipeline multi-tables

Le traitement commence toujours en **table 0**. Une instruction Goto-Table peut envoyer le paquet vers une table de numéro **supérieur** uniquement : le pipeline avance sans jamais revenir en arrière, ce qui exclut les boucles. Le traitement s'arrête quand une entrée ne comporte pas de Goto-Table ; l'ensemble d'actions accumulé est alors exécuté.

Plusieurs tables permettent de séparer les fonctions et d'éviter l'explosion combinatoire des règles. Une organisation classique :

Table	Fonction
0	Classification et filtrage d'entrée
1	Rattachement au segment (VLAN, VNI), écriture d'une métadonnée
2	Commutation de niveau 2 (adresse MAC de destination)
3	Routage de niveau 3
4	Filtrage de sortie

Avec une seule table, filtrer 10 couples source-destination pour 50 destinations obligerait à combiner les deux critères dans chaque règle. Avec deux tables, on écrit 10 règles de filtrage plus 50 règles d'acheminement. OVN pousse cette logique très loin : son pipeline compte plusieurs dizaines de tables.

Exemple. La politique du TP4-A réécrite sur deux tables, avec la correction de l'ICMP (seules les requêtes sont bloquées, pas les réponses) :

```
# Table 0 : filtrage d'entrée
table=0,priority=300,tcp,nw_src=10.0.0.2,nw_dst=10.0.0.3,tp_dst=80,actions=goto_table
:1
table=0,priority=250,ip,nw_src=10.0.0.2,nw_dst=10.0.0.3,actions=drop
table=0,priority=200,icmp,icmp_type=8,nw_src=10.0.0.3,nw_dst=10.0.0.1,actions=drop
table=0,priority=0,actions=goto_table:1
# Table 1 : acheminement
table=1,priority=100,arp,actions=flood
table=1,priority=100,ip,nw_dst=10.0.0.1,actions=output:1
table=1,priority=100,ip,nw_dst=10.0.0.2,actions=output:2
table=1,priority=100,ip,nw_dst=10.0.0.3,actions=output:3
table=1,priority=0,actions=drop
```

Ce fichier s'applique d'un coup avec `ovs-ofctl -O OpenFlow13 replace-flows s1 regles.txt`.

15.6 L'entrée table-miss

L'entrée **table-miss** est l'entrée de priorité 0 sans aucun champ de correspondance : elle attrape tout ce qu'aucune autre n'a pris. Ses actions définissent le comportement par défaut de la table :

- `CONTROLLER` : le paquet remonte au contrôleur (mode réactif) ;
- aucune action : le paquet est détruit (politique « tout ce qui n'est pas permis est interdit ») ;
- Goto-Table : le paquet passe à la table suivante.

↗ traitement d'un paquet OpenFlow · mode réactif

Le contrôleur pose la règle (FLOW_MOD) et réinjecte le paquet (PACKET_OUT) ; les paquets suivants du flux restent dans le switch.

Attention. En OpenFlow 1.3, une table sans entrée table-miss détruit les paquets qui ne correspondent à rien. En OpenFlow 1.0, ils étaient envoyés au contrôleur. Un contrôleur OpenFlow 1.3 doit donc installer lui-même sa table-miss, comme le fait `switch_acl.py` au TP4-B.

15.7 Expiration et suppression des règles

- Une règle avec `idle_timeout=30` disparaît après 30 secondes sans trafic : idéal pour les règles apprises.
- Une règle avec `hard_timeout=3600` disparaît au bout d'une heure, qu'elle serve ou non : utile pour une exception temporaire.
- Avec le drapeau `SEND_FLOW_REM`, le switch envoie au contrôleur un FLOW_REMOVED contenant les compteurs finaux.

La suppression explicite se fait par FLOW_MOD de type DELETE. La suppression **non stricte** retire toutes les règles qui correspondent au critère donné, quelle que soit leur priorité ; la suppression **stricte** ne retire que la règle aux champs et à la priorité identiques. Le cookie, avec un masque, permet de retirer d'un coup toutes les règles d'une application : `ovs-ofctl del-flows br0 "cookie=0x10/-1"`.

15.8 Les groupes

Un **groupe** est une liste de « seaux » (*buckets*), chacun portant ses propres actions. Une règle y envoie le paquet par l'action Group ; le type du groupe décide quels seaux s'appliquent.

Type	Seaux exécutés	Usage
`all`	Tous, chacun sur une copie du paquet	Diffusion, multicast, miroir vers une sonde
`select`	Un seul, choisi par flux (hachage), avec des poids	Répartition de charge, ECMP
`indirect`	L'unique seau	Factoriser une action commune à beaucoup de règles (un prochain saut)
`fast-failover`	Le premier seau dont le port (ou groupe) surveillé est actif	Bascule locale sur panne, sans le contrôleur

```
OF="sudo ovs-ofctl -O OpenFlow13"
# Miroir : trafic vers h1 copié vers la sonde h4
$OF add-group s1 group_id=1,type=all,bucket=output:1,bucket=output:4
# Répartition sur deux liens, poids 2 pour 1
$OF add-group s1
group_id=2,type=select,bucket=weight:2,output:5,bucket=weight:1,output:6
# Secours : port 5, sinon port 6
$OF add-group s1
group_id=3,type=ff,bucket=watch_port:5,output:5,bucket=watch_port:6,output:6
$OF dump-groups s1
```

15.9 Les meters

Un **meter** mesure le débit des paquets qui le traversent et applique une **bande** au-delà d'un seuil.

- **Unité** : kilobits par seconde (`kbps`) ou paquets par seconde (`pktps`).
- **Bande `drop`** : détruit l'excédent, donc plafonne le débit.
- **Bande `dscp_remark`** : abaisse la classe de service de l'excédent, qui sera traité en priorité inférieure plus loin.

- **Rafale** (`burst_size`) : tolérance au-delà du débit, pendant un court instant.

Une règle fait passer ses paquets par un meter avec l'instruction Meter (`actions=meter:1,output:2` dans la syntaxe d'`ovs-ofctl`). La spécification définit aussi un meter virtuel pour le trafic envoyé au contrôleur, qui protège celui-ci des rafales de PACKET_IN (chapitre 13).

En TP. Au TP4-A, un meter de 5 000 kbit/s ramène le débit mesuré par iperf entre h1 et h2 à environ 5 Mbit/s. La commande `ovs-ofctl -O OpenFlow13 meter-features s1` indique si le datapath prend en charge les meters.

15.10 Suivre un paquet dans le pipeline

`ovs-appctl ofproto/trace` simule le traitement d'un paquet et affiche, table par table, l'entrée retenue et les actions exécutées.

```
sudo ovs-appctl ofproto/trace s1
in_port=3,icmp,icmp_type=8,nw_src=10.0.0.3,nw_dst=10.0.0.1
```

La sortie indique la table (`0.`), la règle retenue avec sa priorité et ses champs, puis le résultat final (`Datapath actions: drop`). C'est l'outil de dépannage n° 1 d'Open vSwitch, qui répond à la question « pourquoi ce paquet ne passe-t-il pas ? ».

À retenir. Une entrée = correspondance + priorité + instructions + compteurs + délais + cookie. Préciser les couches inférieures avant de filtrer sur un champ. Le pipeline part de la table 0 et n'avance que vers des tables de numéro supérieur. Sans table-miss, OpenFlow 1.3 détruit. Groupes : `all`, `select`, `indirect`, `fast-failover`. Meters : limitation par bandes `drop` ou `dscp_remark`.

Chapitre 16 — Les messages OpenFlow

OpenFlow 1.3 définit 30 types de messages, tous précédés du même en-tête de 8 octets. Une poignée d'entre eux suffit à comprendre l'essentiel : FLOW_MOD pour écrire les règles, PACKET_IN et PACKET_OUT pour échanger des paquets, MULTIPART pour lire l'état. Ce chapitre les détaille et commente un échange complet. Module 4 de la formation.

16.1 L'en-tête commun

Champ	Taille	Contenu
`version`	1 octet	0x04 pour OpenFlow 1.3
`type`	1 octet	Type du message (16.2)
`length`	2 octets	Longueur totale du message, en-tête compris
`xid`	4 octets	Identifiant de transaction, repris dans la réponse

Les champs sont codés en ordre réseau (octet de poids fort en premier). Le message le plus simple, un HELLO OpenFlow 1.3 sans option, s'écrit en hexadécimal `04 00 00 08 00 00 00 01` : version 4, type 0, longueur 8, transaction 1.

16.2 Les types de messages d'OpenFlow 1.3

Type	Nom	Sens	Rôle
0	HELLO	Les deux	Ouverture, négociation de version
1	ERROR	Les deux	Signalement d'erreur
2, 3	ECHO_REQUEST, ECHO_REPLY	Les deux	Test de vie de la connexion
4	EXPERIMENTER	Les deux	Extensions propres à un fournisseur
5, 6	FEATURES_REQUEST, FEATURES_REPLY	Contrôleur → switch	Identité et capacités du switch
7, 8, 9	GET_CONFIG_REQUEST, GET_CONFIG_REPLY, SET_CONFIG	Contrôleur → switch	Paramètres globaux (taille des remontées, fragments)
10	PACKET_IN	Switch → contrôleur	Remontée d'un paquet
11	FLOW_REMOVED	Switch → contrôleur	Suppression d'une règle, avec ses compteurs
12	PORT_STATUS	Switch → contrôleur	Port ajouté, retiré ou modifié (lien tombé)
13	PACKET_OUT	Contrôleur → switch	Injection d'un paquet
14	FLOW_MOD	Contrôleur → switch	Ajout, modification, suppression de règles
15	GROUP_MOD	Contrôleur → switch	Gestion des groupes
16	PORT_MOD	Contrôleur → switch	Configuration d'un port (activer, désactiver)
17	TABLE_MOD	Contrôleur → switch	Configuration d'une table
18, 19	MULTIPART_REQUEST, MULTIPART_REPLY	Contrôleur → switch	Statistiques et descriptions
20, 21	BARRIER_REQUEST, BARRIER_REPLY	Contrôleur → switch	Synchronisation

Type	Nom	Sens	Rôle
22, 23	QUEUE_GET_CONFIG_REQUEST, QUEUE_GET_CONFIG_REPLY	Contrôleur → switch	Files d'attente d'un port
24, 25	ROLE_REQUEST, ROLE_REPLY	Contrôleur → switch	Rôle MASTER, SLAVE ou EQUAL
26, 27, 28	GET_ASYNC_REQUEST, GET_ASYNC_REPLY, SET_ASYNC	Contrôleur → switch	Filtrage des événements asynchrones
29	METER_MOD	Contrôleur → switch	Gestion des meters

16.3 FLOW_MOD

FLOW_MOD est le message central : c'est par lui que le contrôleur programme le switch.

Champ	Rôle
`cookie`, `cookie_mask`	Identifiant de la règle ; le masque sert de filtre pour modifier ou supprimer
`table_id`	Table visée
`command`	ADD (0), MODIFY (1), MODIFY_STRICT (2), DELETE (3), DELETE_STRICT (4)
`idle_timeout`, `hard_timeout`	Délais d'expiration
`priority`	Priorité de la règle
`buffer_id`	Paquet en mémoire dans le switch à traiter aussitôt la règle installée
`out_port`, `out_group`	Filtres pour la suppression : seulement les règles qui sortent par ce port ou ce groupe
`flags`	`SEND_FLOW_REM`, `CHECK_OVERLAP`, `RESET_COUNTS`, `NO_PKT_COUNTS`, `NO_BYT_COUNTS`
`match`	Champs de correspondance (OXM)
`instructions`	Instructions et actions

Les commandes.

- **ADD** crée une règle. Si une règle de mêmes champs et de même priorité existe, elle est remplacée.

- **MODIFY** change les instructions de toutes les règles qui correspondent au filtre ; **MODIFY_STRICT** d'une seule, à champs et priorité identiques.
- **DELETE** supprime toutes les règles qui correspondent au filtre ; **DELETE_STRICT** une seule.

Si le switch ne peut appliquer une modification (table pleine, champ non pris en charge, chevauchement interdit), il répond par un message ERROR de type FLOW_MOD_FAILED.

16.4 PACKET_IN

Champ	Rôle
`buffer_id`	Le switch peut garder le paquet en mémoire et n'envoyer que son début ; sinon `NO_BUFFER` et le paquet entier
`total_len`	Longueur du paquet d'origine
`reason`	NO_MATCH (0) : entrée table-miss ; ACTION (1) : action explicite vers le contrôleur ; INVALID_TTL (2)
`table_id`, `cookie`	Table et règle qui ont provoqué la remontée
`match`	Contexte du paquet, dont le port d'entrée (`in_port`)
`data`	Le paquet, ou son début

Par défaut, un switch n'envoie que les 128 premiers octets d'un paquet mis en mémoire. L'application du TP4-B demande le paquet complet (`OFPCML_NO_BUFFER`) pour simplifier son traitement.

16.5 PACKET_OUT

PACKET_OUT demande au switch d'émettre un paquet. Il contient soit la référence d'un paquet en mémoire (`buffer_id`), soit le paquet lui-même, avec un port d'entrée fictif et une liste d'actions.

Usages :

- réinjecter le premier paquet d'un flux après une décision réactive ;
- émettre les trames LLDP de découverte de topologie ;
- répondre à une requête ARP à la place de l'hôte (proxy ARP) ;
- envoyer des paquets de test.

Avec l'action `output:TABLE`, le paquet injecté traverse le pipeline comme s'il venait d'arriver.

16.6 FLOW_REMOVED et PORT_STATUS

FLOW_REMOVED signale la disparition d'une règle qui portait le drapeau `SEND_FLOW_REM`, avec la raison (expiration par inactivité, expiration absolue, suppression, suppression du groupe) et les compteurs finaux. C'est la source des statistiques par flux terminé.

PORT_STATUS signale l'ajout, le retrait ou la modification d'un port. Une modification de l'état (lien tombé) déclenche, côté contrôleur, la mise à jour de la topologie et le recalcul des chemins.

16.7 MULTIPART : statistiques et descriptions

Les requêtes MULTIPART lisent l'état du switch. Une réponse volumineuse est découpée en plusieurs messages, tous marqués « suite à venir » sauf le dernier.

Type	Contenu de la réponse
DESC	Constructeur, matériel, logiciel, numéro de série
FLOW, AGGREGATE	Statistiques des règles, une par une ou agrégées
TABLE, TABLE_FEATURES	Occupation et capacités des tables
PORT_STATS, PORT_DESC	Compteurs des ports, description des ports
QUEUE	Statistiques des files d'attente
GROUP, GROUP_DESC, GROUP_FEATURES	Statistiques, définitions et capacités des groupes
METER, METER_CONFIG, METER_FEATURES	Statistiques, définitions et capacités des meters

`ovs-ofctl dump-flows`, `dump-ports`, `dump-groups` et `dump-meters` ne font rien d'autre qu'envoyer ces requêtes et afficher les réponses.

16.8 BARRIER, ERROR et EXPERIMENTER

- **BARRIER** : le switch ne répond à un BARRIER_REQUEST qu'après avoir traité tous les messages reçus avant lui. Le contrôleur sait ainsi que ses règles sont en place avant d'envoyer du trafic ou de passer au switch suivant.
- **ERROR** : signale un message refusé, avec un type (requête invalide, correspondance invalide, échec de FLOW_MOD, de GROUP_MOD…), un code et le début du message fautif.
- **EXPERIMENTER** : transporte les extensions propres à un fournisseur. Open vSwitch y ajoute de nombreuses extensions, dites Nicira : correspondances supplémentaires, action `learn`, suivi de connexions (`ct`), `resubmit`.

16.9 Un échange complet commenté

Voici ce qui circule entre le switch s1 et l'application `switch_acl.py` du TP4-B quand h1 envoie un premier ping à h2.

Étape	Sens	Message	Contenu et effet
1	↔	HELLO	Version 1.3 retenue
2	C → S, S → C	FEATURES_REQUEST, FEATURES_REPLY	Le switch s'identifie : dpid 1
3	C → S, S → C	MULTIPART PORT_DESC	Liste des ports
4	C → S	FLOW_MOD (ADD, priorité 0)	Table-miss : tout paquet inconnu vers le contrôleur

Étape	Sens	Message	Contenu et effet
5	C → S	FLOW_MOD (ADD, priorité 100)	Politique : h3 vers h1 interdit
6	S → C	PACKET_IN (NO_MATCH, port 1)	Requête ARP de h1, diffusée
7	C → S	PACKET_OUT (FLOOD)	Le contrôleur apprend h1 sur le port 1 et diffuse la requête
8	S → C	PACKET_IN (NO_MATCH, port 2)	Réponse ARP de h2 vers h1
9	C → S	FLOW_MOD (priorité 10, `in_port=2`, `eth_dst=h1`, sortie 1, inactivité 30 s), puis PACKET_OUT (sortie 1)	Le contrôleur apprend h2 sur le port 2 ; h1 est connu : règle installée, réponse livrée
10	S → C	PACKET_IN (NO_MATCH, port 1)	Requête ICMP de h1 vers h2
11	C → S	FLOW_MOD (`in_port=1`, `eth_dst=h2`, sortie 2), puis PACKET_OUT	Règle dans l'autre sens
12	—	Aucun	Les paquets suivants sont traités par le switch seul
13	↔	ECHO_REQUEST, ECHO_REPLY	Périodiquement, tant que la session vit

C = contrôleur, S = switch. On voit que les trois premiers paquets échangés entre h1 et h2 passent par le contrôleur, puis plus aucun.

16.10 Observer les messages

Outil	Usage
`tcpdump -i lo -w of.pcap tcp port 6653`	Capturer le canal de contrôle
`tshark -r of.pcap -Y openflow_v4`	Décoder les messages OpenFlow 1.3 (Wireshark dispose du même dissecteur)
`ovs-ofctl -O OpenFlow13 snoop s1`	Afficher en direct les messages échangés entre un bridge et son contrôleur
`ovs-ofctl -O OpenFlow13 monitor s1 watch:`	Suivre en direct les modifications des tables
Journal du contrôleur	Les applications os-ken peuvent tracer chaque événement reçu

À retenir. En-tête de 8 octets : version, type, longueur, xid. FLOW_MOD écrit (ADD, MODIFY, DELETE, en version stricte ou non), PACKET_IN remonte (raison NO_MATCH ou ACTION), PACKET_OUT injecte,

MULTIPART lit, BARRIER synchronise. Seuls les premiers paquets d'un flux passent par le contrôleur en mode réactif.

Chapitre 17 — OpenFlow en pratique : Open vSwitch et os-ken

Open vSwitch est le switch OpenFlow que vous rencontrerez en pratique, et os-ken le moyen le plus direct d'écrire un contrôleur. Ce chapitre réunit le savoir-faire : configurer un bridge, écrire et lire des règles, diagnostiquer, filtrer avec état, puis programmer une application. Module 4 de la formation.

17.1 Préparer un bridge pour OpenFlow

```
sudo ovs-vsctl add-br br-of
sudo ovs-vsctl set bridge br-of protocols=OpenFlow13      # versions acceptées
sudo ovs-vsctl set-fail-mode br-of secure                 # aucune commutation sans
règle
sudo ovs-vsctl set-controller br-of tcp:127.0.0.1:6653    # facultatif : contrôleur
sudo ovs-vsctl list controller                            # is_connected, role
sudo ovs-ofctl -O OpenFlow13 show br-of                   # numéros OpenFlow des
ports
```

Chaque interface reçoit un numéro de port OpenFlow (`ofport`), distinct de son nom. `ovs-ofctl show` affiche la correspondance ; `ovs-vsctl get Interface h1-port ofport` donne le numéro d'une interface. `ovs-ofctl` accepte aussi les noms de ports dans les règles, ce qui les rend plus lisibles.

17.2 Écrire des règles avec ovs-ofctl

Les commandes.

Commande	Effet
`add-flow   "<règle>"`	Ajoute une règle
`add-flows   <fichier>`	Ajoute les règles d'un fichier
`replace-flows   <fichier>`	Remplace toute la table par le contenu du fichier, en ne touchant qu'aux différences
`diff-flows   <fichier>`	Compare la table au fichier
`mod-flows`, `del-flows`	Modifie ou supprime (`--strict` pour une seule règle)
`dump-flows   [filtre]`	Affiche les règles et leurs compteurs (`--no-stats` pour une sortie comparable)
`dump-ports`, `dump-ports-desc`, `dump-tables`, `dump-aggregate`	Compteurs et descriptions
`add-group`, `dump-groups`, `add-meter`, `dump-meters`	Groupes et meters

La syntaxe d'une règle.
`[table=N,][priority=N,][cookie=X,][idle_timeout=S,][hard_timeout=S,]correspondance,actions=liste`. Sans priorité explicite, `ovs-ofctl` utilise 32 768.

Action (`ovs-ofctl`)	Effet
`output:3`, `in_port`, `normal`, `flood`, `all`, `local`	Sorties
`controller`, `controller(max_len=128)`	Remontée au contrôleur
`drop`	Destruction explicite (liste vide)
`push_vlan:0x8100`, `pop_vlan`	Étiquetage VLAN
`set_field:4106->vlan_vid`	VLAN 10 : la valeur inclut le bit « VLAN présent » (0x1000 + 10 = 4 106)
`mod_vlan_vid:10`, `strip_vlan`	Équivalents simplifiés propres à OVS
`set_field:10.0.0.9->ip_dst`, `mod_dl_dst:<mac>`	Réécritures
`dec_ttl`	Décrément du TTL
`goto_table:1`, `resubmit(,1)`	Passage à une autre table (le second est une extension OVS qui autorise aussi les retours)
`group:1`, `meter:1`	Groupe, meter
`ct(...)`, `learn(...)`	Suivi de connexions, apprentissage sans contrôleur (extensions OVS)

Exemple : un port d'accès en OpenFlow. Le trafic entrant par le port 1 est étiqueté VLAN 10 et envoyé vers le trunk (port 3) ; le trafic du VLAN 10 venant du trunk est désétiqueté vers le port 1.

```
priority=100,in_port=1,actions=push_vlan:0x8100,set_field:4106->vlan_vid,output:3
priority=100,in_port=3,dl_vlan=10,actions=pop_vlan,output:1
```

Lire `dump-flows`. Chaque ligne indique le cookie, la durée de vie, la table, les compteurs `n_packets` et `n_bytes`, les délais, la priorité, la correspondance et les actions. Une règle dont les compteurs restent à zéro pendant un test n'est pas celle que l'on croit : c'est souvent le premier indice d'une erreur de priorité ou de prérequis.

17.3 Diagnostiquer

Outil	Ce qu'il montre
`ovs-appctl ofproto/trace   <paquet>`	Le parcours simulé d'un paquet, table par table
`ovs-ofctl dump-flows  `	Les règles OpenFlow et leurs compteurs
`ovs-appctl dpif/dump-flows  `	Les megaflows installés dans le datapath pour ce

Outil	Ce qu'il montre
	bridge
`ovs-appctl bridge/dump-flows  `	Toutes les règles, y compris les règles cachées ajoutées par OVS
`ovs-appctl vlog/set ofproto:file:dbg`	Journalisation détaillée dans `/var/log/openvswitch/ovs-vswitchd.log`
`ovs-vsctl show`, `list interface`	Configuration, état et erreurs des interfaces

Symptôme	Vérification
Aucun trafic	Mode de secours `secure` sans règle ? Table-miss absente ?
Règle sans effet	Prérequis manquant (`ip`, `tcp`) ? Priorité inférieure à une autre règle ? Mauvais numéro de port ?
`ovs-ofctl` répond « version negotiation failed »	Ajouter `-O OpenFlow13`
Le contrôleur ne voit pas le switch	`ovs-vsctl list controller` : adresse, port, `is_connected`
Interface en erreur	`ovs-vsctl list interface <nom>` : champ `error`

17.4 Le filtrage à états avec conntrack

OpenFlow est sans état : une règle ne sait pas si un paquet appartient à une connexion déjà autorisée. Pour autoriser une connexion et ses réponses sans ouvrir le sens inverse, Open vSwitch s'appuie sur le suivi de connexions du noyau Linux, par l'action `ct`.

1. Un paquet non encore suivi (`ct_state=-trk`) est confié à conntrack par `ct(table=1)`, puis réinjecté dans la table 1 avec son état.
2. La table 1 décide selon l'état : `+new` (nouvelle connexion), `+est` (connexion établie), `+rel` (paquet lié, par exemple une erreur ICMP), `+inv` (invalide).
3. Une nouvelle connexion autorisée est enregistrée par `ct(commit)` ; ses réponses seront ensuite reconnues comme `+est`.

```
table=0,priority=100,ip,ct_state=-trk,actions=ct(table=1)
table=0,priority=10,arp,actions=normal
table=0,priority=0,actions=drop
table=1,priority=300,ip,ct_state=+trk+est,actions=normal
table=1,priority=300,ip,ct_state=+trk+rel,actions=normal
table=1,priority=200,tcp,ct_state=+trk+new,nw_src=10.0.0.2,nw_dst=10.0.0.3,tp_dst=80,
actions=ct(commit),normal
table=1,priority=100,ip,ct_state=+trk+new,actions=drop
```

Ici, h2 peut ouvrir une connexion HTTP vers h3 et recevoir les réponses ; h3 ne peut rien initier. C'est ainsi qu'OVN réalise les groupes de sécurité d'OpenStack et les NetworkPolicy de Kubernetes.

17.5 Ce qu'installe OVN

Sur un nœud OpenStack ou OpenShift, `sudo ovs-ofctl dump-flows br-int | wc -l` affiche souvent plusieurs milliers de règles. Chaque étape du pipeline logique d'OVN (filtrage d'entrée, ACL, équilibrage de charge, routage, NAT) y devient une table OpenFlow. Pour dépanner, on raisonne d'abord au niveau logique avec `ovn-trace`, puis on descend au niveau OpenFlow avec `ofproto/trace`.

17.6 Anatomie d'une application os-ken

La structure. Une application est une classe qui hérite de `OSKenApp` (ou `RyuApp` avec Ryu) et déclare les versions d'OpenFlow qu'elle parle.

- `OFP_VERSIONS = [ofproto_v1_3.OFP_VERSION]` fixe la version.
- Le constructeur initialise l'état de l'application, par exemple la table MAC par switch.
- Des méthodes décorées par `@set_ev_cls(<événement>, <phase>)` sont appelées à chaque événement du type indiqué.

Les phases de la connexion (*dispatchers*).

Phase	Moment
`HANDSHAKE_DISPATCHER`	Échange des HELLO
`CONFIG_DISPATCHER`	Attente de la réponse FEATURES : moment d'installer les règles initiales
`MAIN_DISPATCHER`	Fonctionnement normal
`DEAD_DISPATCHER`	Connexion perdue

Les événements courants.

Événement	Message d'origine
`EventOFPSwitchFeatures`	FEATURES_REPLY
`EventOFPPacketIn`	PACKET_IN
`EventOFPPortStatus`	PORT_STATUS
`EventOFPFlowRemoved`	FLOW_REMOVED
`EventOFPFlowStatsReply`, `EventOFPPortStatsReply`	Réponses MULTIPART
`EventOFPErrorMsg`	ERROR
`EventOFPStateChange`	Changement de phase d'un switch

Les objets manipulés.

- `ev.msg` : le message reçu ; `ev.msg.datapath` : le switch concerné.
- `dp.id` : l'identifiant du switch ; `dp.ofproto` : les constantes de la version (ports réservés, commandes) ; `dp.ofproto_parser` : les classes de messages ; `dp.send_msg()` : l'envoi.
- `parser.OFPMatch(...)`, `parser.OFPActionOutput(...)`, `parser.OFPInstructionActions(...)`, `parser.OFPFlowMod(...)`, `parser.OFPPacketOut(...)` construisent les messages.
- `packet.Packet(msg.data).get_protocol(ethernet.ethernet)` analyse un paquet remonté ; les modules `arp`, `ipv4`, `tcp`, `icmp` décodent les couches supérieures.

L'application `switch_acl.py` du TP4-B, reproduite dans le cahier de TP, assemble ces éléments : table-miss et règles de blocage dans `CONFIG_DISPATCHER`, apprentissage et installation réactive dans le gestionnaire de PACKET_IN.

Une extension : relever les statistiques. Ce module, testé avec os-ken 4.2.2, interroge chaque switch toutes les 10 secondes et journalise les compteurs de ses règles.

```
from os_ken.base import app_manager
from os_ken.controller import ofp_event
from os_ken.controller.handler import MAIN_DISPATCHER, DEAD_DISPATCHER, set_ev_cls
from os_ken.lib import hub
from os_ken.ofproto import ofproto_v1_3


class Moniteur(app_manager.OSKenApp):
    OFP_VERSIONS = [ofproto_v1_3.OFP_VERSION]

    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.datapaths = {}
        self.monitor = hub.spawn(self._poll)        # tâche de fond

    @set_ev_cls(ofp_event.EventOFPStateChange, [MAIN_DISPATCHER, DEAD_DISPATCHER])
    def on_state(self, ev):
        dp = ev.datapath
        if ev.state == MAIN_DISPATCHER:
            self.datapaths[dp.id] = dp
        else:
            self.datapaths.pop(dp.id, None)

    def _poll(self):
        while True:
            for dp in list(self.datapaths.values()):
                dp.send_msg(dp.ofproto_parser.OFPFlowStatsRequest(dp))
            hub.sleep(10)

    @set_ev_cls(ofp_event.EventOFPFlowStatsReply, MAIN_DISPATCHER)
    def on_flow_stats(self, ev):
        for stat in ev.msg.body:
            self.logger.info("prio=%d paquets=%d octets=%d match=%s",
                             stat.priority, stat.packet_count, stat.byte_count,
stat.match)
```

Plusieurs applications se lancent ensemble : `python3 lancer_controleur.py switch_acl.py moniteur.py`.

17.7 Bonnes pratiques de développement

- Installer l'état initial (table-miss, politique) à chaque connexion d'un switch, de façon idempotente.
- Marquer les règles de l'application par un cookie, pour les retrouver et les nettoyer.
- Traiter les PORT_STATUS : un port qui tombe doit invalider les adresses MAC apprises dessus.
- Ne jamais bloquer dans un gestionnaire d'événement : os-ken fonctionne par coroutines (eventlet), et un appel bloquant fige tout le contrôleur. Utiliser `hub.spawn` et `hub.sleep`.
- Fixer des délais d'expiration aux règles réactives.
- Tester avec Mininet de façon automatisée (scénarios de ping et de débit), et versionner le code comme toute application.

À retenir. Sur Open vSwitch : `protocols=OpenFlow13`, `fail-mode secure`, `ovs-ofctl -O OpenFlow13` pour écrire, `ofproto/trace` pour comprendre. `ct()` apporte le filtrage à états. Une application os-ken est une classe dont les méthodes réagissent aux événements OpenFlow, par phase de connexion.

Chapitre 18 — Au-delà d'OpenFlow

OpenFlow programme des tables dont la structure est fixée par la norme ; les technologies suivantes rendent programmable le traitement lui-même. Ce chapitre présente P4 pour les switches, eBPF pour les hôtes Linux, les systèmes réseau ouverts, et les tendances à suivre. Module 4 de la formation, approfondissement.

18.1 Ce qui a motivé la suite

- **Une liste de champs figée.** Chaque nouveau protocole (un nouvel en-tête de tunnel, par exemple) suppose une nouvelle version de la norme. Le nombre de champs est passé de 12 en version 1.0 à plus de 40 en version 1.5, sans jamais suivre le rythme des besoins.
- **Un décalage avec le matériel.** Le pipeline abstrait d'OpenFlow, avec ses tables enchaînées, correspond mal aux pipelines fixes des circuits de commutation, d'où des implémentations partielles.
- **Des tables matérielles limitées.** Les mémoires associatives (TCAM) qui permettent les correspondances avec jokers sont coûteuses et petites.

La réponse a été de rendre le plan de données lui-même programmable, indépendamment de tout protocole.

18.2 P4 et P4Runtime

P4 (*Programming Protocol-independent Packet Processors*) est un langage, proposé en 2014, pour décrire le traitement des paquets dans un équipement. Sa version actuelle, P4_16, date de 2016. Un programme P4 décrit :

- un **analyseur** (*parser*) : une machine à états qui reconnaît les en-têtes, y compris des en-têtes nouveaux définis par le programmeur ;
- des **tables de correspondance-action** : les clés lues, les actions possibles, la taille ;

- des **blocs de contrôle** qui enchaînent les tables ;
- un **réassembleur** (*deparser*) qui réécrit les en-têtes en sortie.

Exemple indicatif : une table de filtrage IPv4 dans un bloc de contrôle.

```
action drop() { mark_to_drop(standard_metadata); }

table acl {
    key = {
        hdr.ipv4.srcAddr : ternary;
        hdr.ipv4.dstAddr : ternary;
    }
    actions = { drop; NoAction; }
    size = 1024;
    default_action = NoAction();
}

apply {
    if (hdr.ipv4.isValid()) { acl.apply(); }
}
```

Le programme est compilé pour une cible : un switch logiciel de référence (bmv2), un circuit de commutation programmable, une carte réseau intelligente ou un FPGA.

P4Runtime est l'API, sur gRPC, par laquelle un contrôleur remplit les tables définies par le programme P4. C'est le successeur conceptuel d'OpenFlow : le contrôleur écrit toujours des entrées dans des tables, mais ces tables sont celles que l'on a définies, et non celles fixées par une norme. ONOS et la fabric SD-Fabric le prennent en charge.

P4 a aussi popularisé la **télémétrie dans la bande** (INT) : chaque switch ajoute au paquet des métadonnées (horodatage, occupation de file), ce qui donne une mesure très fine de la latence de chaque saut.

Le marché des circuits P4 s'est resserré (Intel a arrêté en 2023 le développement de sa gamme Tofino), mais le langage reste utilisé pour les cartes programmables et la recherche.

18.3 eBPF et XDP

eBPF permet d'exécuter dans le noyau Linux de petits programmes, vérifiés avant chargement puis compilés à la volée. Ils s'accrochent à des points précis :

- **XDP**, au plus près du pilote de la carte, avant même l'allocation mémoire du paquet : filtrage anti-déni de service, répartition de charge à très haut débit ;
- **tc**, en entrée et en sortie d'interface : politique, redirection, encapsulation ;
- **sockets** et **cgroups** : contrôle par application ou par conteneur.

Les programmes partagent leur état dans des tables appelées *maps*, lisibles et modifiables depuis l'espace utilisateur.

Cilium applique cette technique à Kubernetes : il remplace kube-proxy, applique les NetworkPolicy jusqu'au niveau 7 (HTTP, gRPC) et offre avec Hubble une observabilité fine des flux entre pods. eBPF joue ici le rôle d'un plan de données programmable dans chaque nœud, l'API Kubernetes celui de source de vérité et l'agent Cilium celui de contrôleur local.

Commande	Effet
`bpftool prog show`, `bpftool map show`	Programmes et maps chargés
`ip link show`	Indique un programme XDP attaché à une interface
`cilium status`, `hubble observe`	État de Cilium, flux observés entre pods

18.4 Les systèmes réseau ouverts

La désagrégation sépare le matériel du switch (*white box*) de son système d'exploitation.

- **SONiC**, créé par Microsoft et aujourd'hui hébergé par la Linux Foundation, est un système réseau open source fondé sur Linux. Ses fonctions tournent dans des conteneurs et dialoguent avec le circuit de commutation par une interface normalisée, SAI (*Switch Abstraction Interface*).
- **ONIE** est l'environnement d'installation qui permet de charger le système de son choix sur un switch nu.

Pour une équipe système, la conséquence est directe : le switch devient un serveur Linux spécialisé, administrable avec les mêmes outils (paquets, conteneurs, Ansible, supervision).

18.5 Les tendances à suivre

Tendance	En quoi elle prolonge le SDN
Réseaux à intentions et boucle fermée	La vérification continue de l'intention devient la norme de l'automatisation
Jumeau numérique	Chaque changement est simulé sur une copie du réseau avant d'être appliqué
SRv6 et WAN pilotés	Le contrôleur impose des chemins de bout en bout sans état dans le cœur
Fonctions réseau cloud natives	Les VNF deviennent des CNF sur Kubernetes, avec Multus et SR-IOV pour les interfaces supplémentaires
Maillage de services (*service mesh*)	La politique et l'observabilité montent au niveau 7, entre microservices (Istio, Linkerd)
DPU et cartes intelligentes	Commutation, chiffrement et filtrage déportés sur la carte réseau, hors du CPU de l'hôte
Tranches de réseau 5G	Le découpage logique d'une infrastructure avec des garanties par usage

18.6 Récapitulatif des interfaces de programmation

Technologie	Ce qui est programmé	Où	Par qui

Technologie	Ce qui est programmé	Où	Par qui
OpenFlow	Le contenu de tables de structure normalisée	Switch OpenFlow, Open vSwitch	Un contrôleur
P4 et P4Runtime	La structure des tables (P4), puis leur contenu (P4Runtime)	Circuit programmable, carte, logiciel	Le développeur, puis le contrôleur
eBPF	Le traitement des paquets dans le noyau	Hôte Linux	Un agent local (Cilium, par exemple)
NETCONF, gNMI	La configuration de fonctions existantes	Tout équipement qui les prend en charge	Un outil d'automatisation ou un contrôleur
OVSDB	La configuration d'Open vSwitch	Hôte	OVN, un contrôleur, `ovs-vsctl`

18.7 Se tenir à jour

- La documentation d'Open vSwitch et d'OVN, très complète, et ses pages de manuel (`man ovs-ofctl`, `man ovs-fields`, `man ovs-actions`).
- Le suivi des groupes de travail de l'IETF (NVO3, BESS, SPRING) sur le site datatracker.ietf.org.
- Les projets de la Linux Foundation Networking et de la CNCF.
- Les communautés d'exploitants francophones : FRnOG (groupe des opérateurs réseau français) et les JRES (Journées réseaux de l'enseignement et de la recherche).

À retenir. OpenFlow remplit des tables normalisées ; P4 définit les tables et P4Runtime les remplit ; eBPF programme le noyau Linux des hôtes. Le principe SDN (un contrôle centralisé qui programme un plan de données) survit sous toutes ces formes.

Annexe A — Questions de révision

Ces 44 questions couvrent les quatre modules ; les corrigés suivent. Essayez d'y répondre sans relire le chapitre, puis vérifiez.

Module 1 — La virtualisation des réseaux (chapitres 2 à 4)

1. Citez les trois propriétés recherchées par la virtualisation réseau.
2. Quelle différence y a-t-il entre un VLAN et une VRF ?
3. Citez quatre éléments isolés par un network namespace.
4. Comment un conteneur Docker en mode `bridge` est-il raccordé au réseau, et comment sort-il vers l'extérieur ?
5. Quel outil d'Open vSwitch utilisez-vous pour ajouter un port, pour lire les règles OpenFlow, pour afficher les adresses MAC apprises ?
6. Pourquoi le premier paquet d'un flux est-il plus lent que les suivants dans Open vSwitch ?
7. Quelle différence y a-t-il entre une VNF et une CNF ?

8. Montrez par deux exemples que SDN et NFV sont indépendants.
9. Qu'est-ce que la micro-segmentation, et quel risque traite-t-elle ?
10. Citez deux techniques d'accélération du réseau d'un hôte et leur contrepartie.

Module 2 — Architecture et normalisation (chapitres 5 à 8)

11. Pourquoi une fabric leaf-spine moderne se passe-t-elle du Spanning Tree ?
12. Un leaf offre 32 ports serveurs à 25 Gbit/s et 4 liens montants à 100 Gbit/s. Quel est son taux de sursouscription ?
13. Pourquoi l'ECMP répartit-il le trafic par flux et non paquet par paquet ?
14. Quelle est la taille de l'étiquette 802.1Q, que contient-elle et combien de VLAN permet-elle ?
15. Détaillez le surcoût de VXLAN sur IPv4 et déduisez le MTU interne maximal sur un underlay à 1 500 octets.
16. À quoi sert le port UDP source d'un paquet VXLAN ?
17. Qu'apporte Geneve par rapport à VXLAN ?
18. Quel type de route EVPN annonce une adresse MAC ? Lequel annonce la participation d'un VTEP à un segment ?
19. Quels sont les rôles du NFVO, du VNFM et du VIM ? Donnez un exemple réel de VIM.
20. Qu'apporte NETCONF par rapport à la configuration en ligne de commande ?
21. Sur un overlay, le ping fonctionne mais les transferts de fichiers se bloquent. Quelle est la cause probable, et quels sont les remèdes ?

Module 3 — Les techniques et applications du SDN (chapitres 9 à 13)

22. Énoncez les quatre principes du SDN.
23. Définissez les interfaces nord, sud et est-ouest, avec un exemple pour chacune.
24. Pourquoi ONOS affiche-t-il 8 liens pour une topologie en anneau de 4 câbles entre switches ?
25. Que se passe-t-il pour le trafic, en mode réactif puis en mode proactif, si tous les contrôleurs tombent ?
26. Qu'est-ce qu'une intention ? Que devient-elle quand son dernier chemin disparaît, puis réapparaît ?
27. Quelle différence y a-t-il entre restauration et protection ?
28. Quels sont les rôles de `ovn-northd` et de `ovn-controller` ?
29. Pourquoi un cluster de contrôleurs compte-t-il un nombre impair d'instances ?
30. Citez quatre mesures de durcissement d'une architecture SDN.
31. Décrivez les étapes d'un pipeline GitOps qui déploie une politique réseau.

Module 4 — Les réseaux OpenFlow (chapitres 14 à 18)

32. Quel port TCP utilise OpenFlow aujourd'hui, et lequel utilisaient les premières implémentations ?
33. Dans quel ordre circulent les messages à l'ouverture d'une session OpenFlow 1.3 ?
34. Que permettent les rôles MASTER, SLAVE et EQUAL ?
35. Quelle est la différence entre les modes de secours `secure` et `standalone`, et lequel choisir pour un réseau segmenté par OpenFlow ?
36. En OpenFlow 1.3, que devient un paquet qui ne correspond à aucune entrée d'une table dépourvue de table-miss ?

37. Pourquoi la règle `priority=200,nw_dst=10.0.0.1,actions=drop` est-elle incorrecte ? Corrigez-la.
38. Expliquez la règle `priority=300,tcp,nw_src=10.0.0.3,nw_dst=10.0.0.1,tp_dst=443,actions=group:1`.
39. Écrivez les commandes qui limitent à 10 Mbit/s le trafic IPv4 de 10.0.0.2 vers 10.0.0.1 et le livrent sur le port 1.
40. Quel type de groupe choisir pour un miroir, pour une répartition de charge, pour une bascule sur panne ?
41. Quelles sont les raisons possibles d'un PACKET_IN en OpenFlow 1.3 ?
42. Quelle différence y a-t-il entre DELETE et DELETE_STRICT ?
43. Pourquoi faut-il conntrack pour autoriser une connexion et ses réponses sans ouvrir le sens inverse ?
44. Qu'apporte P4 par rapport à OpenFlow ?

Corrigés — Module 1

N°	Réponse
1	Abstraction, isolation, programmabilité (2.2)
2	Un VLAN découpe un réseau de niveau 2 en segments ; une VRF découpe un routeur en tables de routage indépendantes (niveau 3) (2.3)
3	Interfaces, adresses et routes, cache ARP, règles netfilter, sockets et ports, paramètres `net.*` (3.1)
4	Un namespace par conteneur, une paire veth branchée sur le bridge `docker0` ; sortie par NAT (MASQUERADE), publication de ports par DNAT (3.7)
5	`ovs-vsctl add-port`, `ovs-ofctl dump-flows`, `ovs-appctl fdb/show` (3.5)
6	Il ne correspond à aucune entrée du cache du datapath : il remonte à `ovs-vswitchd`, qui exécute le pipeline OpenFlow et installe un megaflow pour les suivants (3.5)
7	Une VNF s'exécute en VM ; une CNF est conçue en microservices et s'exécute en conteneurs, en général sur Kubernetes (4.4)
8	Un pare-feu virtuel dans un réseau classique est de la NFV sans SDN ; des switches physiques pilotés en OpenFlow sont du SDN sans NFV (4.6)
9	Le filtrage appliqué au plus près de chaque VM ou pod, par identité ou étiquette ; elle limite le mouvement latéral d'un attaquant (4.9)
10	Par exemple : SR-IOV (contourne le switch logiciel, donc ses politiques, et complique la migration) ; DPDK (cœurs CPU dédiés en permanence) (3.8)

Corrigés — Module 2

N°	Réponse

N°	Réponse
11	L'underlay est entièrement routé : il n'y a pas de domaine de niveau 2 entre switches, donc pas de boucle à couper ; tous les liens servent grâce à l'ECMP (5.3)
12	32 × 25 = 800 Gbit/s vers les serveurs, 4 × 100 = 400 Gbit/s vers les spines : 2:1 (5.2)
13	Pour que les paquets d'un même flux suivent le même chemin et arrivent dans l'ordre (5.2)
14	4 octets : TPID 0x8100, priorité (3 bits), DEI (1 bit), VID (12 bits) ; 4 094 VLAN utilisables (6.2)
15	14 (Ethernet externe) + 20 (IP) + 8 (UDP) + 8 (VXLAN) = 50 octets. Paquet externe de 1 500 octets : 1 500 − 20 − 8 − 8 − 14 = 1 450 octets pour le paquet IP interne (6.4, 6.9)
16	Il est calculé par hachage des en-têtes internes, ce qui répartit les flux sur les chemins ECMP de l'underlay (6.4)
17	Un champ indiquant le type de charge transportée et des options TLV de longueur variable pour transporter des métadonnées (6.5)
18	Type 2 (MAC/IP Advertisement) ; type 3 (Inclusive Multicast Ethernet Tag) (7.3)
19	NFVO : compose et orchestre les services ; VNFM : gère le cycle de vie des VNF ; VIM : pilote les ressources virtualisées. Exemples de VIM : OpenStack, Kubernetes (8.4)
20	Des transactions (configuration candidate, validation, `commit`, aucun effet partiel en cas d'erreur) et des données structurées par des modèles YANG (8.5)
21	Un MTU trop grand pour l'encapsulation, aggravé si les messages ICMP sont filtrés. Remèdes : MTU de l'underlay relevé, MTU des interfaces virtuelles abaissé à 1 450, ajustement du MSS TCP (6.9)

Corrigés — Module 3

N°	Réponse
22	Séparation contrôle/données, centralisation logique, abstraction, programmabilité (9.4)
23	Sud : contrôleur vers équipements (OpenFlow, OVSDB, NETCONF). Nord : applications vers contrôleur (API REST d'ONOS). Est-ouest : entre instances ou contrôleurs (réplication du cluster) (9.5)
24	La découverte LLDP produit un lien orienté par sens de chaque câble : 4 × 2 = 8 (11.2)
25	Réactif : les flux déjà installés continuent jusqu'à expiration, les nouveaux flux ne passent plus. Proactif : le trafic continue, seuls les changements sont impossibles (11.1, 10.8)

N°	Réponse
26	La description d'un résultat voulu (« h1 joint h3 »), que le contrôleur traduit en règles et maintient. Sans chemin, elle passe en FAILED ; elle est réinstallée automatiquement dès qu'un chemin réapparaît (11.4)
27	Restauration : le contrôleur recalcule après la panne. Protection : un chemin de secours préinstallé (groupe fast-failover) bascule dans le switch, sans le contrôleur (11.6)
28	`ovn-northd` traduit la configuration logique (base Northbound) en flux logiques (base Southbound) ; `ovn-controller`, sur chaque hôte, traduit ces flux en règles OpenFlow et tunnels sur l'Open vSwitch local (10.6)
29	Le consensus exige une majorité : 3 instances tolèrent 1 panne, 4 aussi ; une instance de plus ne sert que si le total devient impair (10.8)
30	Par exemple : identifiants par défaut changés, TLS sur le canal sud, API nord restreinte au réseau d'administration, mode `secure`, limitation des PACKET_IN, journalisation vers le SIEM (13.3)
31	Branche et demande de fusion, validation du fichier, test en laboratoire, relecture, application par l'API après fusion, vérification, détection de dérive (12.7)

Corrigés — Module 4

N°	Réponse
32	6653, attribué par l'IANA ; anciennement 6633 (14.4)
33	HELLO dans les deux sens, FEATURES_REQUEST et FEATURES_REPLY, MULTIPART PORT_DESC, SET_CONFIG, FLOW_MOD de la table-miss, puis ECHO périodiques (14.4)
34	Plusieurs contrôleurs sur un même switch : un seul MASTER en lecture-écriture, des SLAVE en lecture seule, ou des EQUAL tous en lecture-écriture (14.5)
35	`secure` garde les règles installées et ne commute rien d'autre ; `standalone` repasse en switch classique, ce qui efface de fait la segmentation. Choisir `secure` (14.6)
36	Il est détruit (15.6)
37	Le prérequis manque : il faut préciser IPv4. Correction : `priority=200,ip,nw_dst=10.0.0.1,actions=drop` (15.2)
38	Les paquets TCP de 10.0.0.3 vers 10.0.0.1 port 443 sont confiés au groupe 1 (par exemple un groupe `all` qui livre au serveur et copie vers une sonde), avec une priorité supérieure aux règles générales (15.4, 15.8)
39	`ovs-ofctl -O OpenFlow13 add-meter s1 meter=1,kbps,band=type=drop,rate=10000` puis `ovs-ofctl -O OpenFlow13 add-flow s1 "priority=150,ip,nw_src=10.0.0.2,nw_dst=10.0.0.1,actions=meter:1,output:`

N°	Réponse
	`1"` (15.9)
40	`all`, `select`, `fast-failover` (15.8)
41	NO_MATCH (entrée table-miss), ACTION (action explicite vers le contrôleur), INVALID_TTL (16.4)
42	DELETE retire toutes les règles qui correspondent au filtre ; DELETE_STRICT seulement celle dont les champs et la priorité sont identiques (16.3)
43	OpenFlow est sans état : sans suivi de connexion, autoriser les réponses oblige à autoriser tout le sens inverse. conntrack distingue les paquets d'une connexion établie (`+est`) des nouvelles connexions (`+new`) (17.4)
44	P4 permet de définir les en-têtes reconnus et la structure des tables, au lieu de les subir ; P4Runtime remplit ensuite ces tables (18.2)

Annexe B — Glossaire

Terme	Définition	Chapitre
ACL	Liste de règles de contrôle d'accès qui autorisent ou refusent des flux	10, 13
API nord (northbound)	Interface entre les applications et le contrôleur, en général REST	9
API sud (southbound)	Interface entre le contrôleur et les équipements : OpenFlow, OVSDB, NETCONF, P4Runtime	9
ARP	Protocole qui associe une adresse IPv4 à une adresse MAC sur un réseau local	1
BFD	Protocole de détection rapide de panne d'un lien	5
BGP	Protocole de routage entre systèmes autonomes, utilisé aussi dans les datacenters et par EVPN	5, 7
Bridge	Switch logiciel ; sous Linux, le bridge du noyau ; dans OVS, un switch virtuel	3
BUM	Trafic inondé : Broadcast, Unknown unicast, Multicast	1, 6
CNF	Fonction réseau conçue en microservices et exécutée en conteneurs	4
CNI	Spécification des plugins qui raccordent les pods	3

Terme	Définition	Chapitre
	Kubernetes au réseau	
conntrack	Suivi de connexions du noyau Linux, utilisé par OVS pour le filtrage à états	17
Contrôleur SDN	Logiciel qui centralise le plan de contrôle et programme les équipements	10
Cookie	Valeur attachée à une règle OpenFlow pour identifier l'application qui l'a posée	15
Datapath	Partie d'Open vSwitch qui achemine les paquets à grande vitesse	3
Datapath ID (dpid)	Identifiant de 64 bits d'un switch OpenFlow	14
DPDK	Bibliothèques de traitement des paquets en espace utilisateur, par scrutation des cartes	3
eBPF	Programmes vérifiés exécutés dans le noyau Linux, utilisés notamment par Cilium	18
ECMP	Répartition du trafic sur plusieurs chemins de même coût, flux par flux	5
EtherType	Champ de la trame Ethernet qui indique le protocole transporté	1
EVPN	Plan de contrôle fondé sur BGP qui annonce adresses MAC et IP et appartenance aux segments	7
Fail mode	Comportement d'un switch OpenFlow privé de contrôleur : `secure` ou `standalone`	14
FDB	Table des adresses MAC apprises par un switch	1, 3
FLOW_MOD	Message OpenFlow qui ajoute, modifie ou supprime des règles	16
Flux	Ensemble de paquets partageant les mêmes caractéristiques, souvent le quintuplet	1
Geneve	Encapsulation overlay extensible sur UDP 6081 (RFC 8926)	6
gNMI	Protocole de gestion et de télémétrie sur gRPC, issu d'OpenConfig	8
Groupe (OpenFlow)	Liste de seaux d'actions : `all`, `select`, `indirect`,	15

Terme	Définition	Chapitre
	`fast-failover`	
Intention	Description d'un résultat voulu, traduite et maintenue par le contrôleur	11
Leaf, spine	Switches d'accès et switches d'interconnexion d'une architecture Clos	5
LLDP	Protocole de découverte de voisins, utilisé par les contrôleurs pour découvrir les liens	11
Locataire (tenant)	Utilisateur ou organisation isolé des autres sur une infrastructure partagée	2, 5
MANO	Gestion et orchestration NFV : NFVO, VNFM, VIM	8
Megaflow	Entrée généralisée du cache du datapath d'Open vSwitch	3
Meter	Objet OpenFlow qui mesure et limite un débit	15
Micro-segmentation	Filtrage au plus près de chaque charge, par identité ou étiquette	4
MTU	Taille maximale d'un paquet IP sur un lien, 1 500 octets en Ethernet standard	1, 6
Namespace réseau	Pile réseau isolée au sein du noyau Linux	3
NETCONF	Protocole de configuration transactionnelle des équipements (RFC 6241)	8
NFV	Virtualisation des fonctions réseau : exécuter en logiciel ce que faisaient des boîtiers	4
NFVI	Infrastructure qui héberge les VNF	8
NFVO	Orchestrateur NFV, qui compose les services réseau	8
NVE, NVA	Dans le cadre NVO3 : fonction d'encapsulation (VTEP) et autorité qui la renseigne	7
NVO3	Groupe et cadre de l'IETF pour les overlays de virtualisation réseau	7
ONOS	Contrôleur SDN open source en Java, utilisé en TP	10
OpenDaylight	Plateforme SDN open source en Java, pilotée par modèles YANG	10

Terme	Définition	Chapitre
OpenFlow	Protocole qui permet à un contrôleur de programmer les tables de flux d'un switch	14
os-ken, Ryu	Frameworks Python pour écrire des contrôleurs OpenFlow ; os-ken est le fork maintenu	10, 17
OVN	Couche de virtualisation réseau d'Open vSwitch, utilisée par OpenStack et OpenShift	10
OVS (Open vSwitch)	Switch logiciel programmable, de référence pour la virtualisation	3
OVSDB	Protocole et base de configuration d'Open vSwitch (RFC 7047)	3, 8
Overlay	Réseau virtuel transporté dans des tunnels au-dessus d'un underlay	5, 6
P4, P4Runtime	Langage de programmation du plan de données et API de remplissage de ses tables	18
PACKET_IN, PACKET_OUT	Messages OpenFlow de remontée d'un paquet au contrôleur et d'injection d'un paquet	16
Passerelle anycast	Passerelle présente avec la même adresse sur tous les leafs ou hyperviseurs	5
Pipeline	Enchaînement des tables de flux, de la table 0 vers les tables suivantes	15
Port d'accès, trunk	Port appartenant à un seul VLAN, sans étiquette ; port transportant plusieurs VLAN étiquetés	6
Proactif, réactif	Règles installées d'avance ; règles installées à l'arrivée du premier paquet	11
QinQ	Empilement de deux étiquettes VLAN (IEEE 802.1ad)	6
RESTCONF	Accès REST aux données modélisées en YANG (RFC 8040)	8
Route Target	Attribut BGP qui détermine quels équipements importent une route EVPN ou VPN	7
SDN	Approche qui sépare et centralise le plan de contrôle et le rend programmable	9
SD-WAN	WAN d'entreprise piloté par un orchestrateur, sur plusieurs transports	12

Terme	Définition	Chapitre
Segment Routing	Routage par la source, sur MPLS ou IPv6 (SRv6)	6
SFC, NSH	Chaînage ordonné de fonctions réseau ; en-tête dédié qui le porte	4
Spanning Tree	Protocole qui coupe les boucles d'un réseau de niveau 2	1, 5
SR-IOV	Partage d'une carte physique en cartes virtuelles attribuées directement aux VM	3
Table-miss	Entrée de priorité 0 sans critère, qui définit le comportement par défaut d'une table	15
Tap	Interface de niveau 2 reliée à un processus, carte réseau d'une VM	3
Underlay	Réseau physique qui transporte les paquets encapsulés	5
veth	Paire d'interfaces formant un câble virtuel	3
VIM	Gestionnaire de l'infrastructure virtualisée (OpenStack, Kubernetes)	8
VLAN	Réseau local virtuel identifié par une étiquette de 12 bits (IEEE 802.1Q)	6
VNF, VNFM	Fonction réseau virtualisée ; gestionnaire de son cycle de vie	4, 8
VNI	Identifiant de segment overlay sur 24 bits	6
VRF	Table de routage indépendante au sein d'un routeur	2, 5
VTEP	Équipement qui encapsule et décapsule le trafic VXLAN	5, 6
VXLAN	Encapsulation de trames Ethernet dans UDP 4789 (RFC 7348)	6
XDP	Point d'accroche eBPF au plus près du pilote de la carte réseau	18
YANG	Langage de modélisation des données de configuration et d'état	8
Zero Trust	Modèle de sécurité sans confiance implicite liée à l'emplacement réseau	4

Annexe C — Références et lectures

Normes de l'IETF et de l'IRTF

RFC	Titre abrégé	Chapitre
RFC 1918	Adresses privées IPv4	1
RFC 4364	VPN IP sur BGP/MPLS	2, 6
RFC 6241	NETCONF	8
RFC 7047	OVSDB	3, 8
RFC 7348	VXLAN	6
RFC 7364, RFC 7365, RFC 8014	NVO3 : problématique, cadre, architecture	7
RFC 7426	Couches et terminologie du SDN	9
RFC 7432, RFC 8365	EVPN, EVPN pour les overlays	7
RFC 7637	NVGRE	6
RFC 7665, RFC 8300	Chaînage de services, en-tête NSH	4
RFC 7938	BGP dans les grands datacenters	5
RFC 7950	YANG 1.1	8
RFC 8040	RESTCONF	8
RFC 8402, RFC 8986	Segment Routing, programmation réseau SRv6	6
RFC 8926	Geneve	6
RFC 9136	Routes de préfixes IP dans EVPN	7
RFC 9315	Réseaux à intentions : concepts et définitions	11

Autres normes et spécifications

- IEEE 802.1Q (ponts et VLAN), 802.1ad (QinQ), 802.1AX (agrégation de liens).
- ONF, *OpenFlow Switch Specification*, versions 1.3.5 et 1.5.1.
- ETSI GS NFV 002 (cadre d'architecture NFV), ETSI GS NFV-MAN 001 (gestion et orchestration), séries IFA et SOL.
- UIT-T Y.3011 (cadre de la virtualisation des réseaux) et Y.3300 (cadre du SDN).
- NIST SP 800-207, *Zero Trust Architecture* (2020).
- ANSSI, *Recommandations relatives à l'administration sécurisée des systèmes d'information* (ANSSI-PA-022).

Documentation des outils

- Documentation d'Open vSwitch et pages de manuel `ovs-vsctl`, `ovs-ofctl`, `ovs-fields`, `ovs-actions`.
- OVN et sa documentation d'architecture (`man ovn-architecture`).
- Documentation d'os-ken ; la documentation de Ryu reste valable pour l'API.
- Mininet, son guide de démarrage et son API Python.
- P4, eBPF, Cilium.
- Datatracker de l'IETF, pour suivre les groupes NVO3, BESS, SPRING.

Lectures

- L. Peterson, C. Cascone, B. O'Connor, T. Vachuska, B. Davie, *Software-Defined Networks: A Systems Approach*, en accès libre : l'ouvrage le plus proche de cette formation, écrit en partie par des auteurs d'ONOS.
- P. Göransson, C. Black, T. Culver, *Software Defined Networks: A Comprehensive Approach*, Morgan Kaufmann, 2e édition, 2016.
- *Network Programmability and Automation*, O'Reilly, 2e édition, 2023 : la démarche NetDevOps en pratique.
- N. McKeown et al., « OpenFlow: Enabling Innovation in Campus Networks », *ACM SIGCOMM Computer Communication Review*, 2008.
- B. Pfaff et al., « The Design and Implementation of Open vSwitch », USENIX NSDI, 2015.
- S. Jain et al., « B4: Experience with a Globally-Deployed Software Defined WAN », ACM SIGCOMM, 2013.
- P. Bosshart et al., « P4: Programming Protocol-Independent Packet Processors », *ACM SIGCOMM Computer Communication Review*, 2014.
- D. Kreutz et al., « Software-Defined Networking: A Comprehensive Survey », *Proceedings of the IEEE*, 2015.

Sources consultées pour l'état des outils

- ONF — annonce d'ONOS 2.7.0 LTS
- Docker Hub — étiquettes de l'image onosproject/onos
- PANTHEON.tech — OpenDaylight Titanium
- Tutoriel OpenFlow avec os-ken